Foλang Programming Language

Foλang is a general-purpose programming language designed to be **expressive, consistent, and extensible**, merging functional fluency with object-centric abstractions.

Normative Status

This document is the authoritative and normative definition of the FoLang programming language.

All FoLang syntax, semantics, type-system rules, name-resolution rules, execution behavior, and other language requirements are defined by this document.

Every addition, correction, clarification, modification, or extension to the FoLang language must be recorded in this document before it becomes part of the language.

A compiler or other language implementation may use any internal architecture, parser, intermediate representation, runtime, or backend. However, to be identified as a conforming FoLang implementation, its externally observable behavior must adhere to the requirements defined in this document.

When an implementation conflicts with this specification, the specification governs unless the discrepancy is formally accepted and incorporated as a specification revision.

Implementation-specific extensions must be clearly identified as extensions and must not be represented as standard FoLang features unless they have been incorporated into this specification.

Design Overview

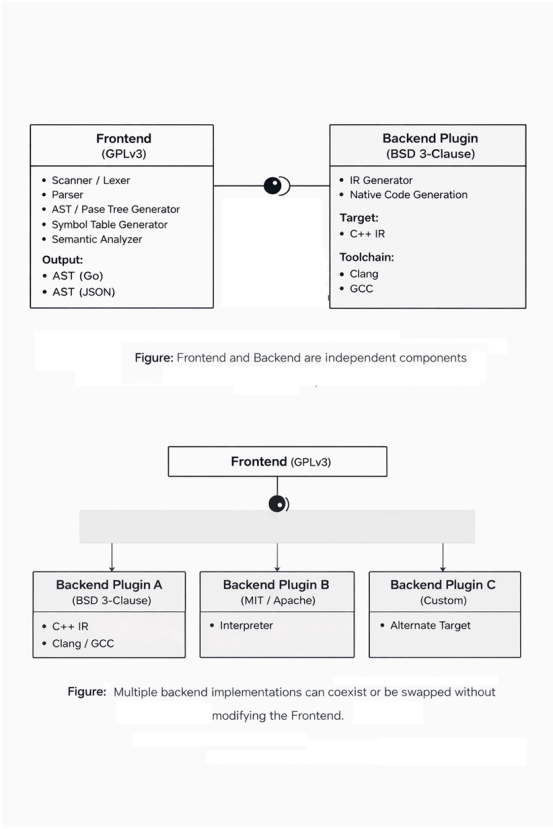


Figure: Frontend and Backend are independent components

Figure: Multiple backend implementations can coexist or be swapped without modifying the Frontend.

FoLang follows a deliberately different approach from conventional programming language designs. The system is structured to ensure **clear separation of concerns**, **license isolation**, and **extensibility through well-defined integration boundaries**.

Compiler and Backend

1. Frontend

The Frontend is responsible for source-level analysis and semantic processing of FoLang programs.

Components

- Scanner / Lexer
- Parser
- AST / Parse Tree Generator
- Symbol Table Generator

- Semantic Analyzer

Implementation

- Implemented in **Go**
- Generates AST representations in **Go structures** or **plain JSON**

License

- **GNU General Public License v3 (GPLv3)**

---

2. Backend

The Backend is responsible for transforming validated frontend output into executable artifacts.

> **Default Backend shouuld be downloaded/build separately. They are not bundled with Frontend Binary**

Components

- Intermediate Representation (IR) Generator
- Native Binary Executable Generation

Implementation

Implemented in any language. The frontend emits HIR over an IPC boundary in the declared wire format. Config declares the full protocol, schema, and wire format.

Default Backend

- Backend orchestration implemented in **Go**
- Code generation target is **C++**
- Uses **Clang** or **GCC** to generate native binaries from generated C++ IR

License

3rd Party Backends can have their own licensing terms and implementation choices. Default backend has the following license. **Default backend is not part of complete compiler binary and is separate** should be downloaded and configured using configuration file.

- **BSD 3-Clause License** ***

Configuration File Structure

Informs the frontend how to generate IR to be consumed by backend. This process is not different for default backend.

```
{
  "protocol":   "folang-plugin/1.0",
  "hir_schema": "folang-hir/1",
  "wire":       "protobuf",
  "output-folder": "<absolute-path>",
}
```

Licensing Summary

Layer	Responsibility	Implementation	License
Frontend	Parsing and semantic analysis	Go	GPLv3
Backend (default)	IR processing and native code generation	Go (orchestration) + C++ (target)	BSD 3-Clause

> The copyrightable material in the FoLang Language Definition and Documentation, including its syntax, grammar, and semantic-rule descriptions, is licensed separately under CC BY 4.0. ***

3. Capability Security Model

FoLang's compiler ships with all language features compiled in but **systems and FFI features are disabled by default**. The compiler has no hardcoded keys — capability configuration happens entirely at install time. This moves authorization from source code (developer-controlled) to the compiler installation (organization-controlled).

Feature Tiers

Tier	Features	Default State
`application`	All standard language features, `co.net`, `co.core`, `co.encoding`, `co.crypto`, etc.	✅ Always enabled
`system`	Raw pointers, pointer arithmetic, `co.sys.unsafe`, MMIO, heap allocators	🔒 Disabled — requires install-time configuration

`ffi`	`@co.dap.native`, `co.sys.ffi`, extern types, `co.lang.void` pointers, C ABI	🔒 Disabled — requires install-time configuration

Quick Start

Hello World

```
// hello.fol — entry file, no annotation needed
co.out.println("Hello FoLang!");
```

Or with an alias for shorter form:

```
@co.ddap.alias(co.out, as="out")

out.println("Hello FoLang!");
```

Variables

```
// typed
name co.lang.string = "Rao";
age  co.lang.int    = 30;

// inferred from value — := errors if already declared
name := "Rao";
age  := 30;

// define infer and assign if not defined, otherwise assing new value
name ?= "Kumar";
```

`co.lang.string` and `co.lang.int` are Builtin Data types to know more about Builtin Data types, plese refer secion Builtin Data Types

`=`, `:=` and `?=` are built in operators to know more about Built in operators please refer section Builtin Operators

Constants and Immutability

FoLang separates two properties that are often confused. One is about *when* a value is known. The other is about *whether* it can change.

```
// compile-time constant — value known while compiling, substitutable
@co.dap.const SIZE co.lang.int = 1024;

// immutable binding — cannot be reassigned, value need not be known early
@co.dap.final startedAt co.lang.int = co.sys.now();
```

Annotation	Guarantees	Value known at compile time	Usable as an index
`@co.dap.const`	value is fixed and known while compiling	✅	✅
`@co.dap.final`	binding cannot be reassigned	❌ not required	❌

Every `@co.dap.const` is also immutable. The reverse does not hold: a `@co.dap.final` binding may be initialised from a function call, a file, or user input, so the compiler cannot substitute a literal for it.

`@co.dap.final` is the declaration-site form of the immutable object kind described in the object mutation policy. `makeImmutable` applies the same property to a value at run time.

Only `@co.dap.const` may name an array size or a dependent type index, because those positions require a value the compiler can substitute. See section Dependent Type Index Rules.

Single Source Application File

FoLang developers can create a complete executable program in one source file. A **single-source application** is an application whose entry file contains the complete program and does not depend on user package source files.

A single-source application file and an application entry file use the same entry-file grammar, context, and restrictions. This section presents the allowed constructs, so a developer can start programming without first reading the complete specification.

Allowed Constructs

The application file may contain:

- built-in directives that are valid for an entry file
- package and library imports
- import aliases declared with `as=`
- file-local aliases for `co.*` paths declared with `@co.ddap.alias`

- type aliases declared with `co.lang.type`
- new types declared with `co.lang.newtype`
- opaque types declared with `co.lang.opaquetype`
- dependent-type aliases and dependent-type usages that do not declare a type-constructor function
- subtype declarations
- supertype declarations
- non-capturing entry-local function-pattern groups
- capturing entry-local `let` function-pattern groups
- variable declarations, initialization, assignment, and mutation
- calls to built-in methods and functions
- calls to imported package and library APIs
- ordinary expressions
- comprehensions
- ternary expressions
- conditions and loops
- containment operations such as `contains`
- iterators such as `each`
- pattern matching and destructuring

```
a co.lang.int = 10;
b co.lang.int = 20;

co.out.println( a + b);
```

> `co` is a keyword/reservedword in `folang` for more details please refer section Reserved Words

> `co` is a built in package in `folang` for more details please check Built in Packages

> a + b is an expression in `folang`

> More about Expression rules please refer section Expressions.

Built-in and Imported Names

All `co.*` paths are always available.

A developer may use the complete built-in path:

```
co.out.println("Hello");
co.core.list.of(1, 2, 3);
```

A developer may optionally create a file-local alias:

```
@co.ddap.alias(co.out, as="out")
@co.ddap.alias(co.core.list, as="list")

out.println("Hello");
values := list.of(1, 2, 3);
```

Creating an alias does not hide the complete `co.*` name; both forms remain valid in that file.

Third party packages User packages and libraries are not automatically available. They must be imported using `@co.ddap.import`. When `as=` is present, the imported API is accessed through that alias. When `as=` is omitted, the complete imported package or library path must be used.

```
@co.ddap.import(package="hr.employee", as="emp")
first := emp.EmployeeService.find(1001);

@co.ddap.import(package="finance.payroll")
second := finance.payroll.calculate(request);
```

A developer needs to import the package to use

> `@co.ddap.import` and `@co.ddap.alias` are built in directives to know more about built in directives please refer section Built-in Directives

> for more details on `folang` import directive please check Import details

> `println` is a built in method in `folang` for more details please check Built in Methods

Variable Kinds

`folang` supports different kind of variables for different purposes, even though language provides, developers cannot use at random places. For more information where they are supported refer section Variable Kind support

Simple Variable Declaration

```
someVar co.lang.int;
someString co.lang.string;
```

With Initialization

```
someBool co.lang.bool = co.const.true;
someInt co.lang.int = 42;
```

With Type Inference

```
someVal := "Hello, World!";
someNum := 3.14;   // if not defined, define and initialize; else throws error
someR ?= "Kamesh"; // if not defined, define and initialize; else reassign
```

Alpha Character and String Literals

The alpha release accepts only the basic literal subset:

```
message := "hello";
letter := 'A';
```

A string is unprefixed, remains on one source line, and cannot contain a backslash or an embedded double quote. A character literal contains exactly one non-backslash character.

Encoding prefixes, escaped characters, raw strings, and universal character names are reserved for a later release. The scanner recognizes the complete spelling and reports an unsupported-feature error instead of tokenizing it as several unrelated expressions. For example, all of these are rejected:

```
x := "quoted: \"text\"";
x := '\n';
x := u8"hello";
x := R"(raw text)";
x := '\N{LATIN CAPITAL LETTER A}';
```

Pointer Declaration

```
somePtr    co.lang.int->(*);
someDblPtr co.lang.int->(**);
someDeepPtr co.lang.int->(*****);
```

The number of consecutive `*` characters is the pointer degree. Any positive degree is permitted; `*` and `**` above are common examples, not a maximum.

Array Declaration

```
someArray       co.lang.int->([5]); // single dimension
someDblArray    co.lang.int->([2,3]); // multi dimension
someJaggedArray co.lang.int->([2][3]); // jagged
someVLArray     co.lang.int->([...]); //varialbe length
someZeroLA      co.lang.int->([0]); //zero length array
someZeroDimA    co.lang.int->([.]); // zerodimension array
```

Array Declaration with Initialization

```
someInitializedArray    co.lang.int->([3])  = [1, 2, 3];
someInitializedArray1   co.lang.int->([])   = [1, 2, 3];
someInitializedDblArray co.lang.int->([,])  = [[1, 2], [3, 4]];
```

Reference Declaration

```
someRef       co.lang.int->(&);    // reference
someLValueRef co.lang.int->(&&);   // LValue reference
someHpRef     co.lang.int->(~);    // heap allocated reference
someAddr      co.lang.int->(@);    // address
someThunk     co.lang.int->(^);    // thunk
someSlice     co.lang.int->([:]);  // slice
```

Range Declaration

```
// Typed range variable declaration
someRange co.lang.int->(..);

// Inferred range declarations
rangeI := 1 .. 10;      // [1, 10]   ExcludeStart=false, ExcludeEnd=false
rangeS := 0 <.. 5;      // (0, 5]    ExcludeStart=true,  ExcludeEnd=false
rangeL := 0 ..< 100;    // [0, 100)  ExcludeStart=false, ExcludeEnd=true
rangeB := 0 <..< 100;   // (0, 100)  ExcludeStart=true,  ExcludeEnd=true
rangeE := .. 100;      // open lower bound  (_, 100]
rangeF := 1 ..;        // open upper bound  [1, _)
```

Auto and Dynamic Variable Declaration

```
someAutoVar    co.lang.auto    = "Hello"; // type inferred from value; initialization required
someDynamicVar co.lang.dynamic;           // dynamic typing
```

Lazy

```
@co.dap.lazy
x = add(1, 2);  //on doing some thing on x calls add(1,2) till that time add function on right hand side is not invoked
```

Bind Variables

`$[0-9]*`

Discard / Wildcard Variable

`_`

Comma and Grouping

```
// Comma
x co.lang.int = 10, y co.lang.string = "Hello", z co.lang.bool = co.const.true;

// Grouping
(x co.lang.int = 10, y co.lang.string = "Hello", z co.lang.bool = co.const.true);

// Tuple/grouping expression
pair := (x, y);
```

A comma inside a parenthesized expression or grouped declaration must introduce another expression or declarator. Therefore `(x,)`, `(x, y,)`, and `(x co.lang.int = 10,)` are invalid. Collection literals have their own rules and may permit a trailing comma where their production says so. Record patterns also require another field after every comma; `Employee{id: value,}` is invalid.

Fat Pointers

```
x co.lang.int->(*, kind="", meta={});

co.lang.int->(*, meta={});

y co.lang.int->(*, meta={len:co.lang.usize, vtab:somepkg.VTable->(*)});

z co.lang.int->(*,kind=region, meta={});
```

```
Pointer
├── base_type: T
├── kind: <FatKind>
│    ├── thin
│    ├── slice
│    ├── relative
│    ├── trait
│    ├── buffer
│    ├── view
│    ├── opaque
│    ├── custom
│    ├── mem
│    ├── nullptr
│    ├── sptr
│    ├── uptr
│    ├── ptrdiff
│    ├── usize
│    ├── ssize
│    └── (region)  ← optional syntactic sugar
└── meta:
     ├── region: heap | stack | global | numa(N) | mmio | constant | …
     ├── len, cap, vtab, bits, endian, …
```

Pointers for address manipulation

```
y co.lang.word->(repr=intptr);
z co.lang.word->(sign=unsigned, repr=uintptr);
p co.lang.word->(repr=ptrdiff);
n co.lang.word->(sign=unsigned, repr=usize);
m co.lang.word->(repr=isize);
o co.lang.void->(repr=nullptr);
```

Relative Pointers

```
z co.lang.int->(*,kind=relative, meta={});
```

Note Other than normal variable declaration rest have restrictions

Conditions, Loops and Iterators

Conditions

```
(boolean truth).do({
}).otherwise(boolean truth).do({
}).otherwise.do({
});
```

Loops

```
(boolean truth).loop({
}).otherwise(boolean truth).loop({
}).otherwise.loop({
});
```

Condition and Loop Mix

```
(boolean truth).do({
}).otherwise(boolean truth).loop({
}).otherwise(boolean truth).do({
}).otherwise.loop({
});
```

Ternary Operator

```
s = (boolean truth).return(some var/value).otherwise.return(some val/var);
s = (boolean truth).return(some var/val).otherwise(boolean truth).return(some var/val).otherwise.return(some var/val);
```

Looping Arrays / Lists / Maps / Ranges

```
arr co.lang.int->([5]) = [6,7,8,9,10];
arr.each(idx, val).do({
    co.out.print(idx);
    co.out.print(" :: ");
    co.out.println(val);
});

arr.each(_, val).do({
    co.out.println(val);
});
```

Array / List / Map / Range — Contains Element

```
arr co.lang.int->([5]) = [35,57,96,81,31];
k co.lang.int = 31;
arr.contains(k).do({
    co.out.println(k);
}).otherwise.do({
    co.out.println("Not Found");
});
```

Comprehensions *(planned)*

```
k := (1 .. 10).filter(|x| => x % 2 == 0).map(|x| => x * x);

result := for (x <- List(1,2,3)).yield(x * 2);         // List(2, 4, 6)
result := for (x <- Set(1,2,3)).yield(x * 2);          // Set(2, 4, 6)
result := for (x <- Some(5)).yield(x * 2);             // Some(10)
result := for (x <- fetchData()).yield(x.process());   // Future

ages  := {"A":30,"B":40,"c":66,"e":88};
upper := for ((name, age) <- ages).yield(name.toUpperCase, age);
```

Pattern Matching

```
x co.lang.int = 10;

x.match.case(n: n > 10 => { n = n+100; "GT" }).case( n: n < 10 => "LT").default("EQ");
x.match.case(n: n > 10 => { n = n+100; "GT" }).case( n: n < 10 => "LT").case(_=>"EQ");
x.match(co.pattern.Type).case(co.lang.int => ...).case(co.lang.float => ...);
x.match(co.pattern.Value).case(0 => ...).case(1 => ...);
x.match(co.pattern.Instance).case(xx.CAT => ...).case(xx.DOG => ...).default("Animal");
x.match(co.pattern.Object).case(xx.Ball => "Ball").case(xx.CAT => "CAT").default("Unknown");
x.match(co.pattern.Shape).case(Point{x, y} => ...).default(...);

x.match(co.pattern.Any).case(co.lang.int => ...).case(co.lang.float => ...).case(0 => ...).default( ...);

x.match(PositiveEvenMatcher).case(0 => "Neither even nor odd").case(2 => "First Even Prime").default(...);
```

A match chain contains one or more `.case(...)` arms followed by at most one terminal `.default(...)` arm. `.otherwise` is not a match arm; it belongs to conditional and ternary return chains. A case or default result may itself be a ternary return expression, and that nested expression may consequently contain `.otherwise.return(...)`.

> **Object vs Instance in FoLang:** Instance is from types of class/structs. Objects are anything — functions, classes, structs, types, etc.

> `_` is a special discard/wildcard variable. In a call it is permitted only as the first, index/key argument of a receiver-qualified `.each`, as in `items.each(_, value)`. Transparent grouping around the member callee, for example `(items.each)(_, value)`, does not change this rule. The value argument of `each` cannot be discarded, and `contains(_)` and `containsVal(_)` are invalid because containment must compare a real value. Patterns and the filename-derived declaration-name form give `_` their own explicitly described meanings; it is not a general expression identifier.

> `PostiveEvenMatcher` is custom matcher for more details about creating custom matcher please refer to section Custom Matcher

Type Declarations

```
// Alias
x co.lang.type = co.lang.int;

// New
x co.lang.newtype = co.lang.int;

// Opaque
x co.lang.opaquetype = co.lang.int;  //opaque types act like alias with base type from which they declared and act like distinct type (new types) for others even when two opaque types derived from same base types

// ADT (tagged union)
y co.lang.type = co.lang.int | co.lang.char;

// Subtype / covariant
test co.lang.subtype = co.lang.int;

// Supertype / contravariant
test co.lang.supertype = co.lang.int;
```

Let Bindings

```
y co.lang.int = let({x = 10}).in({x + 1});
y co.lang.int = let({$ = 10}).in({$ + 1});  // $ refers to the value being defined

x co.lang.int = (x + 1).where(x = 10);
x co.lang.int = ($ + 1).where($ = 10);

offset := 100;

let adjust(0) = offset;
let adjust(n) = n + offset;
```

> `$` is a special identifier usable in ordinary `let` binding expressions for recursive or self-referential expressions.
>
> Ordinary `let` value-binding expressions remain available in language contexts that permit them, but they are forbidden directly in the application entry file. In the entry file, `let` is reserved exclusively for a named function-pattern group that captures at least one surrounding runtime binding. It cannot introduce an anonymous function, a general closure value, or a curried function.

Function Pattern

```
f(Some(x)) => { x + 1 }
f(None())  => { 0 }

// desugars to:
f(v co.lang.type)->(co.lang.int) = {
    this.return v.match()
        .case(Some(x) => x + 1)
        .case(None() => 0);
}
```

`=>` introduces a bare function-pattern clause. `=>>` is the distinct function-delegation operator and `==>>` is the clousre/curry expression and does not introduce a function pattern.

Function-pattern groups are permitted in the application entry file as restricted entry-local dispatch helpers. A bare group cannot capture surrounding runtime variables. A `let` function-pattern group must capture at least one already initialized entry-file runtime binding and is the only entry-file construct that permits such capture. Neither form permits ordinary function declarations, anonymous functions, general closure values, currying, partial application, or escape as a function value.

> More about Let and function patterns please refer section Let and Function Patterns

> Single source application file is for testing and getting feel of `folang` Real world applications contain more than single source application file they use concepts of abstraction, encapsulation, inheritance and polymorphism at the core with many other features. Also these applications depend on external libraries, packages to achive clear boundaries with above features. To develop these kind of applications we need following features at minimum.

1. package source files under packages
2. Entry File
3. Libraries and Library surface files
4. imports

Foλang Supports many features to develop enterprise application with intent and FoLang Philosophy — Uniform Object Model are listed below

Complete Feature list:

In `folang` User defined data types, function, macros, extensions, templates, typeclasses, type constructors, and units must be in its own file and under a package these files are called package source files. , Lets discuss packages before going to UDTs and Functions

Packages

Package Identity

A subfolder containing `.fol` files **is** a package.

- Dot paths start from subfolders.
- The project root is **not** a package.
- The root folder name never appears in any package dot path.
- The folder configured by `operator_library_folder` is a compiler-controlled operator-source area and is excluded from ordinary package discovery even though it contains `operators.fol`.

Examples:

```
/appl/hr/           -> package "hr"
/appl/hr/employee/  -> package "hr.employee"
/appl/auth/         -> package "auth"
```

The project root itself is **not** a package.

Multi-File Packages

Multiple `.fol` files in the same subfolder automatically belong to the same package:

```
hr/employee/
├── Employee.fol      →  hr.employee
├── EmpService.fol    →  hr.employee
└── EmpValidator.fol  →  hr.employee
```

Application Project Layout

```
/appl/
├── app.fol                     ←  entry file — not a package
├── hr/                         package "hr"
│   ├── employee/               package "hr.employee"
│   │   ├── Employee.fol
│   │   └── EmpService.fol
│   └── payroll/                package "hr.payroll"
│       ├── Payroll.fol
│       └── PayrollCalc.fol
├── auth/                       package "auth"
│   └── Auth.fol
└── bindings/                   package "bindings"
    └── CLib.fol
```

Package Aliasing

If there is a folder /appl/hr/empl and under that there is a fol file called Employee.fol then the import statement as we know will be

`@co.ddap.import(package="hr.empl", as="emp")` where `as` is not a mandatory attribute

> An import names a **package**, never a declaration inside it. The package is the folder, so `Employee.fol` under `/appl/hr/empl` belongs to package `hr.empl`; the file name is not part of the path. Once the package is imported, the declaration is reached as `emp.Employee`.

Now we want to change empl to emp, simple way is `change the folder name`, but we want to keep the `physical folder name` as is.

For example /appl/hr/empl shoud be named as hr.emp instead of hr.empl

```
    emp co.lang.package;
```

> The single line should be put in `package.fol` under `/appl/hr/empl`, and any normal fol cgitkode must not use this name to the file it is a `restricted name` for a file in `folang`.

The import will be as below,

`@co.ddap.import(package="hr.emp", as="emp")`

> Note: This is a **Planned** Feature not finalized to be part of initial release.

UDT (User defined Data types)

`folang` provides following constructs to create User Defined Data types, as mentioned UDTs must be in their own source file under package.

1. cstructs
2. structs
3. unionns
4. enums
5. classes
6. Modules
7. interface
8. signature

> For more information about UDTs please refer section Built In Kinds ***

Struct Declaration

```
myStruct co.lang.struct={
    field1 co.lang.int;
    field2 co.lang.string;
    field3 co.lang.bool;
}
```

> More about structs please refer section `Structs in detail`

C-Struct Declaration

`co.lang.cstruct` is a C-like value type — passed by value, simple memory layout, safe to cross zone boundaries. Unlike `co.lang.struct` which is passed by reference, `co.lang.cstruct` is always copied on pass.

```
Point co.lang.cstruct = {
    x co.lang.int;
    y co.lang.int;
}

Rect co.lang.cstruct = {
    origin Point;
    width  co.lang.int;
    height co.lang.int;
}
```

Enum Declaration

```
myEnum co.lang.enum={
    Variant1,
    Variant2,
    Variant3
}
```

Union Declaration

```
myUnion co.lang.union={
    intValue co.lang.int;
    strValue co.lang.string;
}
```

Class Declaration

```
Employee co.lang.class ={
    getEmployeeDetails()->(Employee) = empmodule.getEmployeeDetails;
    // assigning module function to class's method

    getEmployeeInfo()->(Employee) =>> empmodule.getEmployeeDetails();
    // delegating — internally redirecting the call to module function
}

// $1, $2, $3 ... are previous results captured as bind variables
Emp co.lang.class={
    dosomething(a co.lang.int, b co.lang.int)->(co.lang.int)=>>somePack.someMethod(a)=>>someOthPack.someOtherMeth($1, b);
}
```

> More about classes please refer section `Classes in detail` ***

Interface vs Signature

```
// Employee is an ordinary package-level declaration.
MEmployee co.lang.signature = {
    ...
}
```

> For more about signatures please refer section `signatures in detail`

```
IEmployee co.lang.interface = {
  ...
}
```

> For more about interfaces please refer section `interfaces in detail`

Module Declaration

```
// Employee.fol — ordinary package-level type
Employee co.lang.struct = {
  ...
}

// EmployeeModule.signature.fol
EmployeeModule co.lang.signature = {
  ...
}

// EmployeeModImpl.fol
@co.dap.module(signature=EmployeeModule)
EmployeeModImpl co.lang.module->(signature=EmployeeModule, matches=EmployeeModule) = {

   ...
}
```

> More about modules please refer section `Modules in detail` ***

Units

Unit Declaration 🟩

A `unit` is a named, non-instantiable container for functions.

Like other named primary declarations, a unit may use an explicit name or `_` for filename-derived naming. For example, `_ co.lang.unit` in `Math.fol` declares `Math`, while an explicit name such as `Math co.lang.unit` is authoritative regardless of the filename.

Its purpose is structural: it prevents functions from flowing freely at package-file scope and preserves FoLang's rule that every ordinary package source file has one primary top-level declaration. A unit does **not** require its functions to form a domain model, capability, service, or other semantic abstraction.

Functions within a unit may be strongly related by purpose—for example, mathematical, parsing, encoding, or validation functions—or only loosely related. Semantic cohesion is encouraged as a source-design practice but is not enforced by the language.

A unit has two forms:

1. **Standalone unit** — its name does not match a struct in the same package. It acts as an ordinary named container for receiverless functions.
2. **Struct companion unit** — its name matches exactly one `co.lang.struct` in the same package. It provides behaviour associated with that struct while the struct declaration itself remains pure data.

Only `co.lang.struct` can have a companion unit. A same-name unit is not allowed for `co.lang.class`, `co.lang.cstruct`, modules, enums, unions, interfaces, signatures, or other declaration kinds. Classes already contain their own methods and lifecycle behaviour; `cstruct` remains a restricted C-compatible data representation.

```
Text co.lang.unit={

}
```

> for more about units please refer section `units in detail`

Matchers

Custom Matcher

```
@co.dap.matcher
Matcher(T) = {
    matchCase(value T, pattern co.lang.untyped) -> (co.lang.int, co.lang.MatchBindings);
    // int return: 0 = no match, >0 = match
}

PositiveEvenMatcher co.lang.matcher->(for=Matcher, type=co.lang.int) = {
    matchCase(value co.lang.int, pat co.lang.untyped)->(co.lang.int, co.lang.MatchBindings) = {
        // user logic
    }
}
```

Comprehensions *(planned)*

```
k := (1 .. 10).filter(|x| => x % 2 == 0).map(|x| => x * x);

result := for (x <- List(1,2,3)).yield(x * 2);          // List(2, 4, 6)
result := for (x <- Set(1,2,3)).yield(x * 2);           // Set(2, 4, 6)
result := for (x <- Some(5)).yield(x * 2);              // Some(10)
result := for (x <- fetchData()).yield(x.process());    // Future

ages  := {"A":30,"B":40,"c":66,"e":88};
upper := for ((name, age) <- ages).yield(name.toUpperCase, age);
```

Extensions

```
stringextension co.lang.unit={

    @co.dap.extension(fortype=co.lang.string, what=extends)
    upperCase()->(string)={
        return this.upper();
    }

    @co.dap.extension(fortype=[co.lang.string], what=overrides)
    equals(str string)->(bool)={
        this.return this == str;
    }
}
```

Extensions must be **explicitly activated** — they are block-scoped:

```
// same package as stringextension — a bare name is enough
@co.ddap.use(from="stringextension", methods=[equals, upperCase])
k.upperCase();  // ✅ explicitly activated
```

From another package the unit is qualified, by alias or by full package path:

```
@co.ddap.import(package="text.util", as="tu")

@co.ddap.use(from="tu.stringextension", methods=[upperCase])
@co.ddap.use(from="text.util.stringextension", methods=[upperCase])
```

See Activating Instance Methods for the full set of `from` forms and how activation interacts with typeclass instances.

Reflections

```
@co.dap.reflection(enable=True, package="co.meta")

x co.lang.int = 10;
x.reflect().getType();   //co.lang.int
x.reflect().getValue();  //10;
x.reflect().getKind();   // value
```

Type Classes

Monads, Applicatives, Functors, Monoids and Transformers

> `@co.dap.typeclass(kind=...)` is the single annotation for all typeclass definitions. `kind` specifies the algebraic structure — `Functor`, `Applicative`, `Monad`, `Monoid`, `Transformer`, or any user-defined kind. Instances of any typeclass always use `co.lang.instance`.

Functor

```
@co.dap.typeclass(kind=Functor)
Functor(F) = {
    map(value F(A), f (A)->B) -> (F(B));
}

ListFunctor co.lang.instance->(for=Functor, type=List) = {
    map(value List(A), f (A)->B)->(List(B)) = {
        result = List(B){};
        value.each(_, item).do({ result.append(f(item)) });
        this.return result;
    }
}
```

Applicative

```
@co.dap.typeclass(kind=Applicative)
Applicative(F) = {
    pure(x A) -> (F(A));
    apply(fab F(A->B), fa F(A)) -> (F(B));
}

OptionApplicative co.lang.instance->(for=Applicative, type=Option) = {
    pure(x A)->(Option(A)) = { this.return Some(x); }
    apply(fab Option(A->B), fa Option(A))->(Option(B)) = {
        this.return (fab, fa)
            .match
            .case((Some(f), Some(x)) => Some(f(x)))
            .default(None());
    }
}
```

Monad

```
@co.dap.typeclass(kind=Monad)
Monad(F) = {
    pure(x A) -> (F(A));
    flatMap(fa F(A), f (A)->F(B)) -> (F(B));
}

OptionMonad co.lang.instance->(for=Monad, type=Option) = {
    pure(x A)->(Option(A)) = { this.return Some(x); }
    flatMap(fa Option(A), f (A)->Option(B))->(Option(B)) = {
        this.return fa.match().case(Some(x) => f(x)).default(None);
    }
}
```

Monoid

```
@co.dap.typeclass(kind=Monoid)
Monoid(T) = {
    empty() -> (T);
    combine(a T, b T) -> (T);
}

IntMonoid co.lang.instance->(for=Monoid, type=co.lang.int) = {
    empty()->(co.lang.int) = { this.return 0; }
    combine(a co.lang.int, b co.lang.int)->(co.lang.int) = { this.return a + b; }
}
```

Transformer

```
@co.dap.typeclass(kind=Transformer)
Transformer(F(_), G(_)) = {
    map(value F(A), f (A)->B) -> (G(B));
}

ListToSetTransformer co.lang.instance->(for=Transformer, types=[List, Set]) = {
    map(value List(A), f (A)->B)->(Set(B)) = {
        result = Set(B){};
        value.each(_, item).do({ result.insert(f(item)) });
        this.return result;
    }
}
```

Using an Instance

An instance is selected **by name**. There is no implicit search.

```
@co.ddap.import(package="abc.tc", as="tc")

xs List(co.lang.int) = [1, 2, 3];
double(x co.lang.int)->(co.lang.int) = { this.return x * 2; }

ys := tc.ListFunctor.map(xs, double);
```

`map` takes the container as its first argument, exactly as the typeclass declares it. The call names `ListFunctor`, so the compiler resolves it the same way it resolves any other imported declaration.

FoLang does **not** search visible packages for an instance that happens to match a type. A typeclass is a contract, an instance is a named implementation of that contract, and the caller names the one it means. Nothing is inferred, so an unresolved call reports a name the developer actually wrote rather than a failed search they never saw.

> Method syntax such as `xs.map(f)` is unrelated to typeclasses. It calls an associated function declared in a companion unit for the type. Companion units live in the type's own package, so method calls are unambiguous by construction. A companion function and a typeclass instance may both be named `map`; they are different declarations and are reached differently.

Activating Instance Methods

An instance may also be **activated**, which makes its functions callable as methods on the receiver. Activation uses the same directive that activates extensions.

```
@co.ddap.import(package="abc.tc", as="tc")
@co.ddap.use(from="tc.ListFunctor", methods=[map, reduce])

ys := xs.map(double);        // resolves to tc.ListFunctor.map(xs, double)
```

Activation is explicit and block-scoped. Importing a package does **not** activate anything in it, so adding an import can never change how an existing call resolves.

`methods` covers both

`methods` is the only list attribute. It activates named functions whatever `from` resolves to — an extension unit or a typeclass instance.

An extension already declares what it extends, on the method itself:

```
@co.dap.extension(fortype=co.lang.string, what=extends)
upperCase()->(string)={ ... }
```

so the activation site has nothing to add. `from` already says which source is being drawn from, and the compiler knows what that source is.

```
@co.ddap.use(from="tu.stringextension", methods=[upperCase]);   // extension unit
@co.ddap.use(from="tc.ListFunctor", methods=[map, reduce]);     // typeclass instance
```

Listing names is optional. Omit the list to activate everything the source provides; give a list to activate a subset. A subset is how conflicts are resolved — take `map` from one instance and `reduce` from another.

What `from` accepts

`from` names a **declaration**, so it carries a unit or instance name. This is deliberately unlike `package`, which names a package and nothing else.

```
from="stringextension"              // bare — same package
from="tc.ListFunctor"               // alias + instance
from="ext.stringextension"          // alias + unit
from="abc.tc.ListFunctor"           // full package + instance
from="abc.ext.stringextension"      // full package + unit
```

```
@co.ddap.import(package="hr.empl", as="emp")            // package only
@co.ddap.use(from="emp.EmpExtensions", methods=[...])    // package + declaration
```

How a method call resolves

For `xs.map(f)`, where `xs` has type `List(A)`:

1. a class method or companion-unit function on `List`
2. an activated extension for `List`
3. an activated instance function whose typeclass declares `map` with the receiver as its first parameter
4. otherwise, an error

The first match wins. A type's own declarations therefore always take precedence over anything activated into scope, and no activation can silently replace behaviour the type already defines.

Within one scope a given method name may be activated at most once for a given receiver type. Activating `map` for `List` from two sources is an error at the second `@co.ddap.use`, which names both. The conflict is reported where the activation is written, never at a distant call site.

The parser's built-in-method classification is only a lookup candidate. If the frontend recognizes a control-chain shape before receiver-aware lookup, it must retain the complete original member/call chain. A receiver-owned, companion, extension, or activated-instance declaration that wins the order above restores/remains the ordinary method call represented by that chain. An early dedicated control-flow node is therefore only a lowering candidate and becomes final only when the built-in meaning wins.

Activation never affects generic code. A function polymorphic over a typeclass takes the instance as an ordinary parameter and calls it by name.

A Typeclass Is a Type

A typeclass may be used wherever a type is expected. Its values are its instances. This is the same relationship a signature has to a module:

```
mm EmployeeModule = EmployeeModImpl;     // signature as type, module as value
f  Functor(List)  = tc.ListFunctor;      // typeclass as type, instance as value
```

An instance is therefore an ordinary first-class value. It can be held in a variable, stored in a collection, returned from a function, and chosen at run time.

```
inst := (useCache).return(cachedFunctor).otherwise.return(plainFunctor);
result := inst.map(xs, transform);
```

Nothing about an instance is special to the compiler, which is why FoLang needs no instance resolution algorithm and no coherence rules.

Writing Code Generic Over a Typeclass

Activation makes `xs.map(f)` work for a **known** container. Code that must work for **any** container cannot activate anything, because the container type is not known until the caller supplies it. Such code takes the instance as an ordinary parameter.

```
@co.dap.generic(type={F:{typename}, A:{typename}, B:{typename}})
mapAll(inst Functor(F), value F(A), fn (A)->B)->(F(B)) = {
    this.return inst.map(value, fn);
}
```

Parameter	What it is
`F`	the container kind — `List`, `Option`, `Tree`
`inst`	the instance, an implementation of `Functor` for `F`
`value`	the container itself, an ordinary value

The caller supplies the instance:

```
ys   := mapAll(tc.ListFunctor,   xs,  double);
opt2 := mapAll(tc.OptionFunctor, opt, double);
```

The function never learns what `F` is. It knows only that `inst` provides `map`, which is the whole contract. One definition therefore serves every container that has a `Functor` instance.

> A type is never "a Functor" in FoLang. `List` does not become a Functor; an instance implements Functor operations *for* `List`, and the list stays a plain list. The Functor-ness lives entirely in `inst`.

When a wrapper is worth writing

`mapAll` above forwards directly to `inst.map`, so it adds nothing a caller could not write themselves. A wrapper earns its place when it does more than forward — when it combines several typeclass operations, adds logic, or fixes some parameters and leaves others open.

```
@co.dap.generic(type={F:{typename}})
doubleAll(inst Functor(F), value F(co.lang.int))->(F(co.lang.int)) = {
    this.return inst.map(value, (x co.lang.int)->(co.lang.int) = {
        this.return x * 2;
    });
}
```

This one fixes the element type and the operation, so it says something a bare `inst.map` call does not.

Where an Instance Is Declared

An instance is declared in **the package that defines the typeclass**, or in **the package that defines the type**. That exact package, not a sub-package.

```
abc.tc.ListFunctor      for=Functor, type=List           // OK  typeclass's package
myapp.ab.TreeFunctor    for=Functor, type=myapp.ab.Tree  // OK  type's package
other.util.ListFunctor  for=Functor, type=List           // ERR neither is theirs
```

A typeclass is an ordinary declaration and may live in any package; `abc.tc` above is a user package, not a built-in one. Sub-packages are distinct packages, so an instance for `myapp.ab.Tree` belongs in `myapp.ab`, not in `myapp` or `myapp.ab.instances`. This matches the rule for companion units, which also sit in their type's own package. Because a package spans every `.fol` file in its folder, each instance still gets its own file.

The rule is permissive on both sides on purpose. Requiring the typeclass's package alone would make a typeclass usable only by its own author, since nobody could implement it for their own types. Requiring the type's package alone would stop a library from shipping instances for common built-in types.

For library authors. If you define a type and want it usable with someone else's typeclass, declare the instance in your package. If you define a typeclass, you may ship instances for types you do not own, including built-in ones — `IntMonoid` above sits beside `Monoid`, which is exactly this case. If you need an instance for a typeclass and a type you both do not own, wrap the type in a `co.lang.newtype` you do own and declare the instance for the wrapper.

This placement rule is semantic, not syntactic. A misplaced instance parses correctly and is reported during name resolution, so the diagnostic can name the typeclass, the type, and the two packages in which the instance would have been legal.

Reflection

```
@co.dap.reflection(enable=True, package="co.meta")

x co.lang.int = 10;
x.reflect().getType();   // co.lang.int
x.reflect().getValue();  // 10
x.reflect().getKind();   // value
```

Labels and Named Blocks

```


// Label
outer:{
    // statements
}

//Blocks Anonymous block
 {
    // statements
 }

// Named Block
labelBlock co.lang.block={

}

labelBlock.expand();
```

> Blocks have their own scope and context for variables, a variable pre declarred outside the block will be accessible in side the block, a block can have its own variable with same name and different type or same type which overrides/shadows parent or outer blocks variables, and the scope of such variables are limited to that block it is very similar to C/C++

> Blocks cannot live outside functions they must be inside functions or methods only

> Inner blocks for class, struct, typeclass, module or anyother consttruct other than functions/methods are prohibited. // throws compiler error

```
somefun (a co.lang.int, b co.lang.int)->(co.lang.int)={

     some_other co.lang.float =  20.1f;
     {
        some_other co.lang.char = 'c';
        co.out.println( some_other);   //prints c
     }

     co.out.println(some_other); //prints 20.1;

     {
        some_other co.lang.float = 11.1f;
        co.out.println(some_other); // prints 11.1f
     }

     co.out.println(some_other); // still prints 20.1f;

     {
        some_other = some_other + 1.1f;
        co.out.println(some_other); // prints 21.2
     }

     co.out.println(some_other); // prints 21.2; as this was changed in third block
}
```

---

imports

FoLang supports three import forms. User packages and libraries must be imported before use. When an import declares `as=`, symbols are accessed through that alias. When `as=` is omitted, the complete imported package or library path is used.

1. Normal Package Import

Use this for ordinary project packages.

```
@co.ddap.import(package="hr.employee", as="emp")

e := emp.getEmployee(1);
co.out.println(e.name);
```

Resolution:

```
package="hr.employee" -> /appl/hr/employee/
```

2. Source Library Import

Use this for same-owner workspace libraries whose source is available.

```
@co.ddap.import(package="com.abc.ffi", src-library=true, expect="ffi", as="ffilib")
```

Resolution:

```
package="com.abc.ffi", src-library=true -> /appl/com/abc/ffi.fol
```

The resolved file must be a library surface file:

```
@co.dap.library(type="ffi")
ffilib co.lang.library={

}
```

Meaning:

- `package` gives the logical library path
- `src-library=true` means the leaf resolves to a single `.fol` surface file, not a folder
- `expect` is an import-site assertion; the compiler checks it against the actual library type
- only the projected surface API is visible; internal sources are not importable through normal package imports

3. Packaged Library Import

Use this for third-party or prebuilt libraries.

```
@co.ddap.import(library="hrlib", as="hr")
@co.ddap.import(library="paylib", as="pay")
```

Resolution:

```
library="hrlib" -> libs/hrlib.folenc, else libs/hrlib.folib
```

Only the packaged library's projected surface API is visible to the consumer.

Import Directive Fields

Field	Required	Default	Meaning
`package` or `library`	one required	—	logical package path or packaged library name
`src-library`	❌	`false`	when `true`, `package=` resolves to a source library surface file
`expect`	❌	inferred from library surface	expected library kind such as `ffi`, `system`, `advanced`, `dynamicvmrt`, or `application`
`as`	❌	none — full dot path required when omitted	local alias; valid FoLang identifier
`realm`	❌	`app`	isolation domain
`parent-realm`	❌	—	realm shadowing relationship

Notes:

- `as` is optional — when omitted, no short alias is created and the full imported package path must be used to access symbols
- dots are not allowed in `as`
- `expect` is validation, not the source of truth
- the source of truth is `@co.dap.library(type="...")` on the resolved surface file

Examples:

```
@co.ddap.import(package="hr.employee", as="emp")
em emp.Employee;

@co.ddap.import(package="hr.employee")
em hr.employee.Employee;
```

Valid `as` Values

```
as="hr"       ✅
as="v1_hr"    ✅
as="v1.hr"    ❌
as="123hr"    ❌
```

Realms

Realms provide import isolation, coexistence of versions, and controlled shadowing.

Realm declarations are always syntactically valid — you can always write `realm=` on any import. However realm isolation is **active only when the library marked with `dynamicvmrt`**. Without it, realm declarations are not valid and compiler will throw error.

Default realm:

```
app
```

Hierarchy example:

```
core
  └── app
        ├── app1
        │     └── app3
        └── app2
```

Rules:

- `realm` defaults to `app`
- if `parent-realm` is omitted, parent defaults to `app`
- `parent-realm` is meaningful only when `realm` is explicitly provided and is not `app`
- libraries are always static
- `@co.ddap.dynamicruntime` is allowed only on the application entry file

Version Coexistence

```
@co.ddap.import(package="hr", realm="app", as="hr")
@co.ddap.import(package="v1.hr", realm="x", as="v1_hr")
```

Shadowing

```
@co.ddap.import(package="a", realm="app", as="hr")
@co.ddap.import(package="a", realm="x", parent-realm="app", as="hr")
```

When the alias is the same and the child realm points to the parent realm, the child shadows the parent.

Import Binding Rules

Invalid

Same alias, same realm, different package:

```
@co.ddap.import(package="a", realm="app", as="hr")
@co.ddap.import(package="b", realm="app", as="hr")  // error
```

Valid aggregation

```
@co.ddap.import(package="folderx.some", realm="app", as="hr")
@co.ddap.import(package="folderx.some", realm="app", as="hr")
```

Valid coexistence

```
@co.ddap.import(package="a", realm="app", as="hr")
@co.ddap.import(package="a", realm="x", as="v1_hr")
```

Valid shadowing

```
@co.ddap.import(package="a", realm="app", as="hr")
@co.ddap.import(package="a", realm="x", parent-realm="app", as="hr")
```

Cycles

Compiler error if any cycle exists through:

- package imports
- realm parent relationships

Examples:

- `packageA` imports `packageB`, and `packageB` imports `packageA`
- `realm="x", parent-realm="y"` and another import uses `realm="y", parent-realm="x"`

Symbol Resolution

Resolution Order

For symbols without a prefix:

1. current package symbol table
2. parent package chain upward
3. error if not found

For symbols with an alias prefix:

1. current package imported-context map
2. parent package chain imported-context maps
3. error if not found

For imports declared **without** `as`:

1. use the full imported package path directly
2. resolve it through imported-context lookup using that full path
3. error if not found

Example Context Graph

```
pp1 (grandparent package)
    ├── SymbolTable
    ├── ImportedContexts
    └── ChildCtxs

p1 (parent package)
    ├── Parent -> &pp1
    ├── SymbolTable
    ├── ImportedContexts
    └── ChildCtxs

c1 (current package)
    ├── Parent -> &p1
    ├── SymbolTable
    ├── ImportedContexts
    └── ChildCtxs
```

Resolution Examples

```
lookup "OwnType"
    -> current symbol table

lookup "ParentType"
    -> parent chain

lookup "hr.Employee"
    -> imported-context lookup for alias "hr"

lookup "hr.employee.Employee"
    -> imported-context lookup for full imported package path when no alias exists

lookup "unknown.Type"
    -> imported-context lookup fails -> compiler error
```

Short Summary

- folders define packages

- root is never a package
- each ordinary package source file has exactly one primary top-level declaration
- free functions in package source files must be enclosed in a `co.lang.unit`
- the application entry file is an executable, non-package context with its own restricted declaration rules
- entry-local type aliases, newtypes, opaque types, dependent-type aliases/usages, subtypes, supertypes, bare function-pattern groups, and capturing `let` function-pattern groups are allowed
- ordinary `let` value bindings, ordinary functions, anonymous functions, general closures, classes, structs, enums, cstructs, type constructors, generics, macros, templates, units, and reusable behavioral declarations are forbidden in the entry file
- `co.*` is always available and never imported
- `@co.ddap.alias` optionally shortens a `co.*` path; otherwise the complete path is used
- `@co.ddap.import(package="...")` imports normal packages
- `@co.ddap.import(package="...", src-library=true, ...)` imports same-owner source libraries
- `@co.ddap.import(library="...")` imports packaged external libraries
- `expect="..."` is an import-site assertion, not the source of truth
- `@co.dap.library(type="...")` is the source of truth for library kind
- every library surface exports only boundary `struct` / `cstruct` contracts and public function signatures
- surface function bodies are restricted boundary adapters and are hidden from consumer symbol tables
- application-family boundary structs cross by automatic deep snapshot
- system and FFI boundary cstructs cross by ABI value
- internal packages never depend on surface types; the surface converts between public and internal representations

---

Let and Function Patterns

Bare Function-Pattern Group

A bare function-pattern group does not capture surrounding runtime bindings:

```
classify(0) => { "zero" }
classify(n).where(n > 0) => { "positive" }
classify(_) => { "negative" }
```

The formal clause shape is:

```
name(patterns) [.where(guard)] => result
```

`=>` is mandatory. `=>>` belongs to ordinary function delegation, `==>>` belongs to closure/curry expression and are not accepted here.

A bare group may use:

- its parameters and names introduced by its patterns
- its own name for recursion
- entry-local type names
- `co.*` APIs
- imported package and library APIs
- compile-time symbols that do not require runtime capture

It may not reference an entry-file runtime variable from the surrounding context:

```
offset := 100;

adjust(n) => { n + offset }
// compiler error: bare function-pattern groups cannot capture `offset`
```

Capturing `let` Function-Pattern Group

The `let` form exists only to declare an entry-local function-pattern group that captures one or more surrounding entry-file runtime bindings:

```
offset := 100;

let adjust(0) = offset;
let adjust(n) = n + offset;

result := adjust(10);
```

The captured names must resolve to surrounding runtime bindings that are already declared and definitely initialized before the first clause of the group. Built-in names, imported declarations, type names, parameters, and the function's own recursive name are not captures.

A `let` function-pattern group must capture at least one surrounding runtime binding. When no capture is required, the bare form must be used:

```
let fib(0) = 1;
let fib(1) = 1;
let fib(n) = fib(n - 1) + fib(n - 2);
// compiler error: this group captures nothing; remove `let`

fib(0) => { 1 }
fib(1) => { 1 }
fib(n) => { fib(n - 1) + fib(n - 2) }
```

The `let` marker is therefore not an optional spelling of a bare function-pattern group. It explicitly requests restricted lexical capture.

Similarities

Bare and capturing `let` function-pattern groups both:

- group compatible clauses by function name and arity
- dispatch by parameter patterns and optional `.where(...)` guards
- evaluate clauses in normal pattern-selection order
- may call themselves recursively
- must maintain compatible parameter and result types across all clauses
- follow the normal pattern ordering, reachability, overlap, and exhaustiveness rules
- are private to the entry file
- cannot be imported, exported, or annotated with package visibility
- cannot be converted to, assigned as, passed as, or returned as function values
- cannot be partially applied or curried

Clause Syntax and Termination

Both forms accept the same pattern list and optional guard:

```
classify(0) => "zero";
classify(n).where(n > 0) => "positive";
classify(_) => { "negative" }

offset := 10;
let adjust(0) = offset;
let adjust(n).where(n > 0) = n + offset;
let adjust(_) = { offset }
```

Annotations may precede either clause form and are retained on the clause for later checking. Because every function-pattern group is private to the entry file, package visibility/export annotations are invalid even though other entry-local metadata annotations may be used.

An expression-bodied clause is a simple statement and must end with `;`. A block-bodied clause ends at its closing `}` and must not be followed by `;`. Newlines never terminate clauses.

Supported parameter patterns are:

- `_` wildcard patterns;
- literals, including signed numeric literals;
- binding names;
- qualified identity names;
- constructor patterns such as `Some(value)`;
- record patterns such as `Employee{id: value, name: _}`;
- tuple patterns with at least two elements, such as `(left, right)`.

`.where(expression)` is evaluated only after the clause's parameter patterns match. Names bound by those patterns are available to the guard and result. The guard expression must have type `co.lang.bool`.

Clauses are considered in source order. A clause is eligible when its arity matches, all parameter patterns match, and its optional guard evaluates to `co.const.true`. The compiler rejects incompatible arities or result types, unreachable clauses, invalid overlaps, and non-exhaustive groups where exhaustiveness is required by the declared result contract.

Differences

Form	Surrounding runtime capture	Intended use
`name(pattern) => result`	No	Entry-local pattern dispatch that depends only on its arguments, recursion, built-ins, imports, and compile-time names
`let name(pattern) = result`	Yes, at least one capture required	Entry-local pattern dispatch that also depends on existing entry-file runtime bindings

Clauses of the same name and arity must all use the same form. Mixing bare and `let` clauses in one function-pattern group is a compiler error.

A capturing `let` function-pattern group is still not a general closure facility. It is named, entry-local, non-first-class, and non-escaping. The compiler may internally retain a capture environment, but FoLang does not expose that environment as a closure object.

`let` cannot be used directly in the entry file for value bindings, anonymous functions, general closure expressions, or curried functions:

```
let base = 10;                  // compiler error: entry-file let value binding
let result = base + 1;          // compiler error: entry-file let value binding
let add = (a, b) => a + b;      // compiler error: anonymous function
let counter = closure { ... };  // compiler error: general closure expression
let add = a => b => a + b;      // compiler error: curried function
```

```
The compiler may lower either function-pattern form to a private entry helper, but neither source construct is a general-purpose
function declaration.
```

Package in detail

Package Identity

A subfolder containing `.fol` files **is** a package.

- Dot paths start from subfolders.
- The project root is **not** a package.
- The root folder name never appears in any package dot path.

Examples:

```
/appl/hr/           -> package "hr"
/appl/hr/employee/  -> package "hr.employee"
/appl/auth/         -> package "auth"
```

The project root itself is **not** a package.

It may contain:

- the application entry file which is same as single source application file
- the packaged library surface file when the project itself is a library
- subfolders that define packages or same-owner source libraries

Multi-File Packages

Multiple `.fol` files in the same subfolder automatically belong to the same package:

```
hr/employee/
├── Employee.fol      →  hr.employee
├── EmpService.fol    →  hr.employee
└── EmpValidator.fol  →  hr.employee
```

Application Project Layout

```
/appl/
├── app.fol                     ←  entry file — not a package
├── hr/                         package "hr"
│   ├── employee/               package "hr.employee"
│   │   ├── Employee.fol
│   │   └── EmpService.fol
│   └── payroll/                package "hr.payroll"
│       ├── Payroll.fol
│       └── PayrollCalc.fol
├── auth/                       package "auth"
│   └── Auth.fol
└── bindings/                   package "bindings"
    └── CLib.fol
```

Package Access Rules

Four access levels control visibility across package boundaries:

Annotation	Same Package	Parent	Sub-Package	Unrelated	Entry / Surface
`@co.dap.public`	✅	✅	✅	✅	✅
`@co.dap.package`	✅	✅	✅	❌	❌
`@co.dap.protected`	✅	❌	✅	❌	❌
`@co.dap.private`	✅	❌	❌	❌	❌

Meaning:

- `@co.dap.public` -> visible everywhere
- `@co.dap.package` -> visible across the package family
- `@co.dap.protected` -> visible downward into subpackages only
- `@co.dap.private` -> visible only inside the declaring package

Example:

```
// hr/EmployeeAccess.fol — package "hr"

@co.dap.public
EmployeeAccess co.lang.unit = {

    @co.dap.public
    getEmployee(id co.lang.int)->(Employee) = { ... }     // anyone can call

    @co.dap.package
    validateId(id co.lang.int)->(co.lang.bool) = { ... }  // hr package family

    @co.dap.protected
    baseQuery()->(co.lang.string) = { ... }               // visible to subpackages

    @co.dap.private
    normalizeId(id co.lang.int)->(co.lang.int) = { ... }  // private declaration
}
```

`co.*` Paths and Aliases

`co.*` Is Always Available

All `co.*` paths are part of the language and are always in scope. They are never imported through `@co.ddap.import`.

```
co.out.println("hello");
co.in.readln();
x co.lang.int = 42;
```

Being **in scope** does not automatically mean **permitted in every context**.

A package rule, library kind, or other compiler restriction may still forbid use of a particular `co.*` facility. In such cases the name is still known to the compiler, but its use is rejected at compile time.

`@co.ddap.alias`

Use aliases only to shorten `co.*` paths.

```
@co.ddap.alias(co.out, as="out")
@co.ddap.alias(co.core.list, as="list")
@co.ddap.alias(co.encoding, as="enc")

out.println("hello");
list.of(1, 2, 3);
enc.json.serialize(data);

// full form still works alongside
co.out.println("world");
```

Rules:

- target must be a `co.*` path
- alias must be a valid identifier
- alias scope is file-local
- aliasing user packages is not allowed; use `as=` in `@co.ddap.import`
- duplicate aliases in the same file are compiler errors
- when no alias is declared, the complete `co.*` path is used
- declaring an alias does not disable or hide the complete `co.*` path ***

Package Source Files

A `.fol` file located inside a package folder contains **exactly one primary top-level declaration**. This gives every package source file a clear structural identity and prevents unrelated declarations or loose functions from being mixed at file scope.

The primary declaration may be a:

- class
- struct
- cstruct

- enum
- union / ADT
- interface
- signature
- module
- type classes/instance
- type constructors
- patterns or objects
- macro
- template
- generics
- annotations
- decorators
- type, type alias, newtype, or opaque type
- unit

File-level import directives, aliases, and annotations may appear before the primary declaration. They do not count as additional primary declarations.

Forbidden directly at package-file scope:

- free-flowing functions
- variables or mutable state
- executable statements
- explicit package declarations
- project metadata
- library metadata
- multiple unrelated primary declarations

The application entry file and library surface files are special source forms and follow their own rules. The single-primary-declaration rule in this section applies to ordinary files inside package folders.

Primary Declaration Names and Filename Inference

The name of a primary declaration may be written explicitly or inferred from the source filename.

When `_` appears in the primary declaration-name position, the compiler derives the declaration name from the filename:

```
// Employee.fol
_ co.lang.struct = {
    id   co.lang.int;
    name co.lang.string;
}
```

This declares `Employee co.lang.struct`.

In this position, `_` does not declare an anonymous or discarded declaration. It means **use the filename-derived declaration name**.

An explicit declaration name is authoritative and overrides filename inference. The explicit name is not required to match the filename:

```
// employee_model.fol
Employee co.lang.struct = {
    id   co.lang.int;
    name co.lang.string;
}
```

This still declares `Employee`; `employee_model.fol` is only the source filename.

Name-selection precedence is therefore:

```
explicit declaration name  -> use the explicit name
_                          -> derive the name from the filename
```

Filename derivation rules:

- for `Name.fol`, the inferred declaration name is `Name`
- for a kind-qualified filename such as `Name.unit.fol`, the compiler removes both `.fol` and the trailing kind suffix when that suffix matches the declaration kind
- the inferred result must be a valid FoLang identifier
- a kind-qualified filename whose suffix conflicts with the declaration kind is a compiler error
- filename inference supplies only the declaration name; generic parameters and all other declaration details remain explicit

Examples:

```
// Status.enum.fol
_ co.lang.enum = {
    active,
    inactive
}
// inferred name: Status
```

```
// Box.struct.fol
_(T) co.lang.struct = {
    value T;
}
// inferred name: Box; T remains explicit
```

```
// Employee.struct.fol
_ co.lang.class = {
    ...
}
// compiler error: filename kind `struct` conflicts with declaration kind `class`
```

Filename inference may be used by named primary declarations such as classes, structs, cstructs, enums, unions, interfaces, signatures, modules, instances, objects, units, and named type declarations.

Example containing a data declaration:

```
// hr/employee/Employee.fol
_ co.lang.struct = {
    id   co.lang.int;
    name co.lang.string;
}
```

When a file needs only ordinary free functions, those functions must be enclosed in a `co.lang.unit`:

```
// hr/employee/EmpService.fol
_ co.lang.unit = {

    getEmployee(id co.lang.int)->(Employee) = {
        this.return Employee{ id: 1, name: "Rao" };
    }
}
```

Application Entry File

The application entry file is a **special executable source form** located at the application root. It is not a package, does not create an importable namespace, and is not subject to the ordinary package-file rule requiring exactly one primary declaration.

The compiler creates a dedicated **entry-file context** for it:

```
ApplicationEntryContext
├── file directives, imports, and aliases
├── entry-local non-structural type declarations
├── non-capturing function-pattern groups
├── capturing `let` function-pattern groups
└── executable statements and expressions
```

Everything declared directly in this context is private to the entry file. Entry-local symbols cannot be imported, exported, or referenced by package or library source files.

Allowed Entry-File Constructs

The application entry file uses exactly the same grammar and allowed-construct rules described in Single Source Application File. The earlier section provides the developer-facing overview; this section defines the entry file's formal context, privacy, and dependency direction.

Entry-Local Function Patterns

Function-pattern groups are allowed as a special entry-file construct even though ordinary function declarations are forbidden. FoLang provides two entry-file forms with the same pattern-dispatch model but different capture semantics.

Entry-File Dependency Direction

The entry file may depend on packages and libraries, but packages and libraries may never depend on the entry context:

```
application entry file
    ↓ uses
packages and libraries
```

This allows the entry file to coordinate application startup while preserving package and library independence.

Libraries

Library Project Layout

```
/hrlib/
├── hrlib.fol                   ←  library surface — @co.dap.library
├── emp/                        package "emp" — internal, invisible to consumer
│   ├── Employee.fol
│   └── EmpService.fol
└── auth/                       package "auth" — internal, invisible to consumer
    ├── Auth.fol
    └── AuthService.fol
```

Consumer only sees what `hrlib.fol` declares. All subfolders are internal.

Library Surface file

FoLang uses surface files in two situations:

1. **Packaged library project surfaces**
2. **Application-workspace source library surfaces**

A surface file is a special source form annotated with `@co.dap.library`. It defines one library identity, its public boundary data contracts, and the boundary-adapter functions through which consumers call the library.

```
app.fol   -> application entry
hrlib.fol -> packaged library surface
ffi.fol   -> source library surface when imported with src-library=true
```

A library surface is not an ordinary package file. It may contain multiple boundary data declarations and public functions inside one `co.lang.library` declaration.

Packaged Library Project Surface

A packaged library project has no application entry file. It has exactly one surface file at the project root.

```
/hrlib/
├── hrlib.fol                   <- library surface
├── emp/                        <- internal package
│   ├── Employee.fol
│   └── EmployeeService.fol
└── auth/                       <- internal package
    └── AuthService.fol
```

Rules:

- exactly one `@co.dap.library` surface file exists per packaged library project
- the surface file is located at the project root
- it is compiled into the packaged library artifact, such as `.folib` or `.folenc`
- consumers import it with `@co.ddap.import(library="...")`
- internal package folders are compiled into the library but are not directly visible to consumers

Application-Workspace Source Library Surface

A source library may live inside an application workspace.

```
@co.ddap.import(
    package="com.abc.ffi",
    src-library=true,
    expect="ffi",
    as="ffilib"
)
```

The import resolves to one surface file:

```
/appl/com/abc/ffi.fol
```

Rules:

- a source-library surface file must be below the application root, never at the application root
- multiple source libraries may exist in one application workspace
- the surface is imported through `package="..."` with `src-library=true`
- only the projected surface API is importable
- internal packages remain hidden even though their source files are physically available
- once a source tree is treated as a library, its subpackages cannot be imported as ordinary packages by consumers
- packaged libraries and source libraries use exactly the same public-surface and API-projection rules

Unified Surface Model

Every library kind uses the same conceptual surface shape:

```
library surface
├── boundary data contracts
└── public boundary-adapter functions
```

Library kind changes the permitted boundary representation and transfer semantics, not the conceptual form of the API.

Library kind	Boundary data	Transfer semantics
`application`	`co.lang.struct`	automatic deep snapshot
`dynamicvmrt`	`co.lang.struct`	automatic deep snapshot
`advanced`	`co.lang.struct`	automatic deep snapshot
`system`	`co.lang.cstruct`	system ABI value
`ffi`	`co.lang.cstruct`	C ABI value

`co.lang.struct` and `co.lang.cstruct` solve different problems:

```
struct  -> FoLang semantic data contract
cstruct -> physical ABI-compatible value contract
```

Value transfer must not be confused with C-compatible layout. Application-family libraries therefore use expressive `struct` contracts transferred as deep snapshots, while system and FFI libraries use restricted `cstruct` contracts.

Allowed Surface Declarations

A library surface may contain only:

- file- or library-level imports needed by its adapter implementations
- `co.lang.struct` boundary declarations for `application`, `dynamicvmrt`, and `advanced`
- `co.lang.cstruct` boundary declarations for `system` and `ffi`
- public free-function API declarations with boundary-adapter definitions

The following declaration kinds are forbidden directly in every library surface:

- classes
- modules
- interfaces
- signatures
- units and companion units
- associated functions
- operator functions
- enums, unions, and other ADTs
- type aliases, newtypes, and opaque types
- objects, instances, type classes, and dependent types
- macros, templates, annotations, and decorators
- global variables, pointers, references, addresses, or mutable surface state

Surface `struct` and `cstruct` declarations are data contracts only. They cannot have companion units, associated functions, operators, methods, or other behavior on the surface.

Declarations directly inside the `co.lang.library` body are exported by default. Imports and implementation details are never exported.

Public Signature Type Closure

Every public surface function signature must be closed over the library's exported boundary type set.

For `application`, `dynamicvmrt`, and `advanced` surfaces, parameters and results may use only:

- approved built-in types
- `co.lang.struct` types declared in the same library surface

For `system` and `ffi` surfaces, parameters and results may use only:

- ABI-safe built-in types
- `co.lang.cstruct` types declared in the same library surface

The same closure rule applies recursively to fields of surface boundary types:

- a surface `struct` field may use an approved built-in type or another surface-declared `struct`
- a surface `cstruct` field may use an ABI-safe built-in type or another surface-declared `cstruct`
- an internal package type may never appear in a public function signature or surface boundary-type field

- pointers, references, and addresses may never cross any public library surface

A built-in type is not automatically surface-safe merely because it belongs to `co.lang`. An approved surface built-in must be concrete, fully resolved, and transferable under the library kind's boundary semantics.

The following categories are forbidden in public surface fields and signatures:

- inference-only types such as `co.lang.auto` and `co.lang.infer`
- dynamically typed or unconstrained carriers such as `co.lang.dynamic`, `co.lang.any`, `co.lang.typed`, and `co.lang.untyped`
- function, closure, delegate, loader, realm, AST, reflection, or runtime implementation values
- pointer, reference, address, thunk, and implementation-handle types
- any type whose reachable representation contains a forbidden type

For application-family surfaces, managed built-ins such as `co.lang.string` are permitted when the compiler defines deep-snapshot reconstruction for them. For system and FFI surfaces, only built-ins with a defined ABI representation are permitted; for example, `co.lang.string` is not directly cstruct-compatible.

Valid:

```
Employee co.lang.struct = {
    id      co.lang.int;
    name    co.lang.string;
    address Address;
}

Address co.lang.struct = {
    city co.lang.string;
}
```

Invalid:

```
getEmployee(id co.lang.int)->(emp.internal.Employee);
// compiler error: an internal type escapes through the public signature
```

Boundary-Adapter Functions

A public surface function may contain a definition, but its definition is restricted to boundary adaptation.

A boundary-adapter function may:

- read its input parameters and their fields
- declare temporary local values used only for conversion
- construct internal input values
- perform deterministic field-to-field or representation conversion
- invoke exactly one internal implementation operation
- receive that operation's result
- construct and return an allowed surface `struct`, surface `cstruct`, or built-in value
- use a direct delegation expression when no conversion is required

A boundary-adapter function may not:

- implement business rules or domain calculations
- perform business validation or authorization decisions
- orchestrate multiple implementation operations
- call another surface function
- perform persistence, networking, file I/O, retries, caching, or transactions
- start concurrency, scheduling, asynchronous work, processes, or continuations
- contain loops, recursion, workflow branching, or externally observable state mutation
- expose an internal value directly without converting it to an allowed boundary type

The compiler enforces the restricted adapter statement set. The restriction is structural: arbitrary executable logic is not accepted merely because it is placed in a surface file.

A direct delegate is the simplest adapter form:

```
health()->(co.lang.bool)
    =>> health.internal.Service.health();
```

A converting adapter may map between the public contract and an internal model.

Application Surface Example

```
// hrlib.fol
@co.dap.library(type="application")
hrlib co.lang.library = {

    @co.ddap.import(package="emp", as="emp")

    Employee co.lang.struct = {
        name co.lang.string;
        id   co.lang.int;
    }

    getEmployee(empId co.lang.int)->(Employee) = {
        internalEmployee := emp.EmployeeService.getEmployee(empId);

        this.return Employee{
            name: internalEmployee.name,
            id: internalEmployee.id
        };
    }
}
```

The surface owns conversion from the internal `emp.Employee` representation to the public `hrlib.Employee` contract. The `emp` package owns validation, business rules, persistence, and workflow.

The consumer sees the equivalent API contract:

```
Employee co.lang.struct = {
    name co.lang.string;
    id   co.lang.int;
}

getEmployee(empId co.lang.int)->(Employee);
```

The consumer does not see the body of `getEmployee` or the `emp` package.

System and FFI Surface Example

```
// driver.fol
@co.dap.library(type="system")
driver co.lang.library = {

    @co.ddap.import(package="driver.internal", as="impl")

    Point co.lang.cstruct = {
        x co.lang.int32;
        y co.lang.int32;
    }

    getOrigin()->(Point) = {
        internalPoint := impl.DriverService.getOrigin();

        this.return Point{
            x: internalPoint.x,
            y: internalPoint.y
        };
    }
}
```

Pointers, addresses, native bindings, and hardware state belong in `driver.internal`. They are forbidden in the public surface and cannot appear in the exported signature.

Boundary Transfer Semantics

For `application`, `dynamicvmrt`, and `advanced` libraries:

```
consumer boundary value
    ↓ automatic deep snapshot of the complete reachable value graph
library-local built-in or surface struct
    ↓ surface conversion
internal implementation value
    ↓ surface conversion
library-local built-in or surface struct
    ↓ automatic deep snapshot
consumer-local boundary value
```

The snapshot rule applies to both surface structs and approved built-in arguments/results. The library never receives the consumer's live object identity, and the consumer never receives a live internal-library object.

For `system` libraries, `cstruct` values cross according to the selected backend/platform system ABI.

For `ffi` libraries, `cstruct` values cross according to the declared C ABI, including its layout, alignment, and calling-convention requirements.

Surface-to-Internal Dependency Direction

The source-level dependency is one-way:

```
library surface
    ↓ imports and invokes
internal packages
```

Internal packages:

- do not import the library surface
- do not use surface-declared `struct` or `cstruct` types
- define their own implementation and domain types
- return internal values to the surface adapter
- contain all business logic, validation, workflow, I/O, and state management

This prevents a surface/internal compilation cycle and keeps the public contract independent from the internal domain model.

An internal implementation symbol may be visible to its own library surface without becoming part of the consumer API. Library encapsulation takes precedence over ordinary package visibility: consumers cannot resolve internal package symbols even when those symbols are callable from the surface during library compilation.

Consumer API Projection

The compiler does not expose the complete surface compilation context to a consumer. It creates a projected imported symbol table.

The projected API contains:

- the library identity and kind
- complete public surface `struct` or `cstruct` definitions, including field names and permitted field types
- public function names
- public function parameter names and types
- public function result types
- calling-convention, effect, error, and linkage metadata where applicable

The projected API does not contain:

- function bodies or implementation AST/HIR
- imports used by the surface
- local conversion variables
- delegate or implementation targets
- internal package paths or symbols
- private compiler-generated helpers
- business classes, modules, units, or internal data types

Therefore, a function definition may exist in the surface source and compiled artifact while the consumer's symbol table contains only its signature.

The backend may retain implementation bodies for code generation, linking, or whole-program optimization. Backend possession of implementation code does not make that code semantically visible to the consumer.

Source libraries follow the same rule. Availability of source code does not weaken the projected API boundary.

Library Compilation Order

A library is processed in four logical stages:

1. parse the surface header, boundary data declarations, and public function signatures
2. compile internal packages without depending on surface types
3. type-check and link surface boundary-adapter bodies against internal package symbols
4. emit the compiled implementation and the projected consumer API

This order permits the surface to call internal packages while preventing internal packages from depending on the surface.

Library Kinds

Library kinds are declared on the surface file:

```
@co.dap.library(type="ffi")
@co.dap.library(type="system")
@co.dap.library(type="dynamicvmrt")
@co.dap.library(type="advanced")
@co.dap.library(type="application")
```

Shared Meaning

- `application` -> ordinary safe library implementation
- `dynamicvmrt` -> application features plus full `co.meta` dynamic-runtime support
- `advanced` -> macro, template, concurrency, transformation, and runtime-machinery implementation
- `system` -> unsafe low-level runtime, pointer, process, native, and platform-control implementation
- `ffi` -> foreign-interface implementation

Library kind controls internal capabilities. All kinds still expose only boundary data contracts and public boundary-adapter functions.

`application` Libraries

Internally allowed:

- ordinary safe language features
- classes, modules, interfaces, signatures, units, and ordinary structs
- normal application-level packages and libraries

Internally forbidden:

- pointers, references, and addresses
- native functions annotated with `@co.dap.native`
- macros and templates
- low-level concurrency machinery
- advanced transformation or dynamic-runtime machinery

Public boundary:

- surface `struct` contracts
- approved built-in types
- deep-snapshot transfer

`dynamicvmrt` Libraries

Internally allowed:

- all `application` capabilities
- full `co.meta` support
- runtime reflection, instrumentation, dynamic loading, patching, and eval-based facilities

Internally forbidden:

- system and FFI capabilities unless reached through an imported public library surface
- raw pointers, addresses, and native functions in its own implementation

Public boundary:

- surface `struct` contracts
- approved built-in types
- deep-snapshot transfer

`advanced` Libraries

Internally allowed:

- all ordinary safe language features
- macros and templates
- async, parallel, continuation, scheduling, and transformation machinery
- compiler/runtime behavior definitions permitted to advanced libraries

Internally forbidden:

- raw pointers
- references and addresses
- native functions

Public boundary:

- surface `struct` contracts
- approved built-in types
- deep-snapshot transfer

`system` Libraries

Internally allowed:

- pointers, references, addresses, and word-level types
- native functions

- structs and cstructs
- units containing free functions
- basic value-typed functions
- type aliases
- basic threading and process primitives

Internally forbidden:

- classes
- modules
- interfaces and signatures
- overloading and overriding
- new operator definitions
- generics and templates
- macros
- reflection and dynamic runtime
- dependent types and type classes
- function patterns
- closures and currying
- advanced concurrency machinery such as continuations, defer, lazy evaluation, and thunks
- variadic, named, optional, and default parameters

Public boundary:

- surface `cstruct` contracts
- ABI-safe built-in types
- system ABI value transfer

`ffi` Libraries

Internally allowed:

- native and extern declarations
- pointers, references, addresses, and C ABI types
- cstructs and restricted implementation structs
- units containing free functions
- foreign calling-convention and symbol-linkage metadata

Internally forbidden:

- classes
- modules
- interfaces and signatures
- overloading and overriding
- new operator definitions
- generics and templates
- macros
- reflection and dynamic runtime
- dependent types and type classes
- closures, currying, and advanced concurrency machinery

Public boundary:

- surface `cstruct` contracts
- C-ABI-safe built-in types
- C ABI value transfer

Dependency Direction

Allowed dependency flow is one-way:

```
application
    ↓ through public surface APIs only
dynamicvmrt
    ↓ through public surface APIs only
advanced
    ↓ through public surface APIs only
system
    ↓ through public surface APIs only
ffi
```

A library may depend on a library at the same or a lower level only when the dependency does not create a cycle. Reverse dependencies are compiler errors.

Cross-Library Communication

From	To	Allowed?	Notes
`application`	`dynamicvmrt`	✅	projected public surface only
`application`	`advanced`	✅	projected public surface only
`application`	`system`	✅	projected public surface only
`application`	`ffi`	✅	projected public surface only
`dynamicvmrt`	`advanced`	✅	projected public surface only
`dynamicvmrt`	`system`	✅	projected public surface only
`dynamicvmrt`	`ffi`	✅	projected public surface only
`dynamicvmrt`	`application`	❌	reverse dependency
`advanced`	`system`	✅	projected public surface only
`advanced`	`ffi`	✅	projected public surface only
`advanced`	`application`	❌	reverse dependency
`advanced`	`dynamicvmrt`	❌	reverse dependency
`system`	`ffi`	✅	projected public surface only
`system`	`application`	❌	reverse dependency
`system`	`dynamicvmrt`	❌	reverse dependency
`system`	`advanced`	❌	reverse dependency
`ffi`	`application`	❌	reverse dependency
`ffi`	`dynamicvmrt`	❌	reverse dependency
`ffi`	`advanced`	❌	reverse dependency
`ffi`	`system`	❌	reverse dependency

Units in detail

A `unit` is a named, non-instantiable container for functions.

Like other named primary declarations, a unit may use an explicit name or `_` for filename-derived naming. For example, `_ co.lang.unit` in `Math.fol` declares `Math`, while an explicit name such as `Math co.lang.unit` is authoritative regardless of the filename.

Its purpose is structural: it prevents functions from flowing freely at package-file scope and preserves FoLang's rule that every ordinary package source file has one primary top-level declaration. A unit does **not** require its functions to form a domain model, capability, service, or other semantic abstraction.

Functions within a unit may be strongly related by purpose—for example, mathematical, parsing, encoding, or validation functions—or only loosely related. Semantic cohesion is encouraged as a source-design practice but is not enforced by the language.

A unit has two forms:

1. **Standalone unit** — its name does not match a struct in the same package. It acts as an ordinary named container for receiverless functions.
2. **Struct companion unit** — its name matches exactly one `co.lang.struct` in the same package. It provides behaviour associated with that struct while the struct declaration itself remains pure data. For more details please refer section Struct Companion Units

Only `co.lang.struct` can have a companion unit. A same-name unit is not allowed for `co.lang.class`, `co.lang.cstruct`, modules, enums, unions, interfaces, signatures, or other declaration kinds. Classes already contain their own methods and lifecycle behaviour; `cstruct` remains a restricted C-compatible data representation.

```
Text co.lang.unit = {

    trim(value co.lang.string)->(co.lang.string) = {
        ...
    }

    contains(
        value  co.lang.string,
        search co.lang.string
    )->(co.lang.bool) = {
        ...
    }

    repeat(
        value co.lang.string,
        count co.lang.int
    )->(co.lang.string) = {
        ...
    }
}
```

Unit functions are accessed through the unit name:

```
cleaned := Text.trim(input);
found   := Text.contains(input, "FoLang");
```

A unit:

- introduces a named scope for its functions
- contains receiverless functions
- may additionally contain associated and operator functions when it is a struct companion unit
- requires the first declared parameter of every receiverless companion function to have the matching struct type
- uses the explicit receiver, rather than ordinary-parameter matching, to associate an associated function with its struct
- may contain public and private functions
- may use built-in types, user-defined types, or both
- does not introduce a user-defined data type or ADT
- has no fields or unit-level variables
- has no constructors or lifecycle methods
- has no instances, object identity, inheritance, or polymorphic dispatch
- cannot be instantiated or assigned as an object value
- cannot contain nested classes, structs, enums, modules, or other primary type declarations

Physical type or function declarations inside an individual unit function are not permitted. A separately declared class, struct, enum, module, or function may instead be restricted to one or more exact unit functions with `@co.dap.local`, using each function's complete qualified signature in the target set.

```
General co.lang.unit = {

    parseNumber(value co.lang.string)->(co.lang.int) = { ... }

    clamp(
        value co.lang.int,
        min   co.lang.int,
        max   co.lang.int
    )->(co.lang.int) = { ... }
}
```

The functions inside a unit do not need to use UDTs or ADTs. A unit is especially useful for small operations that consume built-in values, perform a computation, and return built-in values.

A unit may also represent a clearly related family of operations:

```
Math co.lang.unit = {

    abs(value co.lang.int)->(co.lang.int) = { ... }

    max(
        a co.lang.int,
        b co.lang.int
    )->(co.lang.int) = { ... }

    sqrt(value co.lang.float)->(co.lang.float) = { ... }
}
```

The common purpose of these functions makes the source easier to understand, but the compiler does not require or attempt to prove that all functions in a unit are semantically related.

CStructs

`co.lang.cstruct` is a C-like value type — passed by value, simple memory layout, safe to cross zone boundaries. Unlike `co.lang.struct` which is passed by reference, `co.lang.cstruct` is always copied on pass.

```
Point co.lang.cstruct = {
    x co.lang.int;
    y co.lang.int;
}

Rect co.lang.cstruct = {
    origin Point;
    width  co.lang.int;
    height co.lang.int;
}
```

cstruct Rules

```
always passed by value — never by reference
simple memory layout — no metadata
can contain only simple types and other cstructs
cannot contain co.lang.struct                ❌  has metadata
cannot contain co.lang.string                ❌  heap allocated
cannot contain co.lang.dynamic               ❌  runtime type info
cannot contain classes                       ❌  vtable, metadata
cannot contain modules                       ❌
cannot contain any heap allocated type       ❌
cannot have methods
cannot have associated functions
cannot embed co.lang.struct
safe to cross direct ABI and zone boundaries

allowed field types:
    co.lang.int, co.lang.uint, co.lang.float  ✅  primitives
    co.lang.bool, co.lang.char, co.lang.byte  ✅  primitives
    co.lang.int->([N])                          ✅  fixed size arrays
    co.lang.cstruct                             ✅  other cstructs
```

Packed cstruct — no padding, exact memory layout

Used for hardware registers, binary protocols, exact memory mapped formats:

```
@co.dap.packed
Register co.lang.cstruct = {
    flags  co.lang.uint8;
    status co.lang.uint8;
    data   co.lang.uint16;
}
```

SIMD cstruct — aligned for vector operations

Used for math, graphics, signal processing:

```
@co.dap.simd(align=16)
Vec4 co.lang.cstruct = {
    x co.lang.float;
    y co.lang.float;
    z co.lang.float;
    w co.lang.float;
}
```

Both together

```
@co.dap.packed
@co.dap.simd(align=32)
AVXVec co.lang.cstruct = {
    data co.lang.float;
}
```

> `@co.dap.packed` and `@co.dap.simd` are specialisations of `co.lang.cstruct` — same rules, same zone boundary safety. They are not separate types.

Structs

```
myStruct co.lang.struct={
    field1 co.lang.int;
    field2 co.lang.string;
    field3 co.lang.bool;
}
```

Struct Rules

```
structs cannot extend other structs
structs cannot contain methods directly
structs can compose other structs
structs cannot have default values to fields/members
structs can embed other structs (Go lang like)
structs can have a same-package companion unit with the same name
```

The struct declaration remains pure data. All behaviour associated with a struct is declared separately in its matching `co.lang.unit`.

Struct Embedding

Embedding promotes fields of an embedded struct directly into the outer struct — they act as the outer struct's own fields at construction and access sites. This is distinct from composition where the embedded struct is a named field.

```
E co.lang.struct = {
    id   co.lang.int;
    name co.lang.string;
}

// ✅ No conflict — id and name promoted as B's own fields
B co.lang.struct = {
    age co.lang.float;
    E;                    // embedded — id and name promoted
}

b := B{ age: 25.0, id: 1, name: "Rao" };   // all fields at same level
b.id    // direct — no b.E.id needed
b.name  // direct — no b.E.name needed
b.age   // direct
```

```
// ❌ Compiler error — name conflict between B.name and E.name
B co.lang.struct = {
    name co.lang.string;   // conflicts with E.name
    E;
    age  co.lang.float;
}
// Fix 1 — rename B's conflicting field
// Fix 2 — use explicit composition instead: e E;
```

```
// Explicit composition — no promotion, always qualified access
B co.lang.struct = {
    name co.lang.string;
    e    E;               // named field — no conflict, no promotion
    age  co.lang.float;
}

b.name ;   // B's own name
b.e.id ;   // E's id — always explicit
b.e.name;  // E's name — always explicit
```

Embedding Rules

Situation	Behavior
Embedded field, no conflict	Promoted — acts as the outer struct's own field
Embedded field, name conflict with outer	❌ Compiler error — rename or use composition
Multiple embeds, no conflict between them	All fields promoted
Multiple embeds, conflict between embedded structs	❌ Compiler error
Explicit composition (`e E`)	No promotion — always accessed via `b.e.field`

> FoLang does **not** silently shadow conflicting fields. Any name conflict is a compiler error — the programmer must make a conscious decision to rename or switch to explicit composition.

Struct Declaration Relationships

A struct cannot physically declare another struct, enum, class, module, function, signature, interface, or other named declaration inside its body. A struct body remains a pure data declaration containing fields and permitted embeddings only.

Use one of the following instead:

- **composition** through a named field
- **embedding** where the declaration kind's embedding rules permit it
- a separately declared target-local declaration restricted with `@co.dap.local`

```
// EmployeeAddress.fol
@co.dap.local(for=hr.employee.Employee)
EmployeeAddress co.lang.struct = {
    street co.lang.string;
    city   co.lang.string;
}
```

```
// Employee.fol
Employee co.lang.struct = {
    id      co.lang.int;
    name    co.lang.string;
    address EmployeeAddress; // composition
}
```

The following physical nesting is invalid:

```
Employee co.lang.struct = {
    Address co.lang.struct = { // ❌ nested declaration
        city co.lang.string;
    }

    address Address;
}
```

```
structs cannot declare inner structs     ❌  compiler error — only through @co.dap.local
structs can declare inner enums/ADTs     ❌  compiler error — only through @co.dap.local or @co.dap.nested
structs cannot declare inner classes     ❌  compiler error — struct is pure data
structs cannot declare inner modules     ❌  compiler error — struct is pure data
```

`@co.dap.local` controls declaration visibility only. It does not automatically compose or embed the annotated declaration into the target struct.

Struct Companion Units

When a unit has the same name as a struct in the same package, it is the companion unit of that struct.

The struct and its companion unit are separate primary declarations and therefore normally reside in separate source files within the same package. They are shown together below only to illustrate their relationship.

```
// Vector.fol
Vector co.lang.struct = {
    x co.lang.float;
    y co.lang.float;
}

// Vector.unit.fol
_ co.lang.unit = {

    distance(
        left  Vector,
        right Vector
    )->(co.lang.float) = {
        ...
    }

    isZero(value Vector)->(co.lang.bool) = {
        this.return value.x == 0.0 && value.y == 0.0;
    }
}
```

Receiverless functions in a companion unit are accessed through the shared struct/unit name:

```
d := Vector.distance(first, second);
zero := Vector.isZero(value);
```

These functions are analogous to **static functions of a class** in call form, but FoLang additionally requires their **first declared parameter** to have the matching struct type. The first parameter establishes ownership by the companion unit. A matching struct type in a later parameter is insufficient.

```
Vector co.lang.unit = {
    distance(left Vector, right Vector)->(co.lang.float) = { ... } // ✅
    convert(value Vector, radix co.lang.int)->(co.lang.string) = { ... } // ✅

    invalid(scale co.lang.float, value Vector)->(Vector) = { ... }
    // ❌ first parameter is not Vector

    zero()->(Vector) = { ... }
    // ❌ no first parameter identifies Vector ownership
}
```

Therefore, factory-style receiverless functions such as `Vector.create(x, y)` and zero-parameter receiverless functions such as `Vector.zero()` are not valid under this rule because no ordinary parameter establishes ownership. A factory may instead be declared as a type-associated function, described below, or placed in a standalone unit with a different name.

Companion-unit functions do not make the struct a class and do not introduce object identity, inheritance, virtual dispatch, lifecycle methods, or unit-level state.

Associated Functions in a Companion Unit

A companion unit may contain instance-associated and type-associated functions. Both forms use an explicit receiver to establish association with the matching struct, but they differ in whether the receiver is a struct value or the struct type itself.

Instance-Associated Functions

An instance-associated function has an explicit receiver value whose type is the matching struct:

```
Vector co.lang.unit = {

    (value Vector) magnitude()->(co.lang.float) = {
        this.return co.math.sqrt(
            value.x * value.x + value.y * value.y;
        );
    }

    (value Vector) scale(factor co.lang.float)->(Vector) = {
        this.return Vector{
            x: value.x * factor,
            y: value.y * factor
        };
    }
}
```

Associated functions may be invoked using method-call syntax:

```
v Vector=Vector{};
length := v.magnitude();
scaled := v.scale(2.0);
```

The method-call form is syntactic association. Conceptually:

```
v.magnitude();
```

resolves to the associated function declared in `Vector co.lang.unit` with `v` supplied as its explicit receiver. Associated functions are therefore analogous to **instance methods**, but they remain externally declared functions and do not acquire class semantics.

Instance-associated-function rules:

- the explicit receiver value must have the struct type whose name matches the unit name
- the receiver establishes association; the ordinary parameter list does not need to contain the matching struct type
- the matching struct and unit must be declared in the same package
- the function cannot be declared loose at package scope
- it has no inheritance, overriding, or virtual dispatch
- it receives no special private access beyond normal visibility rules
- an imported struct with the same short name does not create a companion relationship

```
Employee co.lang.struct = {
    id co.lang.int;
}

Employee co.lang.unit = {
    (emp Employee) isValid()->(co.lang.bool) = { ... }       // ✅
    (dept Department) isValid()->(co.lang.bool) = { ... }   // ❌ receiver does not match Employee
}
```

Type-Associated Functions

A type-associated function uses the matching struct type itself as its explicit receiver. It is analogous to a class-level or static factory function in call form, but it does not introduce class lifecycle, inheritance, or object-oriented dispatch.

```
Vector co.lang.unit = {
    (Vector) zero()->(Vector) = {
        this.return Vector{x: 0.0, y: 0.0};
    }

    (Vector) create(
        x co.lang.float,
        y co.lang.float
    )->(Vector) = {
        this.return Vector{x: x, y: y};
    }
}
```

Type-associated functions are invoked through the struct name:

```
origin := Vector.zero();
value  := Vector.create(10.0, 20.0);
```

Type-associated-function rules:

- the explicit receiver must be the struct type whose name matches the unit name
- the type receiver establishes association, so no ordinary parameter is required to have the matching struct type
- zero ordinary parameters are permitted
- the matching struct and unit must be declared in the same package
- the function cannot be declared loose at package scope
- the struct type is not an object instance and cannot be mutated through the type receiver
- type association does not add constructors, lifecycle methods, inheritance, overriding, or virtual dispatch
- an imported struct with the same short name does not create a companion relationship

The three companion-function forms are therefore:

```
receiverless companion function
    -> first ordinary parameter struct
    -> parameter not matching struct (as companion unit name match and must match with struct name)

instance-associated function
    -> explicit receiver is a value of the matching struct

type-associated function
    -> explicit receiver is the matching struct type itself
```

Companion Name Rules

Within one package, FoLang may contain:

```
one Vector co.lang.struct
one Vector co.lang.unit={}
```

Together they form one struct/companion pair. The following are compiler errors:

```
two units named Vector in the same package
a unit matching a class or cstruct
a companion unit located in a different package from its struct
an instance-associated function whose receiver value is not of the matching struct type
a type-associated function whose type receiver is not the matching struct type
```

Name resolution is determined by context:

```
v Vector;                // type position → Vector struct
v := Vector{x: 1, y: 2}; // construction → Vector struct
z := Vector.isZero(v);   // qualified function lookup → Vector companion unit
m := v.magnitude();      // associated-function lookup → Vector companion unit
```

```
x General = General{};  // ❌ compiler error — a unit is not instantiable
```

> Here one thing is worth mentioning, we know all `folang` statements or expressions terminate with either semcolon or closing brace one excpetion is for pragmas/decorators/annotations/directives they don't need trailing semicolon.

> Take the example `v Vector=Vector{};` or `v := Vector{x: 1, y: 2};` you may see this kind of thing though out `folang` document, this is not block statment it is UDT literal value, so there is a difference between block and literal values; literal values even though contains block kind of construct needs to be terminate with semi colon.

> Never confuse block with UDT object literal all object literal values are `json format or json representation`.

> An object literal can be empty {} means initialized with default values as per `folang`

> Difference between block and object literal:

```
1. Object literal/literal value is in json representation/json format like any other literal values e.g., 10, 'A', "Some string"
2. Block is collection of single/multiple statments/expressions
```

Companion Function Categories and Operator Functions

```
Employee co.lang.struct = {
    id   co.lang.int;
    name co.lang.string;
}

Employee co.lang.unit = {

    // Receiverless companion function: first parameter establishes ownership.
    compare(
        left  Employee,
        right Employee
    )->(co.lang.int) = {
        ...
    }
    // Recieverless companion function: without matching first parameter with type
    getEmployee(id co.lang.string)->(Employee)={...}


    // Instance-associated function: receiver is an Employee value.
    (emp Employee) fetchEmployee(
        empId co.lang.string
    )->(Employee) = {
        ...
    }

    // Type-associated function: receiver is the Employee type.
    (Employee) getInstance()->(Employee) = {
        ...
    }
}
```

Call forms:

```
order  := Employee.compare(emp, other); // receiverless companion function
result := emp.fetchEmployee("E1");      // instance-associated function
emp    := Employee.getInstance();       // type-associated function
emp1   := Employee.getEmployee("E0021");   // receiverless companion function
```

The receiver remains explicit in associated-function declarations even though instance-call or type-call syntax is available at the call site. This does not give the struct class semantics: there is no inheritance, overriding, virtual dispatch, hidden receiver, or lifecycle.

Operator functions associated with a struct must be declared in its companion unit. Ownership is established by exactly one of the same three companion forms: a matching instance receiver, a matching type receiver, or (when there is no receiver) a first ordinary parameter of the matching struct type:

```
Employee co.lang.unit = {

    @co.dap.operator(symbol="+")
    (emp Employee) add(other Employee)->(Employee) = {
        // implementation
    }

    @co.dap.operator(symbol="==")
    equals(
        left  Employee,
        right Employee
    )->(co.lang.bool) = {
        // implementation
    }

    @co.dap.operator(symbol=">")
    (Employee) greater(
        left  Employee,
        right Employee
    )->(co.lang.bool) = {
        // implementation
    }
}
```

A receiverless operator function must have the matching struct as its first ordinary parameter. A type receiver establishes ownership by itself and does not add an operand or require an ordinary parameter; when it does declare the same ordinary operand list as a receiverless form, the two declarations normalize to the same operator signature and are duplicate definitions. An instance receiver establishes ownership and contributes the receiver value as the first operator operand.

Unions

Unions are untagged ADTs

```
 myUnion co.lang.union={
    intValue co.lang.int;
    strValue co.lang.string;
}
```

Enums

```
myEnum co.lang.enum={
    Variant1,
    Variant2,
    Variant3
}
```

An enum value's constant expression may use a registered custom operator at any declared precedence. Runtime assignment is forbidden everywhere in that constant-expression subtree, including inside grouping, arguments, and collection or object elements. Whether the resulting expression can actually be evaluated at compile time—including whether a custom operator is foldable— is checked after parsing.

classes

```
Employee co.lang.class ={
    getEmployeeDetails()->(Employee) = empmodule.getEmployeeDetails;
    // assigning module function to class's method

    getEmployeeInfo()->(Employee) =>> empmodule.getEmployeeDetails();
    // delegating — internally redirecting the call to module function
}

// $1, $2, $3 ... are previous results captured as bind variables
Emp co.lang.class={
    dosomething(a co.lang.int, b co.lang.int)->(co.lang.int)=>>somePack.someMethod(a)=>>someOthPack.someOtherMeth($1, b);
}
```

Classes with Operator methods

```
Employee co.lang.class ={
    @co.dap.operator(symbol="+")
    add(other Employee)->(Employee) = {
       // implicit instance method: operands are this and other
    }

    @co.dap.operator(symbol="==")
    @co.dap.static
    equals(
        left  Employee,
        right Employee
    )->(co.lang.bool) = {
        // static operator implementation
    }

    @co.dap.operator(symbol=">")
    @co.dap.class
    greater(
        left  Employee,
        right Employee
    )->(co.lang.bool) = {
        // class-associated operator implementation
    }

}
```

An unannotated operator function in a class is an implicit instance method; the hidden `this` value contributes the first operand. `@co.dap.instance` is the explicit spelling of the same category. `@co.dap.static` and `@co.dap.class` do not contribute an implicit operand, so their declared parameters are the complete operator operand list. Operator signature normalization includes these method categories so equivalent declarations are diagnosed as duplicates.

Class Declaration Relationships

A class cannot physically contain named class, struct, enum, module, function, signature, interface, or other declaration definitions as nested declarations. Class bodies contain fields, lifecycle declarations, and methods permitted by the class model.

Types and helper declarations used only by one class are declared in their ordinary legal source locations and restricted to that class with `@co.dap.local`:

```
// EmployeeAddress.fol
@co.dap.local(for=hr.employee.Employee)
EmployeeAddress co.lang.struct = {
    street co.lang.string;
    city   co.lang.string;
}
```

```
// EmployeeStatus.fol
@co.dap.local(for=hr.employee.Employee)
EmployeeStatus co.lang.enum = {
    Active,
    Inactive,
    Pending
}
```

```
// Employee.fol
Employee co.lang.class = {
    address EmployeeAddress;
    status  EmployeeStatus;

    getAddress()->(EmployeeAddress) = {
        this.return this.address;
    }
}
```

The local declarations are visible while compiling `hr.employee.Employee`, but they do not become nested names such as `Employee.EmployeeAddress` or `Employee.EmployeeStatus`.

The following is invalid:

```
Employee co.lang.class = {
    Address co.lang.struct = { // ❌ physical nested declaration
        city co.lang.string;
    }
}
```

Ordinary visibility annotations do not widen a target-local declaration beyond the declaration or closed target set named by `@co.dap.local` or `@co.dap.nested`.

Method Types

```
Employee co.lang.class ={

    @co.dap.static
    getEmployee()->(Employee) ={}

    @co.dap.instance
    getEmployee()->(Employee)={}

    @co.dap.class
    getEmployee()->(Employee) ={}

    @co.dap.object
    getEmployee()->(Employee)={}
}

@co.dap.oops(
    A: { inherit:true, virtual:true },
    B: { implements:true },
    C: { inherits:true, abstract=true },
    D: { inherits:true },
    E: { uses:true },
    F: { composes:true },
    G: { extends:true },
    H: { with:true },
    I: {assiociate:true},
)
test co.lang.class ={
    getTest(id int)->(test) ={}
}
```

The @@new and @@init Methods

`@@new` and `@@init` are lifecycle methods — compiler-owned, not user-definable outside the class. `@@` signals they are restricted lifecycle symbols, not regular methods.

Lifecycle names are valid only as class members. A unit (including a struct companion unit), module, interface, signature, local block, or package declaration cannot declare a lifecycle-named function.

```
@co.dap.generic(type={T:{typename}, R:{typename}})
Employee co.lang.class = {

    id T;
    name R;

    // @@new is provided by default even if not overridden.
    // Override only when you genuinely need to change allocation behavior.

    @co.dap.class
    @co.dap.private
    @@new()->(co.lang.uninit) = { self.return co.const.none }

    @co.dap.class
    @co.dap.public
    @@new(a co.lang.typevalue, b co.lang.typevalue)->(co.lang.uninit) = {
        // Manual type aliasing — @co.dap.generic handles this automatically
        // Override @@new only when you need custom allocation logic
        T co.lang.type = a;
        R co.lang.type = b;

        // self keyword is allowed only in class methods
        self.parent.new();

        // uninit instance method internally calls new and init
        self.return co.lang.uninit.instance(Employee, self);
    }

    @co.dap.override
    @co.dap.constructor(access=private)
    @@init() = {}

    @co.dap.override
    @co.dap.constructor(access=public)
    @@init(id T, name R) = {
        this.parent.init();
        this.id   = id;
        this.name = name;
    }

    getEmployee(id T)->(Employee) = {}
}
```

Anonymous Classes/Types

```
emp := co.lang.class{};

empObj := emp.init();

empobj1 := co.lang.class{
    name string;
}.init();
```

Interfaces

```
IEmployee co.lang.interface = {
    storeEmployee(emp Employee)->(Employee);
}
```

Signatures

```
// Employee is an ordinary package-level declaration.
MEmployee co.lang.signature = {
    storeEmployee(emp Employee)->(Employee);
}
```

Structurally they look similar — both are lists of contracts. The difference is **who implements them and how**.

	`co.lang.signature`	`co.lang.interface`
Implemented by	module via `matches=`	class via `implements=[]`
Number of implementations	Any number of distinct modules may match one signature	Any number of classes may implement one interface
Runtime cardinality of one implementation declaration	One module object per loaded module identity	Any number of independently constructed class objects
State model	One shared state for all references to the same module	Separate per-instance state for each class object
May specify required values	✅	❌ — methods only
May reference package-level types	✅	✅
May declare abstract/fixed type components	✅	❌
May require generic type constructors	✅	❌
May declare physical nested/local types	❌	❌
May use `@co.dap.local`	❌	❌
Instantiation involved	Module is declared once, not constructed	Class objects are constructed
Reference use	Compatible module references may be used through the signature type without creating another module	Interface references may refer to any implementing object instance
OOP dispatch	❌	✅ virtual/dynamic
Contract style	module values, functions, and type components	behavioral methods on object instances
Practical analogy	singleton component contract	object-instance behavioral contract
Origin	ML/OCaml-inspired modules	Java/C#/Go interfaces

- A `signature` is a **module contract** over values, functions, existing package-level types, and abstract or fixed type components. A type-component specification is a contract slot, not a physical nested type definition. Multiple modules may match one signature, but each module declaration denotes one module object with shared module state.
- An `interface` is a **behavioral contract** tied to class dispatch and polymorphism. It cannot declare module type components or own nested type definitions. A class implementing an interface may create any number of independent runtime objects.
- The approximation `module + signature ≈ singleton object + interface` is useful for understanding cardinality and shared state, but a module is a language-level component rather than a class-based singleton pattern.

Modules

A module is an ML/OCaml-style abstraction governed by an optional signature. A module may use package-level types and may satisfy type components declared by its signature, but it does not physically own or nest arbitrary type declarations. A module should not be introduced merely to prevent functions from appearing loose in a file; use `co.lang.unit` for that simpler structural purpose.

```
// Employee.fol — ordinary package-level type
Employee co.lang.struct = {
    Id   co.lang.int;
    Name co.lang.string;
}

// EmployeeModule.signature.fol
EmployeeModule co.lang.signature = {
    getEmployee(id co.lang.int)->(Employee);
}

// EmployeeModImpl.fol
@co.dap.module(signature=EmployeeModule)
EmployeeModImpl co.lang.module->(signature=EmployeeModule, matches=EmployeeModule) = {

    getEmployee(id co.lang.int)->(Employee) = {
        this.return Employee{
            Id: 10,
            Name: "Rao"
        };
    }
}

mm EmployeeModule = EmployeeModImpl;
v Employee = mm.getEmployee(10);
```

Module Cardinality and Singleton Analogy

A module declaration defines exactly one named module object for that loaded module identity. It is not an instantiable blueprint and does not create a new module object each time its name is referenced.

```
first  EmployeeModule = EmployeeModImpl;
second EmployeeModule = EmployeeModImpl;
```

`first`, `second`, and `EmployeeModImpl` refer to the same module object. These bindings copy or retain the module reference; they do not clone or instantiate the module. Consequently, values declared directly by the module represent one shared module state for all references to that module.

A signature does not itself create a module object. It is a contract, and any number of separately declared modules may conform to the same signature:

```
DatabaseBackend signature
├── PostgreSQLBackend module   -> one named module object
├── MySQLBackend module        -> one named module object
└── TestDatabaseBackend module -> one named module object
```

Each conforming module declaration contributes its own single module object and its own module state. The signature does not restrict the number of distinct conforming module declarations.

A useful analogy is:

```
signature          ≈ interface contract
conforming module  ≈ singleton object implementing that contract
class              ≈ instantiable object type
```

The analogy is intentionally limited. A FoLang module is a compiler-recognized named component, not a class made singleton through a private constructor, static field, or runtime pattern. It cannot be repeatedly constructed. Because module references are first-class in FoLang, they may be bound and used through a compatible signature type, but every reference to the same module declaration still denotes the same module object.

Modules are also broader than ordinary interface implementations. A matching module may provide module values, functions, and abstract, fixed, or generic type-component bindings required by its signature. An interface constrains object behaviour; it does not provide the same module type-component abstraction.

By contrast, a class declaration defines an instantiable type. Every class construction creates a distinct object with independent identity, state, and lifetime:

```
PostgreSQLBackend module
└── one shared module object and module state

PostgreSQLConnection class
├── connection1 -> independent object and state
├── connection2 -> independent object and state
└── connection3 -> independent object and state
```

> **Formal mental model:** A FoLang module is a single named implementation component that may conform to a signature. It is comparable to a singleton object implementing an interface, but it is not instantiated from a class. Multiple distinct modules may conform to the same signature, while each module declaration denotes one module object. Unlike an ordinary singleton-interface implementation, a module may also satisfy abstract, fixed, and generic type components required by its signature.

> **Module instantiation** A FoLang class or struct declaration introduces an instantiable type but does not create an instance. A FoLang module declaration introduces one named module component directly into its package. The module name acts as the binding for that component, so no separate construction expression is required. The module's runtime state is initialized once according to the language's module-initialization rules.

> A module declaration is a singleton component declaration and binding, rather than merely an object definition.

Module Signature Contents

A `co.lang.signature` is a declarative contract for a module. It may specify required module values, functions, and type components. A signature does not allocate storage, initialize variables, execute statements, or provide function bodies.

A signature may contain:

- value specifications
- function signatures
- references to already existing accessible types
- abstract type-component specifications
- fixed or manifest type-component specifications
- abstract generic type-constructor specifications

A signature may not contain:

- value initializers
- executable statements
- function bodies
- concrete class, struct, enum, module, interface, or signature definitions
- arbitrary nested or target-local declarations

Type-component specifications are part of module conformance. They are not Java-, C++-, or C#-style inner types and do not participate in `@co.dap.local`.

Value Specifications

A declaration such as:

```
Counter co.lang.signature = {
    count co.lang.int;
    increment(amount co.lang.int)->();
}
```

requires a matching module to provide a value named `count` of type `co.lang.int` and a compatible `increment` function. The signature does not initialize `count` and does not define `co.lang.int`; the built-in type already exists.

```
CounterImpl co.lang.module->(
    signature=Counter,
    matches=Counter
) = {
    count co.lang.int = 0;

    increment(amount co.lang.int)->() = {
        count.value = count + amount;
    }
}
```

The same rule applies when a value or function specification uses an existing accessible package type:

```
EmployeeRepository co.lang.signature = {
    current hr.employee.Employee;
    find(id co.lang.int)->(hr.employee.Employee);
}
```

The matching module must provide `current` and `find`. It does not redefine `hr.employee.Employee`.

Abstract Type Components

An abstract type component declares that every matching module must supply a type binding for that component:

```
Repository co.lang.signature = {
    Entity co.lang.type;

    current Entity;
    find(id co.lang.int)->(Entity);
}
```

`Entity co.lang.type;` does not define the representation of `Entity`. It defines a required module type component. A matching module binds it to a compatible existing type:

```
EmployeeRepositoryImpl co.lang.module->(
    signature=Repository,
    matches=Repository
) = {
    Entity co.lang.type = hr.employee.Employee;

    current Entity = ...;
    find(id co.lang.int)->(Entity) = { ... }
}
```

Within a matching module, `Entity co.lang.type = ...` is a **signature-component binding**, not an arbitrary nested type declaration. Its name must correspond to an abstract type component declared by the matched signature. A module cannot use this form to introduce unrelated module-local types.

An abstract type component differs from `forward` and `extern` declarations:

```
abstract signature type component
    -> each matching module supplies its own compatible type binding

forward declaration
    -> one specific declaration is completed later

extern declaration
    -> one specific declaration is defined in another linkage or compilation unit
```

Fixed or Manifest Type Components

A signature may fix a type component to an already known type:

```
IntegerRepository co.lang.signature = {
    Id co.lang.type = co.lang.int;

    find(id Id)->(co.lang.bool);
}
```

Here `Id` is predetermined as `co.lang.int`. A matching module uses that type component but does not choose or redefine it.

```
Entity co.lang.type;              -> abstract; implementor supplies the binding
Id co.lang.type = co.lang.int;    -> fixed; signature supplies the type equality
```

Abstract Generic Type Constructors

A signature may require a generic type constructor without defining its representation:

```
StackSignature co.lang.signature = {
    Stack(T) co.lang.type;

    empty(T)->(Stack(T));
    push(value T, stack Stack(T))->(Stack(T));
    pop(stack Stack(T))->(T, Stack(T));
}
```

`Stack(T) co.lang.type;` declares an **abstract generic type component**, also described as an abstract type constructor of arity one. The signature specifies that `Stack` accepts one type argument, but it does not define what `Stack(T)` is.

A matching module must provide a compatible type-constructor binding with the same name, arity, and declared constraints:

```
ListStackModule co.lang.module->(
    signature=StackSignature,
    matches=StackSignature
) = {
    Stack(T) co.lang.type = co.core.list(T);

    empty(T)->(Stack(T)) = { ... }
    push(value T, stack Stack(T))->(Stack(T)) = { ... }
    pop(stack Stack(T))->(T, Stack(T)) = { ... }
}
```

Another matching module may choose another representation:

```
ArrayStackModule co.lang.module->(
    signature=StackSignature,
    matches=StackSignature
) = {
    Stack(T) co.lang.type = collections.ArrayStack(T);
    ...
}
```

Therefore:

```
StackSignature
    -> requires a generic type constructor Stack(T)

ListStackModule
    -> binds Stack(T) to co.core.list(T)

ArrayStackModule
    -> binds Stack(T) to collections.ArrayStack(T)
```

A signature-component binding does not permit physical type nesting. When an implementation needs a new concrete struct, class, or enum representation, that declaration remains an ordinary package declaration and may be restricted to the implementing module with `@co.dap.local`; the module then binds the signature component to that declaration.

Signature Conformance Rules for Type Components

For every type component in a matched signature:

- an abstract component must receive exactly one compatible module binding
- a fixed component must retain the type equality declared by the signature
- a generic component binding must preserve generic arity, parameter kinds, bounds, variance, and other declared constraints
- component names must be unique within the signature
- component bindings cannot contain executable code
- extra module-local type bindings that do not correspond to signature components are compiler errors
- types referenced by value and function specifications must resolve after applying the module's type-component bindings

Module Declaration Relationships

A module cannot physically declare nested structs, enums, classes, modules, signatures, interfaces, or other arbitrary named declarations. It references ordinary package-level declarations through its functions and signature. The only type-like declarations permitted directly in a matching module are signature-component bindings that satisfy abstract type components declared by its matched signature; such bindings do not create independent nested declarations.

A declaration intended only for one module may be restricted with `@co.dap.local`:

```
// EmployeeModuleConfig.fol
@co.dap.local(for=hr.employee.EmployeeModImpl)
EmployeeModuleConfig co.lang.struct = {
    timeout co.lang.int;
    retries co.lang.int;
}
```

```
// EmployeeModImpl.fol
EmployeeModImpl co.lang.module = {
    connect(cfg EmployeeModuleConfig)->(co.lang.bool) = {
        ...
    }
}
```

The following remains invalid:

```
EmployeeModImpl co.lang.module = {
    Config co.lang.struct = { // ❌ physical nested declaration
        timeout co.lang.int;
    }
}
```

```
A target-local declaration does not automatically become a module member name and is not projected through the module's signature. It
becomes part of the signature view only when an explicit signature type component is bound to it or a signature value/function
specification references it through an allowed type component.
```

Structs vs Classes vs Modules vs Units vs Packages

	Struct	CStruct	Class	Module	Unit	Package
Purpose	Pure data shape	C-like value type	Behaviour + data	Signature-backed ML-style abstraction	Named function container; optionally a struct companion	Folder-based grouping
Fields	✅	✅ simple only	✅ per instance	❌	❌	❌
Module-level values	❌	❌	❌	✅ when declared directly or required by a signature	❌	❌

Functions / methods	Optional static-like functions whose first parameter is the struct, plus receiver-based associated functions, through a matching companion unit	❌	✅ methods	✅ module functions	✅ receiverless functions; associated functions only when matching a struct	❌
Lifecycle (`@@new` / `@@init`)	❌	❌	✅	❌	❌	❌
`this` / `self`	❌	❌	✅	❌	❌	❌
Value/literal construction	✅	✅	❌	❌	❌	❌
Lifecycle instantiation (`new` / `init`)	❌	❌	✅ multiple objects	❌ — one module object per declaration	❌	❌
Runtime state cardinality	Per bound struct object	Per value	Per class object	One shared state for the module declaration	—	—
First class	✅	✅	✅	✅ module reference; referencing does not instantiate	❌	❌
Pass by	Reference	Value	Reference	Reference to the same module object	—	—
Contract	—	—	`interface` via `implements=[]`	`signature` via `matches=`	none	—
OOP / inheritance	❌	❌	✅	❌	❌	❌
Physically nested independent named type/container declarations	❌	❌	❌	❌	❌	N/A — packages contain separate source declarations
May be an `@co.dap.local` target	✅	❌	✅	✅	❌	❌
Pattern matching	✅	✅	✅	❌	❌	❌
Direct ABI / zone boundary safe	❌ — library boundaries require snapshots	✅	❌	❌	❌	❌
Associated functions	✅ through matching companion unit	❌	—	—	✅ only in a struct companion unit	❌
Embedding	✅	❌	—	—	❌	❌
Declared with	`co.lang.struct`	`co.lang.cstruct`	`co.lang.class`	`co.lang.module`	`co.lang.unit`	folder path
C++ backend analogy	struct without methods	plain C struct	class	struct/class abstraction	namespace or static function scope	namespace
Closest mental model	Rust struct	C struct	Java/C# class	singleton implementation component with ML-style type members	named function scope; optional struct companion	filesystem namespace

Mental model:

```
reach for struct   → pure data; use a same-name companion unit for behaviour
reach for cstruct  → physical ABI-compatible value data crossing direct zone or native boundaries
reach for class    → behaviour, lifecycle, multiple instances
reach for module   → one named implementation component with shared state, governed by an optional signature and capable of satisfying
type components
reach for unit     → named function container; same-name struct unit acts as companion
reach for package  → folder-based grouping only, not a value
```

> **Declaration scoping rule:** FoLang does not permit physical nesting of independent named type and container declarations. Classes, structs, cstructs, enums, unions, modules, units, interfaces, signatures, and other package-owned primary declarations remain in their ordinary legal source locations. Two explicit exceptions exist: an ordinary named local function may be declared inside a function body where local-function declarations are permitted, and anonymous constructs may appear wherever their expression grammar permits. Anonymous functions, lambdas and callback blocks, anonymous class/type expressions, and permitted `forall` type expressions do not create independent package-level nested declaration identities. Supported package declarations may restrict visibility to one or more exact targets with `@co.dap.local`. The annotation accepts either one declaration reference or a non-empty closed target list. The local declaration and every target must belong to the same exact folder-derived package; parent and subpackages are different packages. Non-function targets are identified by complete qualified name; overloaded function targets are identified by complete qualified signature. Visibility is the union of the explicitly listed target scopes and is neither inherited nor transitive. Signatures and interfaces cannot own or target local declarations. A signature may nevertheless declare abstract, fixed, and generic type components as module-conformance requirements; these are contract slots rather than physical nested declarations. A matching module may bind only those declared components. Units contain functions only, and a struct companion unit remains a separate declaration from its struct. ***

Local and/or Nested types and functions

FoLang does not provide Java-, C++-, or C#-style physical nesting of independent named type and container declarations. Such declarations remain in their ordinary legal source locations. Ordinary local functions and anonymous expressions are explicit exceptions governed by the rules below.

Physical Nesting Rules

Prohibited Independent Named Declarations

The following declarations cannot be physically declared inside another class, struct, cstruct, enum, union, module, unit, interface, signature, function, or executable block:

- named classes, structs, cstructs, enums, and unions;
- named modules, units, interfaces, and signatures;
- named type aliases, newtypes, opaque types, subtypes, and supertypes;
- named instances, matchers, macros, templates, and other package-owned primary declarations.

These declarations retain package-owned identity and follow their normal source-placement rules. An association or visibility annotation such as `@co.dap.local`, `@co.dap.nested`, or `@co.dap.inner` does not physically move a separately declared declaration inside its target.

Local-Function Exception

A function body may contain an ordinary named local or inner function where the grammar permits a local-function declaration:

```
outer()->() = {
    value co.lang.int = 10;

    inner()->() = {
        co.out.println(value);
    }

    inner();
}
```

The local function has block-local declaration identity. Its free runtime names are resolved from its lexical declaration context, and its lifetime and escape behavior follow the ordinary inner-function rules. It is not a package member and cannot be independently imported or exported.

This exception permits local functions only. It does not permit a named class, struct, enum, module, unit, interface, signature, or another named type/container declaration inside a function body.

Anonymous-Expression Exception

The physical-nesting restriction does not apply to anonymous constructs that are expressions or type expressions rather than independent named declarations. They may be nested wherever their specific grammar category is permitted.

These include:

- anonymous function expressions;
- lambdas and callback blocks;
- anonymous class or anonymous type expressions;
- permitted anonymous polymorphic type expressions introduced by `forall`;
- ordinary nested block, object-construction, map, collection, and other value-producing expressions.

```
process()->() = {
    operation := (value co.lang.int)->(co.lang.int) = {
        this.return value * 2;
    };

    worker := co.lang.class {
        run(value co.lang.int)->(co.lang.int) = {
            this.return operation(value);
        }
    }.init();
}

transformer co.lang.type = forall(T).(T)->(T);
```

An anonymous construct has no independently addressable package declaration identity. Its scope, capture, lifetime, type, and escape behavior are determined by the rules for that specific construct. Syntactic containment of an anonymous expression does not create a Java-, C++-, or C#-style named nested declaration and does not violate the one-primary-declaration-per-package-file rule.

Relationship to Association Annotations

The exceptions above are distinct from FoLang's association annotations:

- an ordinary local function is physically declared inside an enclosing function and is lexically scoped;
- an anonymous construct is an expression without independent declaration identity;
- `@co.dap.local`, `@co.dap.nested`, and `@co.dap.inner` apply to separately declared declarations that retain their normal source identity.

The remainder of this section defines those separately declared association forms.

```
@co.dap.local(for=<declaration-reference>)
```

or:

```
@co.dap.local(
    for=[
        <declaration-reference>,
        <declaration-reference>
    ]
)
```

The annotated declaration is called a **target-local declaration**. The declarations named by `for` form its **local target set**.

Supported Declaration and Target Kinds

`@co.dap.local` may be applied only to these declaration kinds:

- `co.lang.class`
- `co.lang.struct`
- `co.lang.enum`
- `co.lang.module`
- a named function declared in a context that normally permits functions

Every entry in the local target set must resolve to exactly one:

- class
- struct
- enum
- module
- function overload

A target list may contain any combination of supported target kinds.

Signatures and interfaces do not support target-local or physically nested declarations. A `co.lang.signature` or `co.lang.interface` cannot be annotated with `@co.dap.local` and cannot appear in a local target set. A signature may declare abstract, fixed, or generic **type-component specifications** as part of its module contract; these are contract requirements rather than local or nested type definitions. Interfaces do not declare signature type components.

Other declaration kinds are not target-local unless a later section explicitly permits them.

Target-Set Syntax

The `for` argument accepts either:

1. one declaration reference; or
2. a non-empty list of declaration references.

The scalar form is equivalent to a singleton target list:

```
@co.dap.local(for=hr.employee.Employee)
```

```
@co.dap.local(for=[hr.employee.Employee])
```

The scalar form is canonical when only one target is required. Use the list form when two or more declarations require access:

```
@co.dap.local(
    for=[
        hr.employee.Employee,
        hr.employee.EmployeeService,
        hr.employee.EmployeeValidator
    ]
)
EmployeeState co.lang.enum = {
    Active,
    Inactive
}
```

Rules:

- the target list must contain at least one declaration reference
- each reference must resolve independently and unambiguously
- duplicate references to the same resolved declaration are a compiler error
- list order does not affect visibility or declaration identity
- the target list is a closed set; access is not granted to declarations omitted from it
- repeated `@co.dap.local` annotations are not used to accumulate targets; use one annotation with one complete target list

Invalid:

```
@co.dap.local(for=[])
State co.lang.struct = { ... }
// ❌ empty target list
```

```
@co.dap.local(
    for=[
        hr.employee.Employee,
        hr.employee.Employee
    ]
)
State co.lang.struct = { ... }
// ❌ duplicate resolved target
```

Declaration-Reference Syntax

Each `for` entry is a compiler-resolved declaration reference, not a string, runtime expression, function call, or `co.meta.symbol` value.

For a non-function target, use only its complete qualified declaration name:

```
@co.dap.local(for=hr.employee.Employee)
@co.dap.local(for=hr.employee.EmployeeStatus)
@co.dap.local(for=hr.employee.EmployeeRules)
```

For a function target, use its complete qualified function signature because FoLang permits overloads:

```
@co.dap.local(
    for=hr.employee.Employee.calculate(co.lang.decimal)->()
)
```

Function references in a target list follow the same rule:

```
@co.dap.local(
    for=[
        hr.employee.Employee.calculate(co.lang.decimal)->(),
        hr.employee.Employee.calculate(co.lang.int)->(),
        hr.employee.Employee.validate()->(co.lang.bool)
    ]
)
CalculationState co.lang.struct = { ... }
```

Parameter names are not part of the reference:

```
// ✅ canonical
hr.employee.Employee.calculate(co.lang.decimal)->()

// ❌ parameter names are not declaration identity
hr.employee.Employee.calculate(amount co.lang.decimal)->()
```

The parameter and return types must match one exact overload. An abbreviated function name, unresolved target, or ambiguous overload is a compiler error.

```
@co.dap.local(for=hr.employee.Employee.calculate)
// ❌ ambiguous when calculate is overloaded
```

Each listed overload is an independent target. Listing one overload does not grant access to sibling overloads with the same function name.

Same-Package Requirement

The target-local declaration and **every declaration in its local target set** must belong to the same exact package. The compiler compares their complete folder-derived package identities; matching only a parent package, package family, import alias, realm, library, or source root is not sufficient.

For a function target, the target package is the package that owns the function's enclosing class, struct companion unit, module, unit, or other legal function container. A target-local function is checked in the same way: the package containing its legal function-owning declaration must match the package of every listed target.

The invariant is:

```
package(target-local declaration)
    == package(target 1)
    == package(target 2)
    == ...
```

```
// hr/employee/Employee.fol
Employee co.lang.class = { ... }

// hr/employee/EmployeeService.fol
EmployeeService co.lang.class = { ... }

// hr/employee/EmployeeState.fol
@co.dap.local(
    for=[
        hr.employee.Employee,
        hr.employee.EmployeeService
    ]
)
EmployeeState co.lang.enum = { Active, Inactive }
// ✅ all declarations belong to package hr.employee
```

A declaration in another package cannot participate in the local target set, even when ordinary package visibility, imports, aliases, parent/subpackage relationships, or library membership would otherwise make the declaration resolvable.

```
// EmployeeState is declared in package hr.internal
@co.dap.local(
    for=[
        hr.employee.Employee,
        hr.internal.EmployeeService
    ]
)
EmployeeState co.lang.enum = { Active, Inactive }
// ❌ local declaration and every target must have the same package
```

```
// CalculationState is declared in package hr.employee.internal
@co.dap.local(
    for=hr.employee.Employee.calculate(co.lang.decimal)->()
)
CalculationState co.lang.struct = { ... }
// ❌ subpackages are distinct packages; both sides must be exactly hr.employee
```

The same-package rule prevents `@co.dap.local` from becoming a cross-package friendship or visibility-bypass mechanism. A local declaration is a selectively shared implementation detail inside one package, not an imported helper attached from elsewhere.

Visibility Domain

A target-local declaration may be resolved only from:

1. each exact declaration in its local target set; and
2. lexical scopes contained within each listed target's implementation.

For targets `A`, `B`, and `C`, the visibility domain is the union of their implementation scopes:

```
visibility(local declaration)
    = scope(A) ∪ scope(B) ∪ scope(C)
```

It is not an intersection, and code does not need to be simultaneously inside every target.

Consequences:

- a declaration local to a class is available to that class body and its methods
- a declaration local to a module is available to that module body and its functions
- a declaration local to a struct is available while resolving that struct declaration
- a declaration local to an enum is available while resolving that enum declaration
- a declaration local to a function is available only in the body of the exact targeted overload and its nested statement scopes
- when several targets are listed, each listed target receives access independently
- sibling declarations and sibling overloads not listed cannot resolve it
- subclasses, companion units, extensions, callees, callers, and related declarations do not gain access automatically
- visibility is not inherited, transitive, or propagated through calls, composition, embedding, imports, or other local declarations
- it cannot be imported, exported, or projected through a library surface

```
@co.dap.local(
    for=[
        hr.employee.Employee,
        hr.employee.EmployeeService
    ]
)
EmployeeState co.lang.enum = {
    Active,
    Inactive
}

Employee co.lang.class = {
    state EmployeeState; // ✅ listed target
}

EmployeeService co.lang.class = {
    state EmployeeState; // ✅ listed target
}

Payroll co.lang.class = {
    state EmployeeState; // ❌ not listed
}
```

Interaction with Ordinary Visibility

`@co.dap.local` establishes the final visibility boundary. Ordinary visibility annotations such as `@co.dap.public`, `@co.dap.package`, `@co.dap.protected`, or `@co.dap.private` cannot widen the declaration beyond its closed local target set.

```
@co.dap.public
@co.dap.local(
    for=[
        hr.employee.Employee,
        hr.employee.EmployeeService
    ]
)
EmployeeState co.lang.enum = { Active, Inactive }
```

`EmployeeState` is still visible only to the listed targets. The ordinary visibility annotation is redundant for external resolution and may produce a compiler warning, but it never overrides `@co.dap.local`.

When most or all declarations in a package require access, ordinary package visibility should be used instead of maintaining a large target list.

Source Placement and Identity

A target-local declaration remains in the source position normally required for its declaration kind:

- a class, struct, enum, or module remains a normal primary declaration and follows the one-primary-declaration-per-package-file rule
- a function remains inside a unit, class, module, or another declaration context that normally permits functions
- `@co.dap.local` does not permit a free function to appear loose at package-file scope

The declaration retains a stable package-owned compiler identity. Its package identity must be exactly the same as the package identity of every target. It does not become physically nested and is not addressed as `Target.LocalName`.

```
hr.employee.EmployeeState
    local-for [
        hr.employee.Employee,
        hr.employee.EmployeeService
    ]
```

The normalized local target set is declaration metadata. Target-list order does not create a different declaration identity.

No Implicit Relationship or Privilege

`@co.dap.local` changes visibility only. It does not imply:

- composition
- embedding

- inheritance
- lifecycle ownership
- memory ownership
- friendship or privileged private-member access
- automatic membership in any listed target
- shared runtime state among the targets

Composition must still be written explicitly through a field or another supported relation. Embedding remains a separate facility: only struct and enum declarations are eligible for embedding, and each use must follow the embedding rules defined for that declaration kind.

Escape Restrictions

A target-local type must not leak outside its complete visibility domain.

It cannot appear in:

- a public, package, or protected API visible outside the local target set
- a library surface
- the parameter or return types of a function to which it is local
- a field or method signature that exposes it beyond the local target set
- an exported generic specialization or type alias

A value whose static type is target-local must be converted to an externally visible type before leaving the union of the listed targets' visibility domains.

Usage

```
@co.dap.public
@co.dap.local(
    for=[
        hr.employee.Employee,
        hr.employee.EmployeeService
    ]
)
EmployeeState co.lang.enum = { Active, Inactive }

Employee co.lang.struct={

    state EmployeeState;
}
```

Invalid Physical Nesting

```
Employee co.lang.class = {
    Address co.lang.struct = { ... } // ❌
}

EmployeeModule co.lang.module = {
    Config co.lang.struct = { ... } // ❌
}

process()->() = {
    State co.lang.enum = { Ready, Done } // ❌ named type declaration
}

EmployeeContract co.lang.signature = {
    Employee co.lang.struct; // ❌
}

EmployeeApi co.lang.interface = {
    Result co.lang.struct; // ❌
}
```

The `process` example rejects the named enum declaration; it does not reject ordinary local functions or anonymous expressions. Those are permitted only under the explicit exceptions in Physical Nesting Rules.

Use separately declared package declarations, composition, embedding where allowed, and `@co.dap.local` when a closed set of declarations requires selective access.

There is another annotation `@co.dap.nested` which is similar to local but captures target state.

@co.dap.nested(target=hr.emp.Employee)

Instead of `for` we use `target` and `target` is always a single fully qualified type/function.

`@co.dap.nested` bechaves exactly like nested or inner classes or functions

Comparision Table

Annotation	attribute	multiple targets	capture the target state

@co.dap.local	for	✅ as list for single target can mention without list syntax	❌
@co.dap.nested	target	❌	✅

`@co.dap.inner` Declarations

`@co.dap.inner` is an association annotation. It does **not** create a physically nested type, method, or function. The annotated declaration remains in its ordinary legal source location, retains its package-owned identity, and must satisfy the same exact-package rule that applies to `@co.dap.local` and `@co.dap.nested`.

Unlike `@co.dap.local` and `@co.dap.nested`, `@co.dap.inner` does not contain a `for` or `target` attribute. Its target relationship is established at the use site by explicitly embedding or attaching the declaration to a legal target. An `@co.dap.inner` declaration cannot be used as a standalone declaration.

Usage

```
@co.dap.public
@co.dap.inner
EmployeeState co.lang.enum = {
    Active,
    Inactive
}

Employee co.lang.struct = {
    EmployeeState;
    state EmployeeState;
}
```

The use of `EmployeeState;` establishes the inner association for this target. The annotation itself does not name the target.

Scope of `@co.dap.inner` Executable Declarations

For an `@co.dap.inner` function or method, parameters and local declarations resolve normally. A free runtime name is resolved from the **lexical context of the active call or attachment site**, not from the declaration site of the separately declared `@co.dap.inner` function.

The lookup order is:

```
1. parameters and local declarations of the @co.dap.inner function or method
2. the innermost lexical scope at the active call or attachment site
3. enclosing lexical scopes of that call or attachment site
4. statically resolved types, annotations, imports, and built-in co.* names
5. compiler error
```

This model is called **call-site lexical-context resolution**.

It is distinct from an ordinary inner function. An ordinary inner function is physically declared inside another function and resolves free runtime names from its enclosing lexical **declaration** context. An `@co.dap.inner` function is declared separately and therefore obtains its runtime context from the lexical scope in which it is actively attached or called.

It is also distinct from `@co.dap.dynamicscope`. Call-site lexical-context resolution does not search arbitrary runtime caller frames beyond the lexical scope chain of the active call site, and it does not use mixed-scope fallback. At every statically known use or call site, the compiler validates that each required runtime binding exists, is definitely initialized, has a compatible type, and permits the requested access or mutation.

For `@co.dap.inner` types and other non-executable declarations, ordinary static name and type resolution continues to apply. Call-site lexical-context resolution concerns only free runtime names used by executable function or method bodies.

An implementation may lower an `@co.dap.inner` declaration by copying, inlining, specialization, or another equivalent mechanism. Such lowering is not observable language semantics; the association and scope rules above are normative.

Exception

> `@co.dap.inner`, `@co.dap.local`, and `@co.dap.nested` cannot be used with `co.lang.cstruct`.

Statements

A statement is a complete executable or declarative instruction. It may contain one or more expressions and may change program state, control execution, introduce declarations, or produce observable effects.

Common statement categories in `Folang`

1. Declaration Statement
2. Initialization Statement
3. Expression Statement
4. Conditional Statement
5. Loop Statement etc,.

Expressions

An expression is a construct that evaluates to a value, produces an effect, or both, and may be contained within a larger expression or statement.

Expression Evaluation Order

FoLang defines a deterministic evaluation order for expressions.

Expression structure is determined first by:

1. explicit grouping with parentheses;
2. operator precedence;
3. operator associativity.

After the expression structure has been determined, operands and subexpressions are evaluated from left to right in their source-code order, except where a construct explicitly defines conditional, lazy, asynchronous, concurrent, or otherwise specialized evaluation behavior.

Ordinary Expressions

For an ordinary expression:

```
result = first() + second();
```

the evaluation order is:

1. evaluate `first()`;
2. evaluate `second()`;
3. apply the `+` operator;
4. assign the resulting value to `result`.

Precedence and Evaluation Order

Left-to-right evaluation does not override operator precedence.

For example:

```
result = first() + second() * third();
```

operator precedence determines the expression structure as:

```
result = first() + (second() * third());
```

The function calls are evaluated in source order:

1. evaluate `first()`;
2. evaluate `second()`;
3. evaluate `third()`;
4. multiply the results of `second()` and `third()`;
5. add the multiplication result to the result of `first()`;
6. assign the final result to `result`.

Therefore, precedence determines how values are combined, while evaluation order determines when each operand or subexpression is evaluated.

Function Calls

For a function call, the callable expression is evaluated first, followed by the arguments from left to right.

```
result = calculate(first(), second(), third());
```

The evaluation order is:

1. evaluate the callable expression `calculate`;
2. evaluate `first()`;
3. evaluate `second()`;
4. evaluate `third()`;
5. invoke `calculate` with the resulting argument values;
6. assign the returned value to `result`.

Name resolution performed at compile time does not constitute runtime evaluation.

Method Calls

For a method call, the receiver expression is evaluated first, followed by the arguments from left to right.

```
result = getEmployee().calculate(first(), second());
```

The evaluation order is:

1. evaluate `getEmployee()`;
2. resolve the resulting object as the method receiver;
3. evaluate `first()`;
4. evaluate `second()`;
5. invoke `calculate`;
6. assign the returned value to `result`.

The receiver expression must be evaluated exactly once.

Member Access

For member access, the expression producing the containing object is evaluated before the member is accessed.

```
value = getEmployee().address.city;
```

The evaluation order is:

1. evaluate `getEmployee()`;
2. access `address` on the resulting object;
3. access `city` on the resulting address;
4. assign the resulting value to `value`.

Each intermediate receiver is evaluated only once.

Indexing

For an indexing expression, the indexed object is evaluated first, followed by index expressions from left to right.

```
value = getMatrix()[row()][column()];
```

The evaluation order is:

1. evaluate `getMatrix()`;
2. evaluate `row()`;
3. perform the first indexing operation;
4. evaluate `column()`;
5. perform the second indexing operation;
6. assign the resulting value to `value`.

Assignment

For assignment, FoLang evaluates the assignment target first, then evaluates the right-hand-side expression, and finally performs the assignment.

```
array[index()] = calculate();
```

The evaluation order is:

1. evaluate `array`;
2. evaluate `index()`;
3. determine the destination location;
4. evaluate `calculate()`;
5. assign the resulting value to the destination.

Determining the assignment target does not read the previous value stored at that location unless the assignment operation explicitly requires it.

Simple Variable Assignment

For assignment to a simple variable:

```
result = first() + second();
```

the evaluation order is:

1. determine the binding represented by `result`;
2. evaluate `first()`;
3. evaluate `second()`;
4. apply the `+` operator;
5. assign the resulting value to `result`.

Compound Assignment

A compound assignment evaluates its target exactly once.

```
array[index()] += calculate();
```

The evaluation order is:

1. evaluate `array`;
2. evaluate `index()`;
3. determine the destination location;
4. read the current value from that location;
5. evaluate `calculate()`;
6. apply the `+` operator;
7. assign the resulting value to the same destination.

The expression above must not behave as though it were expanded into a form that evaluates `array` or `index()` more than once.

Multiple Assignment

When an assignment contains multiple right-hand-side expressions, those expressions are evaluated completely from left to right before values are assigned to their targets.

```
a, b = first(), second();
```

The evaluation order is:

1. determine the target for `a`;
2. determine the target for `b`;
3. evaluate `first()`;
4. evaluate `second()`;
5. assign the first resulting value to `a`;
6. assign the second resulting value to `b`.

Right-hand-side evaluation must complete before any target receives its new value.

This permits value exchange without an explicit temporary variable:

```
a, b = b, a;
```

Operator Expressions

Built-in and user-defined operators follow the same operand-evaluation rules.

For a binary operator:

```
left() + right()
```

FoLang evaluates `left()` before `right()`.

For a prefix unary operator:

```
-operand()
```

FoLang evaluates `operand()` before applying the operator.

For a postfix unary operator:

```
operand()!
```

FoLang evaluates `operand()` before applying the operator.

Operator overloading must not change the specified evaluation order of operands.

Short-Circuit Boolean Operations

Short-circuit Boolean operators evaluate the left operand first.

The right operand is evaluated only when it is required to determine the result.

For logical AND:

```
left() && right()
```

the evaluation order is:

1. evaluate `left()`;
2. when the result is false, return false without evaluating `right()`;
3. otherwise, evaluate `right()` and use its Boolean result.

For logical OR:

```
left() || right()
```

the evaluation order is:

1. evaluate `left()`;
2. when the result is true, return true without evaluating `right()`;
3. otherwise, evaluate `right()` and use its Boolean result.

A skipped operand produces no value, mutation, side effect, or error.

Conditional Expressions

A conditional expression evaluates its condition first and then evaluates exactly one selected result expression.

```
result = condition()
    .return(whenTrue())
    .otherwise.return(whenFalse());
```

The evaluation order is:

1. evaluate `condition()`;
2. when the condition is true, evaluate `whenTrue()` only;
3. otherwise, evaluate `whenFalse()` only;
4. assign the selected result to `result`.

The unselected expression must not be evaluated.

Conditions and Branches

For a condition-and-branch construct, conditions are evaluated sequentially from left to right.

```
firstCondition().do({
    firstBranch();
}).otherwise(secondCondition()).do({
    secondBranch();
}).otherwise.do({
    finalBranch();
});
```

The evaluation order is:

1. evaluate `firstCondition()`;
2. when true, execute `firstBranch()` and skip the remaining conditions and branches;
3. otherwise, evaluate `secondCondition()`;
4. when true, execute `secondBranch()` and skip the final branch;
5. otherwise, execute `finalBranch()`.

Only the selected branch is evaluated.

Pattern Matching

A pattern-matching subject is evaluated exactly once.

Cases are examined in source order unless a particular matcher explicitly defines another ordering rule.

```
getValue().match
    .case(firstPattern => firstResult())
    .case(secondPattern => secondResult())
    .default(defaultResult());
```

The evaluation order is:

1. evaluate `getValue()` exactly once;
2. test `firstPattern`;
3. when it matches, evaluate `firstResult()` and stop;
4. otherwise, test `secondPattern`;
5. when it matches, evaluate `secondResult()` and stop;
6. otherwise, evaluate `defaultResult()`.

Results belonging to unmatched cases are not evaluated.

Pattern guards are evaluated only after their corresponding structural pattern has matched.

Collection Literals

Elements of an array, list, tuple, set, map, or other collection literal are evaluated from left to right as they appear in the source.

```
values = [first(), second(), third()];
```

The evaluation order is:

1. evaluate `first()`;
2. evaluate `second()`;
3. evaluate `third()`;
4. construct the collection from the resulting values.

For a map entry, the key expression is evaluated before its corresponding value expression.

```
values = {
    firstKey(): firstValue(),
    secondKey(): secondValue()
};
```

The evaluation order is:

1. evaluate `firstKey()`;
2. evaluate `firstValue()`;
3. evaluate `secondKey()`;
4. evaluate `secondValue()`;
5. construct the map.

Range Expressions

For a bounded range, the lower-bound expression is evaluated before the upper-bound expression.

```
range = lower() .. upper();
```

The evaluation order is:

1. evaluate `lower()`;
2. evaluate `upper()`;
3. construct the range.

An omitted bound is not evaluated and does not produce an implicit function call or side effect.

Comprehensions and Iteration

A comprehension or iteration construct follows its own defined iteration semantics.

Within each iteration:

- source expressions are evaluated in the order declared;
- filter conditions are evaluated before result expressions;
- a result expression is evaluated only when its filters succeed;
- individual operand evaluation remains left to right.

The language does not implicitly evaluate comprehension iterations concurrently unless concurrency is explicitly requested.

Lazy Expressions

An expression declared lazy is not evaluated when the lazy binding is created.

Its evaluation occurs only when demanded according to the rules of the corresponding lazy construct.

Once evaluation begins, the expression itself follows the normal FoLang evaluation-order rules unless the lazy construct specifies otherwise.

The specification for each lazy construct must also state whether its result is:

- evaluated once and cached;
- evaluated again for every demand;
- safe for concurrent demand;
- permitted to propagate errors more than once.

Asynchronous and Concurrent Expressions

Left-to-right evaluation determines the order in which asynchronous or concurrent operations are created, invoked, or submitted.

It does not necessarily determine the order in which independently executing operations complete.

```
submit(first());
submit(second());
```

FoLang evaluates and submits `first()` before `second()`. However, completion order depends on the execution-model rules unless explicit ordering is requested.

Concurrent execution never arises implicitly from an ordinary expression. It must be introduced by an explicit FoLang concurrency, parallelism, asynchronous-execution, scheduling, or execution-model construct.

Errors During Evaluation

When evaluation of a subexpression produces an error that prevents further evaluation, later subexpressions are not evaluated.

```
result = first() + failing() + third();
```

When `failing()` terminates evaluation with an error:

1. `first()` has already been evaluated;
2. `failing()` produces the error;
3. `third()` is not evaluated;
4. no final value is assigned to `result`.

The applicable error-handling rules determine whether the error is propagated, matched, converted, recovered from, or terminates execution.

Side Effects

All observable side effects occur according to the defined evaluation order.

Observable effects include:

- object mutation;
- variable rebinding;
- input and output;
- file-system operations;
- network operations;
- synchronization;
- exception or error generation;
- interaction with foreign code;
- interaction with the runtime environment.

An implementation must not reorder operations when doing so could change any observable effect.

Compiler Optimizations

A compiler may optimize, combine, eliminate, or internally reorder operations only when the transformation preserves all behavior required by this specification.

In particular, an optimization must not change:

- the resulting value;
- the order or presence of observable side effects;
- the identity or aliasing behavior of objects;
- the timing or presence of specified errors;
- the selected conditional or pattern-matching branch;
- the number of times an effectful expression is evaluated;
- synchronization or concurrency guarantees.

Pure internal operations may be reordered when no conforming FoLang program can observe the difference.

Implementation Requirement

Every conforming FoLang implementation must preserve the evaluation order defined in this section.

An implementation may use any parser, intermediate representation, optimizer, runtime, or backend, but those implementation choices must not alter the externally observable behavior required by these rules.

Operators

FoLang distinguishes a **symbolic spelling** from an **operator**. A sequence of symbol characters is not automatically an operator merely because it resembles one. Its syntactic role is determined by the grammar context in which the whole spelling occurs.

For example:

```
a co.lang.int->(**);
```

In this declaration, `->` is the structural type-derivation marker and `**` is pointer-degree metadata inside the derivation. Neither occurrence is parsed as an expression operator. The same `**` spelling in `left ** right` is parsed as the registered power operator because it occurs in an operator-expression position.

FoLang uses one uniform implementation model for every expression operator. The difference between built-in, pre-declared, and project-local custom operators is only who registers the symbol and its parse properties.

```
built-in operator
    symbol and parse properties registered by the language
    one or more implementations may already exist

pre-declared operator glyph
    symbol and parse properties registered by the language
    no implementation is required to exist

project-local custom operator
    symbol and parse properties registered once in operators.fol
    no implementation is contained in the operator declaration

all three categories
    implementations are ordinary mode=overload operator functions
    duplicate normalized operand signatures are errors
```

Symbolic Runs, Classification, and Boundaries

After comments, literals, and closed scanner-known composite spellings such as `@@new` and `@@init` are recognized, the lexer consumes each remaining complete contiguous run of symbol characters as one symbolic token candidate. It does not backtrack or split an unrecognized run into shorter valid tokens. Whitespace, comments, and delimiters such as `(`, `)`, `[`, `]`, `{`, and `}` end a symbolic run. Comment openers are recognized before ordinary symbolic-run scanning.

The complete run is then classified by its grammar context as one of the following:

1. a fixed structural or declaration spelling such as `->`, `=>`, `:=`, or `?=`;
2. a contextual metadata spelling, such as a contiguous run of one or more `*` characters inside `->(...)` to express pointer degree;
3. a registered expression operator; or
4. an unrecognized symbolic token, which is a compilation error.

There is no fallback splitting. Therefore, when `++` is not registered:

```
++ a;       // error: unrecognized symbolic token `++`
a ++ b;     // error: unrecognized symbolic token `++`
a++b;       // error: the contiguous run is still the single candidate `++`

+ +a;       // valid: two `+` operators separated by whitespace
+(+a);      // valid: `(` creates the token and operand boundary
a + +b;    // valid: binary `+` followed by unary `+`
```

A registered expression operator whose spelling contains more than one symbol character must have an explicit boundary on every operand-facing side. A boundary may be whitespace, a comment, or an applicable delimiter. Boundary presence is checked from the original source before whitespace and comments are discarded. The rule is based on the operator's fixity:

- an infix multi-symbol operator requires a boundary before and after it;
- a prefix multi-symbol operator requires a boundary after it;
- a postfix multi-symbol operator requires a boundary before it.

Suppose `+-` is registered as an infix operator:

```
a +- b;       // valid
(a)+-(b);     // valid: the parentheses provide both operand boundaries
a+-b;         // error: missing boundaries
a +-b;        // error: missing the right boundary
a+- b;        // error: missing the left boundary
a + -b;       // valid: separate `+` and unary `-` operators
a + (-b);     // valid: the parenthesis separates the operators
```

This boundary requirement applies uniformly to built-in, pre-declared, and custom multi-symbol expression operators. It does not apply merely because a structural token or metadata spelling contains multiple symbols. Thus `co.lang.int->(**)` remains valid without spaces around `->` or inside the pointer metadata.

The statement-level definition spellings `:=` and `?=` are not expression operators, but they deliberately use the analogous two-sided boundary rule. Write `name := value` and `name ?= value`; compact forms such as `name:=value` and `name?=value` are invalid.

Operator `mode=override` and `mode=extends` are unsupported. Ordinary class method overriding through `@co.dap.override` remains a separate class feature.

Operator Implementations

An operator implementation is a normal function with operator metadata. It cannot be declared loose at package scope. Every implementation must be in one of these legal function-owning locations:

Operand owner	Required implementation location

built-in type	an `@co.dap.extension` function inside a unit
`co.lang.struct`	the struct's same-package companion unit
`co.lang.class`	an operator method declared by the class
module, enum, union, interface, signature, `co.lang.cstruct`	unsupported

An implementation uses `mode=overload`, or omits `mode` because omission is shorthand for `mode=overload`. A one-character symbol may use a character literal; a multi-character symbol must use a string literal:

```
@co.dap.operator(symbol='+', mode=overload)
add(left Employee, right Employee)->(Employee) = {
    ...
}

@co.dap.operator(symbol="+-", mode=overload)
combine(left Employee, right Employee)->(Employee) = {
    ...
}
```

FoLang does not attempt to prove that an overload follows the conventional meaning of its symbol. A developer may overload `+` with any implementation. Using one symbol consistently for one recognizable concept is recommended for readability, but it is not a compiler-enforced semantic restriction.

For a receiverless struct-companion operator function, the first declared operand must have the matching struct type. A matching struct type in a later operand is insufficient. For a unary operator, the sole operand must have the matching struct type.

A matching instance receiver contributes the first operand. A matching type receiver establishes ownership but contributes no operand; its ordinary parameter list is the complete operator signature. In a class, an ordinary or `@co.dap.instance` operator method has an implicit `this` first operand, while `@co.dap.static` and `@co.dap.class` operator methods use only their declared parameters and require the first declared operand to have the enclosing class type.

After receiver normalization, the operand count must match the registered arity of the operator. Each distinct normalized operand signature is an overload. A second implementation with the same normalized signature is a duplicate and is rejected. This rule applies equally to built-in operators, pre-declared glyphs, and project-local custom operators.

Language-Owned Operators

Language-owned operators already have a registered symbol, fixity, precedence, associativity, and arity. They must not be redeclared in the project operator source. They receive additional implementations through `mode=overload`.

This category includes:

1. ordinary built-in operators such as `+`, `-`, `*`, and `==`;
2. pre-declared operator glyphs whose parse properties are fixed by the language but which may initially have no implementation.

Pre-Declared Operator Glyphs

The language pre-declares the documented mathematical and modifier glyph set, including `λ`, `∀`, `∃`, `∪`, `⇛`, `∂`, `⊥`, `↓`, `⇓`, `∘`, `Π`, and `𝓘`, with fixed parse properties. These symbols are reserved against project-local re-declaration, but they are available for implementation through ordinary operator overloads:

```
// ∪ is already registered by the language; only its implementation is supplied.
@co.dap.operator(symbol='∪', mode=overload)
union(left Set, right Set)->(Set) = {
    ...
}
```

Until a visible implementation matches the operand types, an expression using a pre-declared glyph parses successfully and then fails during operator resolution.

Hard-reserved spellings such as `::=`, `->>`, `<->`, backtick, backslash, `#`, and comment openers are different: they are not overloadable or declarable unless a later language revision explicitly assigns them operator semantics.

Project-Local Custom Operator Source

A custom operator is a symbol that is neither language-owned nor hard-reserved. Its symbol and parse properties are registered in the fixed operator source selected by `fol-conf.yaml`:

```
output_folder: out
lib_folder: lib
exe_folder: build
back-end: GCC
env_type: Compile
operator_library_folder: operators
```

The compiler resolves a relative path from the project root and checks:

```
<project-root>/operators/operators.fol
```

If the configuration entry, folder, or fixed file is absent, the project introduces no custom symbols. The configured folder is excluded from ordinary folder-derived package discovery.

The file is parsed by a dedicated operator-source lexer and parser before the ordinary FoLang lexer and parser run. It must contain exactly one source-only operator library declaration:

```
// operators/operators.fol
@co.dap.library(type=operator)
_ co.lang.library = {

    ⊗ co.lang.operator = {
        fixity=infix,
        precedence=60,
        associativity=left,
        arity=binary,
        commutative=co.const.false,
        idempotent=co.const.false,
        identity=co.const.none,
        foldable=co.const.false,
        vectorizable=co.const.false,
        distributes_over=[],
        desugar="intrinsic:tensor_product"
    }

    +- co.lang.operator = {
        fixity=infix,
        precedence=60,
        associativity=left,
        arity=binary
    }
}
```

`@co.dap.library(type=operator)` and `_ co.lang.library` identify this fixed bootstrap surface. It is not an ordinary library surface, is not imported, and does not produce a `.folib` or `.folenc`. The body may contain only `co.lang.operator` declarations. Imports, functions, types, variables, expressions, and nested libraries are forbidden.

`co.lang.operator` is valid only in this dedicated source grammar. It is not an ordinary FoLang declaration kind and cannot appear in package source, an entry file, or an ordinary library surface.

Operator declaration attributes

Every custom declaration must contain each required parse attribute exactly once. Optional metadata may appear at most once. Duplicate and unknown attributes are errors.

Attribute	Required	Accepted value	Meaning
`fixity`	yes	`infix`, `prefix`, `postfix`, or a reserved future fixity name (`circumfix`, `postcircumfix`, `precircumfix`,etc)	parser placement
`precedence`	yes	decimal integer from `0` through `100`	binding strength
`associativity`	yes	`left`, `right`, or `none`	grouping for equal precedence
`arity`	yes	`unary`, `binary`, `ternary`, or a positive decimal integer	normalized operand count
`commutative`	no	`co.const.true` or `co.const.false`	semantic/optimization metadata
`idempotent`	no	`co.const.true` or `co.const.false`	semantic/optimization metadata
`identity`	no	a FoLang literal, including `co.const.none`	identity-value metadata
`foldable`	no	`co.const.true` or `co.const.false`	compile-time folding metadata
`vectorizable`	no	`co.const.true` or `co.const.false`	vectorization metadata
`distributes_over`	no	a list of quoted operator symbols	distributivity metadata
`desugar`	no	a string literal	intrinsic or lowering hook metadata

In the alpha profile, only `infix`, `prefix`, and `postfix` are implemented. Consequently, an alpha `prefix` or `postfix` declaration must be unary and an alpha `infix` declaration must be binary. Other fixity names remain reserved for future delimiter and slot grammars and are rejected by the alpha conformance profile.

The four required parse attributes construct the tokenizer and precedence table. Optional semantic/optimization attributes do not change tokenization or parsing.

Declaration and implementation are separate

The operator library registers only the symbol and metadata. It contains no implementation. Implementations use the same `mode=overload` form as built-in and pre-declared operators:

```
// vector/Vector.unit.fol
_ co.lang.unit = {

    @co.dap.operator(symbol='⊗', mode=overload)
    tensorProduct(left Vector, right Vector)->(Vector) = {
        ...
    }
}
```

A custom symbol has exactly one declaration in the operator library, but it may have any number of distinct implementation overloads in legal owners:

```
one ⊗ declaration
    Vector ⊗ Vector -> Vector
    Matrix ⊗ Matrix -> Matrix
    Tensor ⊗ Tensor -> Tensor
```

The declaration cannot be duplicated, aliased, merged, selected, or remapped. The implementations participate in ordinary operator overload resolution.

Symbol registration and exact recognition

The dedicated operator-source lexer reads the declaration name as one maximal contiguous symbol run. After the operator table is built, the ordinary lexer uses the same whole-run rule. A registered custom symbol is recognized only when the complete run matches that symbol; an unknown run is rejected without being split into shorter operators.

Multi-symbol operator uses must also satisfy the operand-boundary rule defined in Symbolic Runs, Classification, and Boundaries. Consequently, registering `+-` permits `a +- b`, not `a+-b`. Unicode is not required for custom operators.

A custom symbol:

- must not be language-owned;
- must not be a hard-reserved spelling;
- must not contain `//` or `/*`, because a comment opener terminates the preceding symbolic run and is recognized before further operator matching;
- must have exactly one declaration in the operator library.

Symbols are global; implementations are resolved by scope and type

Once a custom symbol is registered—or a language-owned symbol is loaded—the ordinary tokenizer recognizes it throughout that compilation. The parser can therefore build the same operator expression in every package.

Whether an implementation is available is determined later through ordinary operator resolution. Receiver-owned class and companion implementations follow their normal lookup rules. Extension implementations require normal `@co.ddap.use` activation. A symbol with no matching visible implementation produces a resolution error, not a syntax error.

Using one symbol for one recognizable concept across its overloads is strongly recommended for readability, but the compiler does not and cannot enforce that recommendation. The type/signature rules, not the conventional meaning of the symbol, determine whether an overload is legal.

Operators Do Not Cross a Library Boundary

Operator notation is local compilation syntax, not a library API element. An ordinary library surface exports named function signatures and boundary data contracts. It exports no operator table.

```
geometry.folib
├── projected symbol table
└── compiled implementation/linkage        (no operator table)
```

A library may use custom or pre-declared operators internally. Those expressions are resolved and lowered while compiling that library. Importing the resulting artifact does not add any symbols or parse properties to the importing compilation.

An application that wants operator notation for an imported boundary type must register a local custom symbol when necessary and provide a legal local extension implementation that delegates to the library's named public function.

Bootstrap Order

```
1. Read fol-conf.yaml.
2. If configured, parse operators/operators.fol with the dedicated
   operator-source lexer and parser.
3. Validate the exact operator-library marker and body restrictions.
4. Reject duplicate custom declarations, language-owned redeclarations,
   hard-reserved spellings, invalid attributes, and invalid alpha fixity/arity
   combinations.
5. Combine language-owned registrations with the project-local custom
   declarations and build the immutable maximal-munch and precedence tables.
6. Run the ordinary FoLang lexer and parser over application or library source.
7. Resolve operator implementations by owner, scope, operand types, and normalized
   callable signature.
```

Imports contribute no operator metadata, so this bootstrap has no import-order or transitive-dependency dependency.

Forward / Extern Declarations

Variables extern declaration

```
@co.dap.declare(extern)
someBool co.lang.bool;
```

Functions forward declaration

```
@co.dap.declare(forward)
getEmployee(id co.lang.int)->(somepack.Employee);

// or — @co.dap.declare is optional for functions
getEmployee(id co.lang.int)->(somepack.Employee);
```

Types external declaration

```
@co.dap.declare(extern)
Employee co.lang.struct;

// or — @co.dap.declare is optional for types
Employee co.lang.struct;
```

> For functions and types `@co.dap.declare` is optional. For variables it is required.

Functions

`folang` doesn't allow free flowing functions as UDTs they must be in package source file apart from that they must be enclosed in a kind of container call Unit `co.lang.unit`

Normal

```
General co.lang.unit = {

    fun1(k co.lang.int, b co.lang.char)->(co.lang.int, co.lang.char) = {
        // function body
    }
}
```

`folang` function can return multiple values

Default Parameters

```
fun1 (k co.lang.int, b co.lang.char = 10)->(co.lang.int, co.lang.char)={
}
```

Variadic Functions

Curried functions are not allowed to be variadic, and vice versa.

```
fun1 (k co.lang.int, ...b co.lang.char)->(co.lang.int, co.lang.char)={
}
```

Optional Parameters

```
fun1(k? co.lang.int)->()={
    k.omitted.do({

    }).otherwise({

    });

}
```

Named Parameters

```
fun1(~k co.lang.int)->()={

}
```

Named Returns

```
doManythings(a co.lang.int, b co.lang.int->(&, meta={type=out}))->(r co.lang.int, e co.lang.exception)={}
```

Function Delegates

```
@co.dap.delegate someDelegate co.lang.delegate = (a co.lang.int, b co.lang.int) -> (co.lang.int, co.lang.int);
```

Function Chaining

```
fetchEmployee(empId co.lang.string)->(Employee)=>>empMod.getEmployee(this, empId);
```

Anonymous Functions

```
add := (a int, b int) -> (int) {
    this.return a + b;
};

res := (a int, b int) -> (int) {
    this.return a * b;
}(10, 20);
```

> for more details on functions please refer section Functions in Details

---

Functions in detail

Inline

```
Math co.lang.unit = {
    @co.dap.inline
    add(a co.lang.int, b co.lang.int)->(co.lang.int) ={
        this.return a + b;
    }
}
```

Anonymous Functions

```
add := (a int, b int) -> (int) {
    this.return a + b;
};

res := (a int, b int) -> (int) {
    this.return a * b;
})(10, 20);
```

Lambda

Only allowed as an inline callback argument to receiver-qualified collection operations (e.g. `map`, `filter`, `reduce`, `forEach`, `sortBy`, `groupBy`). Transparent grouping around the member callee is permitted, so `(nums.map)(|x| => x*x)` is equivalent to the ordinary call spelling. A bare function call such as `map(|x| => x)` is not a collection-method context. Using `|...|` anywhere else is a syntax/lint error.

```
// Callback literal syntax, shown only in its valid collection-call context
nums.map(|x| => x*x);
words.filter(|s| => s.len() > 3);
pairs.reduce(|acc, e| => acc + e, 0);
dict.map(|k, v| => v * 10);
list.sortBy(|a, b| => a.score - b.score);
```

The lambda must be a direct argument of the allowed collection call. That call may itself be nested, for example `consume(nums.map(|x| => x*x))`; the enclosing call does not make the lambda an argument of `consume`.

Inner Function

```
myfun(a co.lang.int, b co.lang.int)->(co.lang.int)={
    p co.lang.int = 10;
    someother()->()={
        co.out.println(p);
    }
    someother();
    p = 20;
    someother();
}
```

The function `someother` is an ordinary inner function because it is physically declared inside `myfun`. Its free runtime names are resolved from the lexical declaration context of `myfun`; therefore `p` denotes the binding declared in `myfun` regardless of where `someother` is called within its permitted lifetime.

`@co.dap.local`, `@co.dap.nested`, and `@co.dap.inner` apply to separately declared functions and do not change the lexical-scope rule for an ordinary inner function. They provide visibility, target-state capture, or call-site association without requiring the function body to be physically placed inside the target declaration.

The permitted local-function form above is useful when the inner function is small and belongs naturally to one enclosing function. This is the named local-function exception described in Physical Nesting Rules.

Curried

```
add(first co.lang.int)(second co.lang.int)->(co.lang.int)={
    this.return first + second;

}
```

Closure

```
adder() -> ((co.lang.int) -> co.lang.int) ={
    sum co.lang.int = 0;
    this.return  (x co.lang.int) -> (co.lang.int) = {
        sum += x;
        this.return sum;
    }
}
```

Functions Taking and Returning Functions

Syntax 1 — Inline signature

```
someFunction (r (co.lang.int, co.lang.int)->(co.lang.int))->((co.lang.int)->(co.lang.int))={}
```

Syntax 2 — Named type alias

```
someFArg co.lang.type = (co.lang.int, co.lang.int)->(co.lang.int);
someFRet co.lang.type = (co.lang.int)->(co.lang.int);

someFunction (r someFArg)->(someFRet)={}
```

Syntax 3 — Function objects

```
someFArg co.lang.function = (a co.lang.int, b co.lang.int) -> (co.lang.int)={
    this.return a + b;
}

someFRet co.lang.function = (a co.lang.int) -> (co.lang.int)={
    this.return a * 2;
}
```

Other ways to declare clsures/function objects and types/ curried functions

```
myobj co.lang.function = (a co.lang.int, b co.lang.int)->(co.lang.int)={
    this.return a + b;
}

add (a co.lang.int, b co.lang.int)->(co.lang.int){ this.return a + b; }
oObj co.lang.function = add;

funtype co.lang.type = (a co.lang.int, b co.lang.int)->(co.lang.int);

closure=(factor co.lang.int, val co.lang.int) ==>> factory * val;

curry = (factor co.lang.int) (x co.lang.int) ==>> x * factor;
```

Associated Functions

For a user-defined struct, associated functions must be declared inside the same-package companion unit whose name matches the struct. For more details on associated function please refer section Associated Functions in a Companion Unit

Some Restrictions on Special Functions

1. **Special functions**
   - Curried functions
   - Functions with named arguments
   - Functions with optional arguments
   - Functions with default arguments
   - Variadic functions
   - Functions that take or return functions or function types
   - Dynamically scoped and mixed-scoped functions
2. **Restrictions**
   - They cannot be overloaded.
   - They cannot be used as callbacks.
   - They cannot participate in Execution Models and Control Abstractions.

Scoping Rules for Functions

All functions in FoLang have a defined scope — the set of variables a function can access.

Default — Lexical / Static Scope

All of the following are **always** lexically scoped — this cannot be changed:

```
methods
inner methods
free functions
inner functions
target-local named functions
closures
lambdas
Generic Functions
Anonymous functions
Curried functions
```

Lexical scope means a function resolves names from its **declaration site**, not from the scope of its caller. Unit-level variables are forbidden, so a unit-scoped free function receives runtime values through parameters or introduces them locally. An ordinary inner function captures from the enclosing lexical declaration context. Calling that inner function does not replace its captured context with the caller's runtime scope.

```
ScopeExample co.lang.unit = {

    foo()->() = {
        x co.lang.int = 10;

        printX()->() = {
            co.out.println(x);  // ✅ x from the enclosing lexical scope
        }

        printX();
    }
}
```

`@co.dap.inner` — Call-Site Lexical Context

`@co.dap.inner` is not the syntax for an ordinary inner function. It annotates a separately declared function or method that becomes associated with a target at an explicit use or attachment site.

Its free runtime names use **call-site lexical-context resolution**:

```
1. @co.dap.inner parameters and local declarations
2. the innermost lexical scope at the active call or attachment site
3. enclosing lexical scopes of that call or attachment site
4. statically resolved types, annotations, imports, and co.* names
5. compiler error
```

The compiler validates these requirements at every statically known use or call site. This is a fixed property of `@co.dap.inner`; it is not selected through `@co.dap.lexicalscope`, `@co.dap.dynamicscope`, or `@co.dap.mixedscope`.

This differs from ordinary lexical inner functions, whose free runtime names are fixed by their declaration site, and from dynamically scoped associated functions, which may search outward through the runtime caller activation chain.

Associated Functions — Additional Scope Options

Only associated functions support non-lexical scoping via annotations:

The examples below are members of the same-package `Employee` companion unit.

`@co.dap.lexicalscope` — default, explicit declaration

```
Employee co.lang.unit = {
    @co.dap.lexicalscope
    (emp Employee) process()->() = {
        co.out.println(emp.name);   // ✅ declaration scope
    }
}
```

`@co.dap.dynamicscope` — accesses caller's scope

```
Employee co.lang.unit = {
    @co.dap.dynamicscope
    (emp Employee) process()->() = {
        co.out.println(name);   // name comes from caller's scope
    }
}
```

`@co.dap.mixedscope` — accesses both scopes

```
Employee co.lang.unit = {
    // caller scope takes priority — shadows declaration scope on conflict
    @co.dap.mixedscope
    (emp Employee) process()->() = {
        co.out.println(name);   // caller scope takes priority
        co.out.println(emp.id); // falls back to declaration scope
    }
}
```

Callback Scope Inside Dynamically Scoped Associated Functions

A callback block or lambda does not independently select lexical, dynamic, or mixed scope. The associated function that executes the callback determines how the callback's free runtime names are resolved.

Callback parameters and declarations made inside the callback always belong to the callback's local scope. Names that are not callback parameters or callback-local declarations follow the executing associated function's scope policy.

```
nums.reduce(|acc, e| => {
    total.value += e;
    acc + e;
}, 0);
```

For this example:

```
acc, e              -> callback parameters
temporary locals    -> callback-local context
total               -> reduce's dynamic caller context
co.* and imports    -> normal built-in/import resolution
```

The callback syntax remains uniform. `reduce` is dynamically scoped, so unresolved runtime names in the callback are resolved through the active caller-context chain. A lexically scoped associated function would instead resolve free runtime names through its declaration context.

Why Dynamic Scope Exists — `.do`, `.loop`, `.each`, and Collections

FoLang's control-flow model is built on dynamically scoped associated functions. The executing function supplies the scope policy, so blocks do not require separate capturing and non-capturing forms.

```
x     co.lang.int = 10;
total co.lang.int = 0;
arr   co.lang.int->([5]) = [1, 2, 3, 4, 5];

// .do reads and modifies the caller's x
(x > 5).do({
    x.value = 20;
    co.out.println(x);
});

// .loop modifies the caller's x
(x > 0).loop({
    x.value -= 1;
});

// .each modifies the caller's total
arr.each(_, val).do({
    total.value += val;
});

// .filter, .map, and .reduce use the same dynamic caller context
nums.filter(|x| => x > 10);
nums.reduce(|acc, e| => {
    total.value += e;
    acc + e
}, 0);
```

The compiler does not create a separate capture description for each control block. It resolves names according to the scope mode of the associated function executing that block.

FoLang Control Flow Uses Dynamic Scope

```
no if/else keywords    -> .do / .otherwise  — dynamic scope
no for/while keywords  -> .loop             — dynamic scope
no foreach keywords    -> .each             — dynamic scope
no in keyword          -> .contains         — dynamic scope
no map/filter keywords -> .map / .filter     — dynamic scope

all control flow is implemented as associated functions
control associated functions are dynamically scoped
caller variables are accessible and mutable under the dynamic lookup rules
block syntax requires no separate capture mode
```

Dynamic and Mixed Lookup Order

For `@co.dap.dynamicscope`, runtime variable lookup follows this order:

```
1. function/callback parameters and local declarations
2. immediate caller activation context
3. caller activation contexts outward through the dynamic call chain
4. built-in `co.*` names and imported declarations visible to the source file
5. compiler error if no compatible declaration is available
```

For `@co.dap.mixedscope`, lookup follows this order:

```
1. function/callback parameters and local declarations
2. immediate caller activation context
3. caller activation contexts outward through the dynamic call chain
4. lexical declaration context
5. built-in `co.*` names and imported declarations visible to the source file
6. compiler error if no compatible declaration is available
```

Local declarations shadow caller declarations. Caller declarations shadow declarations from the lexical declaration context in mixed scope.

Types, annotations, imports, and other compile-time declarations continue to use ordinary static resolution. Dynamic lookup applies to runtime bindings, not to the identity of types or imported APIs.

Call-Site Validation of Dynamic Requirements

A dynamically or mixed-scoped associated function may reference a caller-provided runtime name that is not declared in its lexical context:

```
@co.dap.dynamicscope
(Employee) addToTotal(value co.lang.int)->() = {
    total.value += value;
}
```

At every statically known call site, the compiler validates that the active caller-context chain provides a definitely initialized binding named `total` with a compatible type and permitted mutability. A call is rejected when the required binding cannot be proven to exist.

Because non-lexically scoped functions are non-first-class and non-escaping, their call sites remain statically identifiable. This avoids runtime string-based name lookup and allows the compiler to represent dynamic requirements using context and symbol-table references.

Scope Rules Summary

Function Type	Lexical	Dynamic	Mixed
methods	✅ declaration-site only	❌	❌
ordinary inner methods	✅ declaration-site only	❌	❌
free functions	✅ declaration-site only	❌	❌
ordinary inner functions	✅ enclosing declaration context only	❌	❌
target-local named functions	✅ declaration-site only	❌	❌
`@co.dap.inner` functions/methods	call-site lexical context	❌ no unrestricted caller-chain lookup	❌
anonymous functions	✅ declaration-site only	❌	❌
closures	✅ declaration-site capture	❌	❌
lambdas/callback blocks	determined by executing associated function	determined by executing associated function	determined by executing associated function
associated functions	✅ default	✅ opt-in	✅ opt-in
`.do` / `.loop` / `.each`	❌	✅ built-in	❌
`.map` / `.filter` / `.reduce`	❌	✅ built-in	❌

Additional Restrictions

- dynamically or mixed-scoped functions cannot be returned
- dynamically or mixed-scoped functions cannot be stored as values
- dynamically or mixed-scoped functions cannot be passed as ordinary callbacks
- dynamic or mixed caller contexts cannot cross thread or process boundaries
- dynamic or mixed execution cannot continue after the caller frame ends
- dynamic or mixed-scoped functions cannot participate in Execution Models and Control Abstractions (library type=advanced)
- dynamically or mixed-scoped functions cannot be curried
- named, optional, variadic, and default-parameter forms follow the same non-escaping and call-site-validation rules

Types

The three axes — each adds one new power:

Axis 1: Polymorphism (terms depend on types)

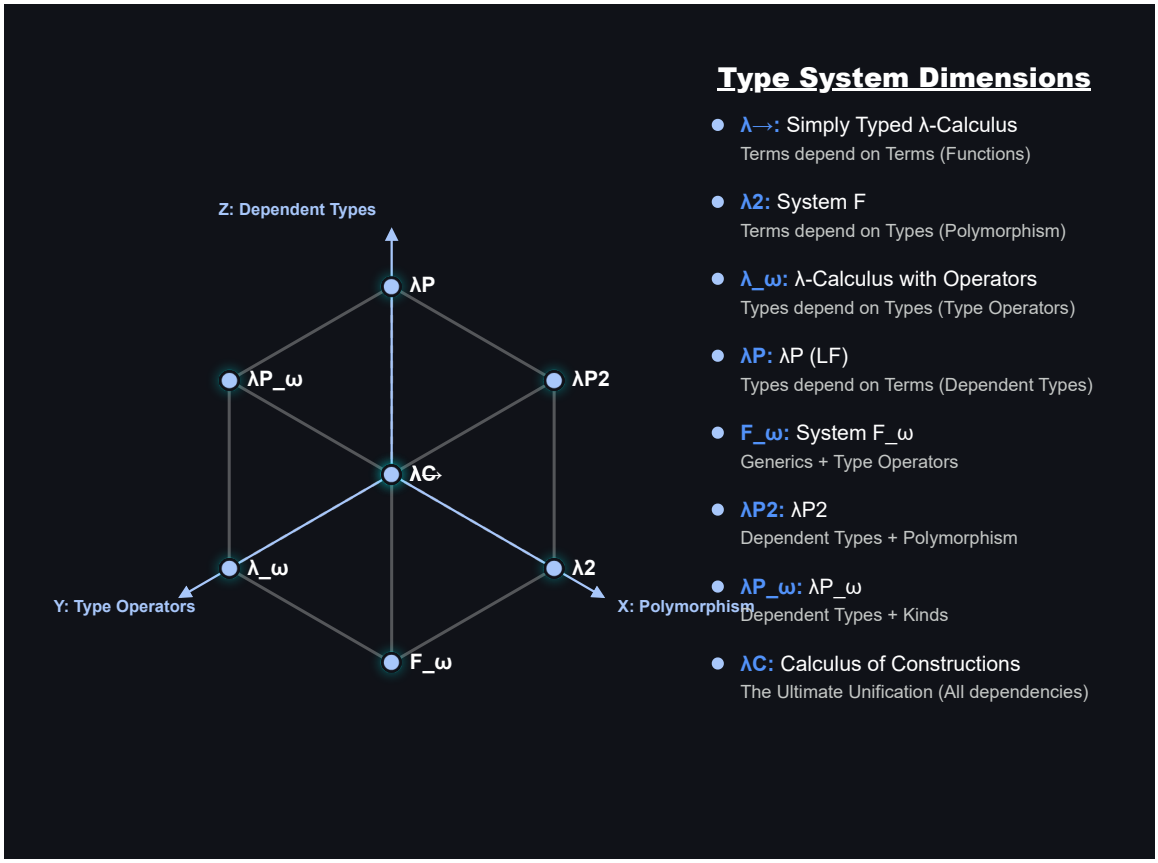
```
// "Give me a type, I'll give you a value"

// Without: write separate functions
identityInt(x int) → int={}
identityStr(x string) → string={}

// With: one function works for all types
identity(T)(x T) → T={}

// This is generics / parametric polymorphism
// System F, Java generics, your @co.dap.generic
```

Axis 2: Type operators (types depend on types)

```
// "Give me a type, I'll give you a type"

List(Int)     → List ={}            // List of ints type → type
Map(String, Int) → Map  ={}         // type → type → type
Option(T)     → Some(T) | None;     // type constructor

// This is kinds / higher-kinded types
// Your folang: Option(T) co.lang.type = Some(T) | None()
```

Axis 3: Dependent types (types depend on values)

```
// "Give me a value, I'll give you a type"

Vector(3)      → array of exactly 3 elements
Matrix(2, 3)   → 2x3 matrix
NonZero(n)     → proof that n ≠ 0

// The type changes based on a runtime value
divide(a int, b NonZero(int)) → int={}
// compiler PROVES b can't be zero
```



Lambda Cube

Dependent Types

Type Constructors — Functions That Return Types

A type constructor is a function that takes a value or type and returns a type. The returned type depends on the input value — this is what makes it a dependent type.

```
// Vector — type constructor function
// takes  → co.lang.int (size)
// returns → co.lang.dependentType (a type)
Vector(n co.lang.int)->(co.lang.dependentType) =
    co.lang.int->([n]);

// calling Vector(3) returns a TYPE at compile time
// that type is co.lang.int->([3])
v3 Vector(3) = [1, 2, 3];    // type is Vector(3)
v4 Vector(4) = [1, 2, 3, 4]; // type is Vector(4) — different type!

// Vector(3) ≠ Vector(4) — completely different types
// size is part of the type — compiler knows at compile time
```

More About Type

```
Name(T) co.lang.data = variants;
    → concrete ADT type-constructor definition
    → right-hand-side definition is mandatory

Name(T) co.lang.type;
    → abstract type-constructor requirement
    → permitted only inside a signature

Name(T) co.lang.type = ExistingType(T);
    → concrete type alias or signature-component binding
    → permitted in modules and ordinary type declarations
```

Type Constructor Is A Function

```
Vector       →  function (type constructor)
Vector(3)    →  function call → returns type co.lang.int->([3])
Vector(4)    →  function call → returns type co.lang.int->([4])

just like:
    add(1, 2)  →  returns a value  (3)
    Vector(3)  →  returns a type   (int[3])
```

Compiler Enforced Size Safety

```
// dot product — only valid for same size vectors
// compiler enforces this via dependent types
dotProduct(a Vector(n), b Vector(n))->(co.lang.int) = {
    // n is same for both — compiler verified
}

v3 Vector(3) = [1, 2, 3];
v4 Vector(4) = [1, 2, 3, 4];

dotProduct(v3, v3);   // ✅ same type Vector(3)
dotProduct(v3, v4);   // ❌ compiler error — Vector(3) ≠ Vector(4)
```

Matrix — Two Parameter Type Constructor

```
// Matrix — takes rows and cols, returns dependent type
Matrix(r co.lang.int, c co.lang.int)->(co.lang.dependentType) =
    co.lang.int->([r, c]);

m34 Matrix(3, 4) = [[1,2,3,4],[5,6,7,8],[9,10,11,12]];
m45 Matrix(4, 5) = ...;

// matrix multiply — cols of A must equal rows of B
// n must match — compiler verified
multiply(a Matrix(r, n), b Matrix(n, c))->(Matrix(r, c)) = {
    // compiler ensures dimensions are compatible
}

multiply(m34, m45);   // ✅ Matrix(3,4) × Matrix(4,5) = Matrix(3,5)
multiply(m34, m34);   // ❌ compiler error — 4 ≠ 3
```

Stack — Value and Type Parameter

```
// Stack — takes size and element type
Stack(n co.lang.int, T co.lang.type)->(co.lang.dependentType) =
    T->([n]);

s Stack(10, co.lang.int)    = ...;  // stack of max 10 ints
t Stack(5,  co.lang.string) = ...;  // stack of max 5 strings
```

Type Is Value + Kind Combined

```
Vector(3):
    kind  = Vector    (what it is)
    value = 3         (how many)
    type  = Vector(3) (both together — the dependent type)

Vector(3) ≠ Vector(4)   ←  different types entirely
Vector(3) = Vector(3)   ←  same type
```

Connects to Type Constructor in Spec

```
// Option — type constructor for ADT
@co.dap.hokrt
Option(T) co.lang.data = Some(T) | None();

// Vector — type constructor for dependent type
Vector(n co.lang.int)->(co.lang.dependentType) =
    co.lang.int->([n]);

// same concept:
//   Option(T) takes a type  → returns ADT type
//   Vector(n) takes a value → returns dependent type
```

Simple Dependent Type

```
identity(x co.lang.int)->(x.type) ={ this.return x; }
```

Compile-Time and Runtime Values in Type-Related Computation

FoLang distinguishes value-indexed dependent types, compile-time type computation, runtime type descriptors, and runtime values whose concrete types may differ. These mechanisms are related, but they are not interchangeable.

1. Value-Indexed Dependent Types

A dependent type may contain a value as part of its type identity. The value index may be a compile-time constant, a symbolic type-level value, or a runtime function parameter whose relationship to the result is tracked by the compiler.

```
readVector(n co.lang.int)->(Vector(n)) = {
    ...
}
```

The compiler does not need to know the concrete value of `n` while compiling `readVector`. It records the relationship:

```
input index = n
result type = Vector(n)
```

Likewise:

```
dotProduct(
    left  Vector(n),
    right Vector(n)
)->(co.lang.int) = {
    ...
}
```

requires both vectors to have the same value index. Dependent typing means that a type contains or is constrained by a value; it does not mean that an arbitrary runtime branch can silently change the static type of an already compiled variable.

---

2. Compile-Time Type Computation

A function may compute and return a type when it is guaranteed to execute during compilation.

```
@co.dap.comptime
@co.dap.eager
chooseType(value co.lang.int)->(co.lang.type) = {
    (value < 100)
        .return(co.lang.string)
        .otherwise.return(co.lang.bool);
}
```

The arguments must be compile-time evaluable when the result is used in a static type position:

```
Selected co.lang.type = chooseType(10);
value Selected = "Hello";
```

Invalid:

```
input := co.in.readInt();
Selected co.lang.type = chooseType(input);
// compiler error: `input` is not compile-time evaluable
```

Conceptually:

```
compile-time value
    -> compiler executes the type function
    -> one concrete static type is available before ordinary type checking completes
```

---

3. Built in compile type computation

A function may compute and return a type when it is guaranteed to execute during compilation.

> `decltype` built in method

The arguments must be compile-time evaluable when the result is used in a static type position:

```
    someIntVar co.lang.int ;
    someVar co.hokrlt.type.decltype(someIntVar) = 200;
```

---

4. Runtime Type Descriptors

An ordinary function returning `co.lang.type` produces a runtime type descriptor when it is not executed at compile time.

```
selectType(value co.lang.int)->(co.lang.type) = {
    (value < 100)
        .return(co.lang.string)
        .otherwise.return(co.lang.bool);
}

selectedType co.lang.type = selectType(input);
```

Here, `selectedType` is a runtime object that describes a type. It may be used for reflection, runtime validation, dynamic loading, metadata inspection, or dynamic object creation where those capabilities are permitted.

A runtime type descriptor cannot ordinarily be used as the static type of a declaration:

```
value selectType(input);
// compiler error: runtime type descriptor used in a static type position
```

Conceptually:

```
runtime value
    -> runtime type-selection function
    -> co.lang.type descriptor object
```

A value that represents a type is not automatically a compile-time-resolved static type. ***

5. `@co.dap.typefromvalue`

`@co.dap.typefromvalue` derives a type from the type of a returned compile-time value. In the initial FoLang specification, it is permitted only for compile-time evaluation.

```
@co.dap.comptime
@co.dap.typefromvalue
inferType(value co.lang.int)->(co.lang.type) = {
    (value < 100)
        .return("Hello")
        .otherwise.return(co.const.true);
}
```

The compiler derives:

```
"Hello"       -> co.lang.string
co.const.true -> co.lang.bool
```

Every argument must be compile-time evaluable when the result is used in a type position. Runtime value-based type selection must instead use an explicit runtime representation.

6. Runtime Values with Different Concrete Types

When a runtime branch may produce unrelated concrete value types, the function must return one stable outer type. FoLang may use a tagged value, an ADT, `co.lang.dynamic` where permitted, or another explicitly packaged representation.

Using `co.lang.tag`:

```
selectValue(value co.lang.int)->(co.lang.tag) = {
    (value < 100)
        .return(co.lang.tag(co.lang.string, "Hello"))
        .otherwise.return(
            co.lang.tag(co.lang.bool, co.const.true)
        );
}
```

The outer static type is always `co.lang.tag`:

```
co.lang.tag
├── runtime type descriptor
└── value compatible with that descriptor
```

An ADT provides a more strongly typed alternative:

```
SelectedValue co.lang.data =
      StringValue(co.lang.string)
    | BoolValue(co.lang.bool);

selectValue(value co.lang.int)->(SelectedValue) = {
    (value < 100)
        .return(StringValue("Hello"))
        .otherwise.return(BoolValue(co.const.true));
}
```

Summary

```
Vector(n)
    -> value-indexed dependent type

@co.dap.comptime function returning co.lang.type
    -> compile-time type computation

ordinary runtime function returning co.lang.type
    -> runtime type descriptor

runtime branch returning unrelated value types
    -> tag, ADT, dynamic value, or another explicit package
```

A runtime type descriptor is a value that represents a type. It must not be confused with a statically resolved dependent type.

The Three Parameterized Type Forms

Three declarations produce a type from a parameter. Which spelling applies depends on one thing: whether any parameter is a **value**.

```
// all parameters are types -> generic parameter clause
Option(T) co.lang.data = Some(T) | None();
someAlias(F) co.lang.type = Functor(F);

// a parameter is a value -> function syntax
Vector(n co.lang.int)->(co.lang.dependentType) = co.lang.int->([n]);
Stack(n co.lang.int, T co.lang.type)->(co.lang.dependentType) = T->([n]);
```

A type parameter needs no annotation, so a bare identifier is enough and the generic parameter clause carries it. A value parameter needs a type, so the function form is used; it also states what is produced through its `->(co.lang.dependentType)` return clause. `Stack` shows why the function form exists: it is the only one that can mix a value parameter and a type parameter.

`co.lang.dependentType` is both a type-producing return kind and a direct type declaration kind. A function-shaped type constructor uses it when a value parameter determines the produced type. A direct declaration may use it when no value-parameter list is required:

```
LengthBound co.lang.dependentType = co.lang.int;
```

The kind is also a usable type in a declarator. If a function returns `co.lang.dependentType`, a binding that receives that result may therefore be declared `co.lang.dependentType`.

A function-shaped type constructor has exactly one unnamed type-producing result. That one result may be a union using `|`, but comma-separated multiple results are invalid:

```
Choice(n co.lang.int)->(co.lang.dependentType | co.lang.type) = co.lang.int;
Bad(n co.lang.int)->(co.lang.dependentType, co.lang.type) = co.lang.int; // invalid
```

Parameterized aliases are transparent

An alias declared with `co.lang.type` names the same type, not a new one.

```
someAlias(F) co.lang.type = Functor(F);

someAlias(List);    // the same type as Functor(List)
someAlias(Option);  // the same type as Functor(Option)
```

Because the alias adds no identity, it creates no separate instance slot: an instance cannot be declared for `someAlias` as distinct from one for `Functor`. Declaring one would be a duplicate.

Named parameters also allow reordering and partial application, which a positional placeholder could not express.

```
Pair(F, G) co.lang.type = Transformer(F, G);
Flip(F, G) co.lang.type = Transformer(G, F);
Fixed(F)   co.lang.type = Transformer(F, Set);
```

Use `co.lang.newtype` instead when a **distinct** identity is wanted, such as the wrapper described in Where an Instance Is Declared.

Dependent Type Index Rules

An **index** is an argument to a dependent type, such as the `n` in `Vector(n)`, or a dimension in an array derivation, such as the `n` in `co.lang.int->([n])`. Both positions obey the same rules.

An index is a literal or a name

An index is an integer literal or a name. Arithmetic, function calls, indexing and every other operator are rejected.

```
@co.dap.const SIZE co.lang.int = 1024;

v Vector(3);                    // ✅ literal
v Vector(SIZE);                 // ✅ @co.dap.const name
buf co.lang.int->([SIZE]);      // ✅ same rule for array sizes

v Vector(n + 1);                // ❌ arithmetic is not permitted in an index
v Vector(computeSize());        // ❌ a call is not permitted in an index
buf co.lang.int->([n * 2]);     // ❌ same rule for array sizes
```

This restriction applies only to the **size** of an array, never to element access. Indexing an array is an ordinary expression and arithmetic is fine.

```
buf co.lang.int->([SIZE]);      // size — restricted
buf[i + 1] = 42;                // access — unrestricted
buf[compute(x)] = 7;            // access — unrestricted
```

What a named index may resolve to

A name used as an index resolves in exactly one of two ways.

A parameter bound by the enclosing signature. A type constructor or a function signature introduces the name, and every use of it inside that signature and its body refers to the bound parameter. The name is not a constant; it stands for whatever value the caller supplies.

```
// n is introduced here, and bound for the whole declaration
Vector(n co.lang.int)->(co.lang.dependentType) = co.lang.int->([n]);

// n is introduced by this signature, and both parameters must share it
dotProduct(a Vector(n), b Vector(n))->(co.lang.int) = {
    // ...
}

// n is introduced as a value parameter and reused in the return type
readVector(n co.lang.int)->(Vector(n)) = {
    // ...
}
```

A `@co.dap.const` compile-time constant. Outside a signature that binds it, a name has nothing to bind to, so it must be a constant the compiler can substitute.

```
@co.dap.const SIZE co.lang.int = 1024;
buf co.lang.int->([SIZE]);      // ✅ SIZE substitutes to 1024
v Vector(SIZE);                 // ✅ same rule for dependent types
```

Nothing else qualifies. `@co.dap.final` marks an immutable binding, and an immutable value need not be known while compiling, so it cannot be substituted.

```
@co.dap.final n co.lang.int = readInput();
bad Vector(n);                 // ❌ immutable, but not known at compile time

m co.lang.int = 10;
alsoBad Vector(m);             // ❌ an ordinary variable is not an index
```

So in a plain variable declaration, where no signature is binding anything, the only legal names are `@co.dap.const` constants.

An index is non-negative

Zero is permitted; a negative index is not.

```
empty co.lang.int->([0]);      // ✅ zero-length array

buf co.lang.int->([-1]);       // ❌ rejected while parsing
v Vector(-1);                  // ❌ rejected while parsing

@co.dap.const OFFSET co.lang.int = -1;
buf co.lang.int->([OFFSET]);   // ❌ rejected after substitution
```

A negative literal cannot be written at all, because no prefix operator is reachable in an index position. A negative constant is rejected when the compiler substitutes it. Both are compile-time errors.

When two dependent types are equal

Two dependent types are equal when their constructors are the same and their indices are pairwise equal. An index comparison has exactly three cases.

Index form	Compared by
integer literal	value
`@co.dap.const` name	substituted literal value
parameter	name identity

```
Vector(3)    vs Vector(3)      // equal
Vector(n)    vs Vector(n)      // equal
Vector(n)    vs Vector(m)      // NOT equal — rejected
Vector(SIZE) vs Vector(1024)   // equal when @co.dap.const SIZE = 1024
```

Rejecting `Vector(n)` against `Vector(m)` is the point of the feature. It is what lets the compiler catch a size mismatch without the developer writing a single check.

What FoLang deliberately does not do

FoLang does not decide index equality up to arithmetic. `Vector(n+1)` and `Vector(1+n)` are not merely unequal — they cannot be written.

Accepting them would require symbolic reasoning, and there is no partial version of it. Once `n+1 == 1+n` is accepted, the next reasonable request is `2*n == n+n`, and the type checker becomes a theorem prover by accretion. That is the complexity FoLang is built to avoid.

The cost is narrow. Length-arithmetic signatures such as `concat(Vector(n), Vector(m)) -> Vector(n+m)` are out of scope; return a dynamically sized type and check at run time instead. Everything that needs only same-parameter identity still works, and that covers the common cases.

```
multiply(a Matrix(r, n), b Matrix(n, c)) -> (Matrix(r, c))
dotProduct(a Vector(n), b Vector(n))     -> (co.lang.int)
zip(a Vector(n), b Vector(n))            -> (Vector(n))
```

Matrix multiplication, the usual demonstration of dependent types, needs only that the shared `n` matches.

Dependent types are checked, never inferred

Every dependent type appears in a written signature. FoLang never infers one.

This is what keeps checking decidable without a constraint solver, and it is why FoLang does not adopt Hindley-Milner style whole-program inference. Inferring a dependent type would mean inferring the index **value**, not merely the type, which is the step that makes checking undecidable in general.

Indexer

Indexer functions for a struct are associated functions and must be declared inside the matching companion unit.

```
MyList co.lang.struct ={
    eles co.lang.int->([...]);
}

MyList co.lang.unit = {

    @co.dap.indexer(symbol="[]")
    (g MyList) get(index co.lang.int)->(co.lang.int) ={
        this.return g.eles[index];
    }

    @co.dap.indexer(symbol="[]=")
    (g MyList) set(index co.lang.int, value co.lang.int)->() ={
        g.eles[index] = value;
    }
}

lst MyList;
co.out.println(lst[0]);
lst[1] = 22;
```

Generics

```
@co.dap.generic(
    at=runtime,
    types=[
        {name= T, variance=invariant, bound=Number, kind=param},
        {name= R ,variance=invariant, bound=Number, kind=result}
    ],
    impredicative=false,
    resolution=compiletime
)
add(a T, b T)->(R) = { this.return a + b; }
```

Generic annotation fields:

Attribute	Values
types	map with type and other details
requires	
resolution	`runtime` , `compiletime`
refied	`true` or `false`
where	`usesite` or `callsite`
specializable	`true` or `false`
impredicative	`true` or `false`

types attributes |Attribute | Values| |—|—| |name|| |constraints|| |upper-bound|| |lower-bound|| |default|| |variance| `covariant` , `invariant` , `contravariant` | |nullable|| |inference| `param` , `result` , `arg` , `var` | |capabilities|| |where-rules|| |typekind| `type` , `class` , `function` , `struct` , `typeconstructor` | |inclusive||

> The above ones will be `types` attribute's sub attributes in a map format

> E.g., @co.dap.generic(types={ T: {variance:..., bound:..., ...} })

> These are not independent attributes these are depend on type

Generic Functions — Parameters and Return Values

Rank-1: Outer function is generic; parameter uses the same type variable

`T` is fixed at the call site before the function parameter is used. The passed function is already monomorphic inside the body.

Syntax 1 — Inline signature

```
@co.dap.generic(types=[{name=T}])
someFunction(f (T, T)->(T), a T)->(T) = {}
```

Syntax 2 — Named type alias

```
@co.dap.generic(types=[{ name=T}])
someFArg co.lang.type = (T, T)->(T);

@co.dap.generic(types=[{ name=T}])
someFunction(f someFArg, a T, b T)->(T) = {}
```

Rank-2: The function parameter is itself polymorphic (higher-rank)

The passed function stays generic **inside the callee**. The callee decides what `T` is. Uses existing `forall`.

Syntax 1 — Inline signature

```
someFunction(f forall(T).(T, T)->(T))->(co.lang.int) = {
    this.return f(1, 2);
}
```

Syntax 2 — Named type alias

```
someFArg co.lang.type = forall(T).(T, T)->(T);

someFunction(f someFArg)->(co.lang.int) = {}
```

```
// Correct — Syntax 2 with co.lang.type
someFArg co.lang.type = forall(T).(T, T)->(T);

someFunction(f someFArg)->(co.lang.int) = {}
```

Returning Generic Functions

Rank-1 return

```
@co.dap.generic(types=[{name=T}])
makeAdder(a T)->((T)->(T)) = {
    this.return (b T)->(T) = { this.return a + b; };
}
```

Rank-2 return — returning a polymorphic function

```
makeIdentity()->( forall(T).(T)->(T) ) = {
    this.return forall(T).(x T)->(T) = { this.return x; };
}
```

Rank-3: A Parameter is Itself a Rank-2 Function

Rank-3 works naturally in FoLang via `forall` nesting. No new constructs needed.

Syntax 1 — Inline

```
// f takes a Rank-2 function as its argument — that is Rank-3
applyRank2(
    f (forall(T).(T, T)->(T)) -> (co.lang.int)
) -> (co.lang.int) = {
    this.return f(1, 1);
}
```

Syntax 2 — Named type aliases (cleaner)

```
rank2FnType  co.lang.type = forall(T).(T, T)->(T);
rank3ArgType co.lang.type = (rank2FnType) -> (co.lang.int);

applyRank2(f rank3ArgType) -> (co.lang.int) = {
    this.return f(1, 1);
}
```

Rank-3 return

```
makeRank2Consumer() -> ((forall(T).(T)->(T)) -> (co.lang.int)) = {
    this.return (f forall(T).(T)->(T)) -> (co.lang.int) = {
        this.return f(42);
    };
}
```

Impredicativity — Instantiating `T` with a `forall` Type

Impredicativity is when a type variable `T` in a generic is itself instantiated with a `forall` type. Example of what this means:

```
@co.dap.generic(types=[{name=T}])
box(x T) -> (Box(T)) = {}

// Impredicative call — T being set to forall(U).(U)->(U)
result := box(forall(U).(U)->(U));   // ❌ not legal without explicit opt-in
```

Most type systems reject this by default. FoLang takes an opt-in approach.

v1 Workaround — Option C: Wrapping with `co.lang.type`

Not true impredicativity but solves 90% of practical cases:

```
polyId co.lang.type = forall(U).(U)->(U);

// box takes co.lang.type — no impredicative unification needed
box(x co.lang.type) -> (Box(co.lang.type)) = {}

result := box(polyId);   // ✅ works — x is co.lang.type, not a forall type
```

v2 — Option A: `impredicative:true` in `@co.dap.generic`

Explicit opt-in via the existing annotation. The compiler only permits `forall` instantiation where declared:

```
@co.dap.generic(
    types=[{name=T,variance=invariant}],
    impredicative=true
)
box(x T) -> (Box(T)) = {}

polyId co.lang.type = forall(U).(U)->(U);
result := box(polyId);   // ✅ legal — impredicative:true explicitly opts in
```

Generic Function Rank Support Matrix

Scenario	Allow?	Notes
Rank-1 generic param (Syntax 1, 2, 3)	✅ Yes	Natural extension, no new concepts
Rank-1 generic return (Syntax 1, 2, 3)	✅ Yes	Same as above
Rank-2 param via `forall` (Syntax 1, 2)	✅ Yes	`co.lang.type` naturally holds polymorphic types; no new kind needed
Rank-2 param via Syntax 3 `co.lang.function`	❌ Compiler error	Function objects are concrete values; use `co.lang.type = forall(T).(T)->(T)` instead
Rank-2 return via `forall` (Syntax 1, 2)	✅ Yes	Same reasoning as param
Rank-3 via `forall` nesting (Syntax 1, 2)	✅ Yes	No new constructs — `forall` nesting works naturally
Rank-3 return	✅ Yes	Same reasoning as Rank-3 param
Rank-3 via Syntax 3 `co.lang.function`	❌ Compiler error	Same rule as Rank-2; function objects are concrete
Impredicative — v1 workaround (Option C)	✅ Yes	Wrap `forall` type in `co.lang.type`; solves 90% of real cases
Impredicative — true opt-in (Option A)	➡️ SOON v2	`impredicative:true` in `@co.dap.generic`; explicit opt-in

`@co.dap.generic` remains the declaration annotation for constraints, variance, reification, and the other generic metadata described below. A direct generic clause supplies the named parameters and their arity; the two forms may be used together when that metadata is required.

```
@co.dap.generic(types=[{name=T}])
LinkedList co.lang.struct={
    value T;
    next  LinkedList;
    prev  LinkedList;
}

k := LinkedList.new(co.lang.int); // when we call new it returns an object of type co.lang.uninit
actualList := k.init(); // this is what create a fully formed object of type class



@co.dap.generic(types=[{name=T},{name= R}])
Employee co.lang.class ={
    id   T;
    name R;

    @co.dap.override
    @co.dap.constructor(access=private)
    @@init() = {}

    @co.dap.override
    @co.dap.constructor(access=public)
    @@init(id T, name R) = {
        this.parent.init();
        this.id   = id;
        this.name = name;
    }

    getEmployee(id T)->(Employee)={}
}

a := Employee.new(co.lang.int, co.lang.string);
b := a.init(1, "Rao");

Normally we need not use @@new and @@init it is special case only applicable for Generics
Normal conditions to create/instantiate object of class we just call init which internally call new
In specific cases as above we need to do two calls or use call chain like below

c := Employee.new(co.lang.int,co.lang.string).init(1,"Rao");

or

without initialization of data

c := Employee.forTypes(co.lang.int, co.lang.string);
```

Generics Inheritances and Types

```
This is in conceptual stage not supported.

A) Abstract vs concrete type members
B) Path-dependent types
    1. Type-level projection
    2. Path-dependent In folang how it would be
```

forall

What `forall` Is — and Is Not

`forall` is **not** a general-purpose generic declaration keyword. It is a **type-level expression only**, restricted to contexts where a polymorphic type must appear as an anonymous value inline — specifically Rank-2 and Rank-3 parameter and return positions, and `co.lang.type` aliases.

Named generic declarations use `@co.dap.generic`, a declaration-name generic clause, or both as described above. `forall` is not a declaration mechanism; `forall` at declaration level is a **compiler error**.

Where `forall` Is Allowed — Type Expression Form Only

`forall(T).` followed by an anonymous type body. The `.` is the syntactic signal that what follows is a type body, not a declaration name.

Pattern:

```
forall(T).  <anonymous type body>
```

```
// co.lang.type alias — naming a polymorphic type for reuse
someFArg co.lang.type = forall(T).(T, T)->(T);

// Rank-2 inline parameter — callee decides what T is
someFunction(f forall(T).(T)->(T)) -> (co.lang.int) = {}

// Rank-2 return type — returning a polymorphic function
makeIdentity() -> (forall(T).(T)->(T)) = {}

// Rank-3 inline parameter — f takes a Rank-2 function
applyRank2(f (forall(T).(T, T)->(T)) -> (co.lang.int)) -> (co.lang.int) = {}
```

Where `forall` Is Banned — Use `@co.dap.generic` Instead

```
// ❌ compiler error — forall at declaration level
forall(T) identity(x T)->(T) = {}

// ✅ correct
@co.dap.generic(types=[{name=T,variance=invariant}])
identity(x T)->(T) = {}
```

```
// ❌ compiler error
forall(T) LinkedList co.lang.struct = { value T; next LinkedList; }

// ✅ correct
@co.dap.generic(types=[{name=T}])
LinkedList co.lang.struct = { value T; next LinkedList; }
```

```
// ❌ compiler error — Rank-1 generics belong to @co.dap.generic
forall(T) someFunction(f (T,T)->(T), a T)->(T) = {}

// ✅ correct
@co.dap.generic(types=[{name=T,variance=invariant}])
someFunction(f (T,T)->(T), a T)->(T) = {}
```

Quick Reference

Form	Status	Context
`forall(T) name ...`	❌ Compiler error	Declaration level — use `@co.dap.generic` instead
`forall(T).(T)->(T)`	✅ Allowed	Type level only — Rank-2/3 param, return, `co.lang.type` alias

The rule in one sentence: `forall(T).` is a type constructor for anonymous polymorphic types; it is never a declaration keyword.

> Generics are applicable to only Classes, Structs and Functions/methods of class provided class is delcared with Generic annotation

> Therre should not be any Cache(T) co.lang.struct or Cache(T) co.lang.class, it is

```
Cache(T) co.lang.module = {} ❌ Compiler error
Operations(F(_)) co.lang.unit = {} ❌ Compiler error
Callback(T) co.lang.delegate = (T)->(T); ❌ Compiler error


Option(T) co.lang.type =  Some(T) | none();   ✅ Only thing Allowed  type constructors

with or without annotation `@co.dap.hokrt`
```

```
    @co.dap.generic(types=[{name=T}])
    LinkedList co.lang.struct={
        value T;
        next  LinkedList;
        prev  LinkedList;
    }

    myIntList LinkedList = LinkedList.forTypes(co.lang.int);

The Generic Type is inside annotation not outside with some special syntax.
same way for class

    @co.dap.generic(types=[{name=T},{name=R}])
    Employee co.lang.class ={
        id   T;
        name R;

    }

    emp Employee = Employee.forTypes(co.lang.int,co.lang.string);

    @co.dap.generic(types=[{name=T},{name=R}] )
    add (a T, b T) ->(R)={}

    add_int_int := add.forTypes(co.lang.int,co.lang.int);

    or

    add_int_int co.lang.function =  add.forTypes(co.lang.int,co.lang.int);

    k := add_int_int(12,10);
```

Specialization

`@co.dap.specialize` to specialize generics for specific types upfront

```
@co.dap.generic(
    types=[
        {name=T}
    ],
    requires=[
        co.lang.Add(left=T, right=T, result=T)
    ]
)
add(a T, b T)->(T) = {
    this.return a + b;
}
```

for the above generic want to specialize for `co.lang.int`

```
@co.dap.specialize(
    target=add,
    types=[
        {name=T, type=co.lang.int}
    ]
)
addInt(a co.lang.int, b co.lang.int)->(co.lang.int) = {
    this.return co.intrinsic.intAdd(a, b);
}
```

`folang` provides partial specialization below is the example for partial specializationn

```
@co.dap.generic(
    types=[
        {name=T},
        {name=R}
    ]
)
transform(value T)->(R) = {
    ...
}
```

```
@co.dap.specialize(
    target=transform,
    types=[
        {name=T, type=co.lang.string},
        {name=R}
    ]
)
transformString(value co.lang.string)->(R) = {
    ...
}
```

*fields of specialize

Attribute	Values
target	the generic
types	resoultion types

Templates

Typed

```
@co.dap.template
add(a co.lang.int, b co.lang.int)->(co.lang.int) ={
    this.return a + b;
}
```

Untyped

```
@co.dap.template
add(a, b)->(co.lang.untyped) ={
    this.return a + b;
}
```

Annotations and Decorators

```
// Annotation — static object, can carry data


myAnnotation co.lang.object->(for=annotation) = {
    value   co.lang.string;
    enabled co.lang.bool;
}

// Decorator — function, transforms target, returns
@co.dap.decorator
myDecorator(target co.lang.function)->(co.lang.function) = { }

Note Directives and Pragmas are not allowed to create as they are language internals
```

Macros

```
// a. Basic macro
@co.dap.macro
say()->()={ this.return co.macro.quote({ println("Line 1") println("Line 2") }); }

// b. Escape assign
@co.dap.macro
yes_esc_assign()->(co.lang.untyped)={
    this.return co.macro.quote({
        co.macro.esc(y) = 42;
        co.out.println("Inside macro: y = ", y);
    });
}

// c. Debug macro with gensym
@co.dap.macro
debug(expr)->(co.lang.untyped)={
    tmp := co.macro.gensym(co.lang.var, "tmp");
    this.return co.macro.quote({
        tmp = co.macro.esc(expr);
        co.out.println("Result: ", tmp);
        tmp;
    });
}

// d. if/else condition macro
@co.dap.macro(
    group = {items:["if","else"], chain:true},
    sugarform={forms:["if expr block"]},
    bind={vars:["x"]},
    isolate={vars:["temp", "index"]},
    gensym={prefix:"tmp_"},
    hygienic=true,
    argtransform={param:"body", wrap:"lambda", whentype:"block"},
    desugar={exprs:["if($cond) { $block }" => "if($cond,$block)"]},
    mode="inject"
)
if(condition expr, body block)->()={}

blockormacro co.lang.kind = block | macro

@co.dap.macro(
    group={items:["if","else"], chain:true},
    sugarform={forms:["else block","else if"]},
    chainswith={macro:"if", position:"immediate", required:true},
    argtransform={param:"body", wrap:"lambda", whentype:"block"},
    standalone=false,
    desugar={exprs:[
        "else if($cond) { $block }" => "else(if($cond, $block))",
        "else { $elseblock }" => "else($elseblock)"
    ]},
)
else(body blockormacro)->()={}
```

Other macro utilities: 1. `@co.dap.compose(using=["base_if", "blockify"])` 2. `@co.dap.guard(expr="is_bool_expr(expr)")` 3. Quasiquote macros use `co.macro.quote` and `co.macro.unquote`

Execution Models and Control Abstractions (library type=advanced)

Fολang executes code sequentially by default. It also provides a uniform execution model for concurrency, parallelism, asynchronous execution, coroutines, continuations, scheduling, and structured task execution.

Developers express the intended execution semantics by applying annotations such as `@co.dap.thread`, `@co.dap.task`, or `@co.dap.process` to a method. When the method is submitted through facilities such as `co.cpca.submitToPool`, `co.cpca.submitThread`, or `co.cpca.submitToEventLoop`, the Fολang runtime selects and manages the appropriate execution mechanism.

Depending on the annotation, submission operation, runtime environment, and execution policy, Fολang may use a thread pool, virtual or green threads, an event loop, a dedicated operating-system thread, or a separate process. Communication operations such as sending and receiving values are also handled through the `co.cpca` package. Developers therefore describe the required execution behavior without directly managing the underlying threads, processes, pools, or event loops.

The `@co.dap.continuation` annotation enables continuation support for a function. An annotated function can use constructs provided by the `co.cpca` package to suspend execution, yield control or a value, preserve its execution state, and later resume from the suspension point.

Native Code (Library type system/ffi)

The `@co.dap.native` annotation enables access to the `co.native` package. Through this package, developers can write assembly and machine-level code using facilities such as `co.native.asm` and `co.native.inline`, providing low-level capabilities similar to those available in C++.

Native Functions

```
@co.dap.native
nativeMethod(a co.lang.int, b co.lang.int)->(co.lang.int) ={
    // native implementation
}
```

Dynamic Runtime (library type=dynamicvmrt)

The `@co.ddap.dynamicruntime` annotation enables full access to the `co.meta` package. Through this package, developers can use dynamic class and type loading, monkey patching, runtime reflection, instrumentation, eval-based code execution, and other advanced metaprogramming capabilities.

Variable Kinds Support

Kind	Where
Normal	All
Pointers	Library of type `system` and `ffi`
Arrays	All
References Heap, Lvalue, Rvalue	Library of type `system` and `ffi`
Addresses	Library of type `system` and `ffi`
Thunks	Library `application` or `system` or `ffi` or `advanced` or `dynamicvmrt`
Ranges	All
Slices	All

Builtin Data Types

Type	Kind
`co.lang.string`	
`co.lang.int`	
`co.lang.bit`	
`co.lang.double`	
`co.lang.float`	
`co.lang.long`	
`co.lang.byte`	
`co.lang.char`	
`co.lang.any`	
`co.lang.dynamic`	
`co.lang.auto`	
`co.lang.infer`	
`co.lang.bool`	
`co.lang.void`	
`co.lang.data`	
`co.lang.value`	
`co.lang.typed`	
`co.lang.untyped`	
`co.mem.region`	
`co.lang.nothing`	
`co.lang.word`	
`co.lang.MatchBindings`	

`co.lang.tag`	
`co.lang.typevalue`	
`co.lang.uninit`	
`co.lang.literal`	
`co.lang.pointer`	
`co.lang.address`	
`co.lang.reference`	
`co.lang.thunk`	
`co.lang.array`	
`co.lang.slice`	
`co.lang.range`	
`co.lang.just`	

Builtin Directives

Built-in Directives

Kind		
`PRAGMA`	“@co.pdap.compiler”, “@co.pdap.scale”	
`DIRECTIVE`	“@co.ddap.movetotop”, “@co.ddap.import”, “@co.ddap.dynamicruntime”, “@co.ddap.use”, “@co.ddap.alias”	
`ANNOTATION`	“@co.dap.template”, “@co.dap.macro”,“@co.dap.operator”, “@co.dap.annotation”, “@co.dap.library”, “@co.dap.module”, “@co.dap.pragma”, “@co.dap.directive”,“@co.dap.native”, “@co.dap.class”, “@co.dap.static”,“@co.dap.instance”, “@co.dap.object”, “@co.dap.inline”,“@co.dap.ctfe”, “@co.dap.friend”, “@co.dap.sealed”, “@co.dap.extension”,“@co.dap.override”, “@co.dap.virtual”, “@co.dap.abstract”, “@co.dap.delegate”, “@co.dap.dynamicscope”,“@co.dap.lexicalscope”,“@co.dap.staticscope”,“@co.dap.mixedscope”, “@co.dap.typeclass”,“@co.dap.matcher”, “@co.dap.constructor”, “@co.dap.oops”, “@co.dap.hokrt”,“@co.dap.hokrlt”, “@co.dap.indexer”, “@co.dap.generic”, “@co.dap.comptime”, “@co.dap.typefromvalue”, “@co.dap.local”, “@co.dap.private”,“@co.dap.public”,“@co.dap.package”,“@co.dap.protected”,“@co.dap.internal”,“@co.dap.export”,“@co.dap.eager”, “@co.dap.lazy”, “@co.dap.packed”, “@co.dap.declare”,“@co.dap.simd”, “@co.dap.reflection”, “@co.dap.mop”,“@co.dap.nested”,“@co.dap.inner”,“@co.dap.final”,“@co.dap.const”,“@co.dap.decorator”,“@co.dap.specialize”	//mop => meta object programming
`DECORATOR`	“@co.dap.before”, “@co.dap.after”,“@co.dap.around”, “@co.fx.onErrExcept”, “@co.fx.InvokeAlways”,“@co.fx.HandleEffect”, “@co.dap.callback”, “@co.dap.defer”,“@co.dap.continuation”, “@co.dap.event”, “@co.dap.scale”, “@co.dap.distributed”,“@co.dap.concurrent”, “@co.dap.parallel”, “@co.dap.subroutine”, “@co.dap.generator”, “@co.dap.goroutine”, “@co.dap.coroutine”,“@co.dap.async”, “@co.dap.promise”, “@co.dap.future”, “@co.dap.thread”, “@co.dap.task”, “@co.dap.fiber”, “@co.dap.process”,“@co.dap.spawn”, “@co.dap.exec”, “@co.dap.fork”, “@co.dap.csp”,“@co.dap.actor”, “@co.dap.synthetic”, “@co.dap.bridge”,“@co.dap.greenlet”, “@co.dap.channel”, “@co.dap.callable”, “@co.dap.iterator”	

Builtin Kinds

Kind	Purpose
`co.lang.type`	
`co.lang.struct`	
`co.lang.cstruct`	
`co.lang.realm`	similar to loader where symbols reside
`co.lang.loader`	class, type, functions loader where objects reside loader takes realm as parameter
`co.lang.class`	
`co.lang.interface`	
`co.lang.union`	
`co.lang.role`	
`co.lang.record`	
`co.lang.property`	
`co.lang.indexer`	
`co.lang.object`	

`co.lang.instance`	
`co.lang.matcher`	
`co.lang.trait`	
`co.lang.mixin`	
`co.lang.extension`	
`co.lang.delegate`	
`co.lang.typeclass`	
`co.lang.concept`	
`co.lang.typealias`	
`co.lang.module`	
`co.lang.unit`	named, non-instantiable function container; same-name struct unit acts as companion
`co.lang.macro`	
`co.lang.template`	
`co.lang.lambda`	
`co.lang.block`	
`co.lang.behavior`	
`co.lang.package`	
`co.lang.signature`	
`co.lang.function`	
`co.lang.method`	
`co.lang.operator`	operator-source-only declaration kind; invalid in ordinary FoLang source
`co.lang.namespace`	
`co.lang.stex`	
`co.lang.kind`	
`co.lang.level`	
`co.lang.order`	
`co.lang.rank`	
`co.lang.newtype`	
`co.lang.opaquetype`	
`co.lang.subtype`	
`co.lang.supertype`	
`co.lang.dependentType`	
`co.lang.refinementType`	
`co.lang.associatedtype`	
`co.lang.hokrlt`	higer rank order kind level types
`co.lang.data`	
`co.lang.enum`	
`co.lang.typetype`	
`co.lang.typekind`	
`co.lang.alias`	
`co.lang.value`	
`co.lang.just`	
`co.lang.nothing`	
`co.lang.library`	
`co.lang.symbol`	
`co.lang.reservedkeyword`	

Builtin Operators

Arithmetic operators

`+`, `-`, `*`, `/`, `%`, `**`

Logical operators

`&&`, `||`, `!`, `&`, `|`

Comparison operators

`==`, `!=`, `<`, `>`, `<=`, `>=`

Other operator and language-token spellings

`@`, `#`, `!`, `~`, `$`, `^`, `(`, `)`, `_`, `` ` ``, `?`, `{`, `[`, `]`, `}`, `\`, `:`, `;`, `"`, `'`, `=`, `.`, `?=`, `:=`, `::=`, `,`, `..`, `...`, `<..`, `..<`, `<..<`, `==>>`, `=>>`, `=>`, `->`, `<-`, `->>`, `<->`, `@@`

Contiguous symbolic spellings that are absent from this inventory and from the active custom-operator table are unrecognized symbolic tokens; the lexer does not split them into shorter operators.

This inventory includes punctuation and reserved token spellings. Presence in this list does not make a spelling declarable as a `co.lang.operator`, nor usable with `mode=overload`, `mode=implements`, `mode=extends`, `mode=inherits` or `mode=override`.

Pre-Declared Operator Glyphs

`λ`, `⒪`, `â`, `t̃`, `∀`, `∃`, `ο`, `ö`, `∪`, `Ṡ`, `Ŝ`, `ṁ`, `Π`, `⇛`, `ƒ`, `ℐ`, `ν`, `ℱ`, `↓`, `∂`, `⊥`, `↧`, `⇓`

Every glyph in this list is language-owned and already has fixed parse properties. It cannot be redeclared with `co.lang.operator`, but it can receive implementations through `mode=overload` in a class, struct companion unit, or built-in extension unit. Until a matching implementation is visible, use of the glyph fails during resolution.

See Pre-Declared Operator Glyphs.

Reserved words

`co`, `let`, `this`, `self (contextual keyword)`, `for`, `forall`, `fo (reserved word)`

Difference between `this` and `self`

- `this` is for instances and objects
- `self` is for classes
- `static` — no shortcut; can be on variable or classname
- Both `self` and `this` can access member variables

Builtin Methods

method	Responsibilit
to_str	
to_int	
to_float	
to_double	
classof	
typeof	
new	
prototype	
proto	
make	
objectof	
instanceof	
is	
as	
iskindof	
has	
hasown	
uses	

match	
matchall	
matchany	
matchnone	
matchtype	
case	
with	
print	
println	
printsp	
echo	
contains	
cast	
to	
dummy	
clone	
of	
for	
when	
where	
then	
callback	
getAttr	
inject	
isinstance	
cast_to	
cast_from	
do	
map	collection operation — admits a lambda callback
flatMap	
orElse	
filter	collection operation — admits a lambda callback
fold	
recover	
peek	
loop	
reduce	collection operation — admits a lambda callback
forEach	collection operation — admits a lambda callback
sortBy	collection operation — admits a lambda callback
groupBy	collection operation — admits a lambda callback
istrue	
isfalse	
if	
elif	
else	
return	
otherwise	
each	
containsVal	

in	
decltype	deduce the type at compile time
replace	
send	
receive	
submitToPool	
submitToEventLoop	
forTypes	will create appropriate object with types for generics (classes, structs, and functions/methods)

Special methods

Method	Responsibility
@@new	Lifecycle method for class
@@init	Lifecycle method for class

Builtin Packages

`co` — root (reserved word)

The only package provided by default.

Sub-package	Responsibility
`co.lang`	All data types and kinds
`co.sys`	file, concurrent, parallel, goto, invoke, bind, call, apply, settimeout, setinterval, scheduler, cron, event
`co.os`	signal, cmd, execute, run, env, getenv, setenv, sleep, exit, cwd, chdir, fork, wait, pipe, dup, dup2, close, readfd, writefd ,random
`co.meta`	ast, instrument, transform, augment, reflect, introspect, patch, inject, create, runtime(eval,proto,prototype,etc), realm
`co.core`	list, set, map, tree, trie, sort, search, array, pointer, ref, address, ptr, matrix, word
`co.native`	load, register, asm, inline, emit, ffi, spawnon[gpu,cpu,npu,apu,fpga,asic,tpu,mki,mcu],arch[x86,x86-64,risc,arm,vliw]
`co.in`	read, readln
`co.out`	println, print
`co.regex`	stex, pattern, match, search
`co.crypto`	rsa, aes, hash, md5, rand, uuid, ssl, tls
`co.dap`	built-in decorators, annotations
`co.ddap`	built-in directives
`co.pdap`	built-in pragmas
`co.net`	tcp, udp, http
`co.const`	`true`, `false`, `none`
`co.encoding`	base64Encode, base64Decode, json, yml, bson
`co.utils`	makeImmutable, makeShared, copyOnWrite, toSnapshot — object behaviour policies
`co.dynamic`	dynamic capabilities
`co.runtime`	
`co.compiletime`	
`co.macro`	
`co.pattern`	
`co.control`	
`co.cpca`	concurrent, async, await, defer, lazy, parallel, process, thread,fiber, task, coroutine,continuation,cps, pool, channel ...
`co.hokrtl`	

`co.hokrt`	

FoLang Philosophy — Uniform Object Model

Core Principle

Everything in **FoLang** is an object. That is the reason for the name **FO** — **Functional Objects**.

FoLang gives the programmer one uniform object model across all types instead of forcing them to switch between separate type families for:

- ordinary values
- immutable values
- shared/concurrent values
- copy-on-write values
- literal values

The programmer writes against a single conceptual model and opts into the required object behaviour only when needed.

Uniform Object Principle

In FoLang, **everything is an object**.

Unless explicitly stated otherwise, the reference and mutation rules in this section apply to **managed FoLang objects**. `co.lang.cstruct` remains an object in the language model, but it is an explicitly value-semantic ABI representation: assignment and parameter passing copy its value rather than a managed object reference.

This includes:

- scalar objects
- collection objects
- user-defined objects
- ADT objects
- function objects

Functions are objects in the same sense as all other values.
This is true for both **named functions** and **anonymous functions**.

FoLang does not introduce a separate semantic model for functions, scalars, UDTs, or ADTs.
They all follow the same core object principles:

- assignment of managed objects copies references
- assignment of `co.lang.cstruct` copies its value
- `==` compares values
- mutation is applied through the object
- behaviour policies such as immutability, shared, copy-on-write, and literal conversion apply uniformly where meaningful

So in FoLang, the programmer does not need one mental model for data objects and another for function values.
The language treats them under one consistent object model.

---

1. Default Object Model

Every value in FoLang is an object and is **mutable by default**.

This includes:

```
co.lang.int, co.lang.float, co.lang.string   → built-in scalar types
user-defined types (structs, classes, ADTs)  → user-defined objects
co.core.list, co.core.map, co.core.array     → built-in collections
```

Example:

```
positive_int co.lang.int = 10;
```

`positive_int` denotes an object.
Its current value is `10`.
By default, it is mutable.

---

2. Assignment, Aliasing, and Mutation

FoLang distinguishes three related ideas:

- **assignment**
- **aliasing**
- **mutation**

2.1 Assignment copies references

When one object variable is assigned to another, FoLang copies the **reference**, not the internal contents.

```
b := a
```

After this, `a` and `b` refer to the same object.

2.2 Mutation changes object contents

If two names refer to the same object, mutating through one name is visible through the other.

```
Employee co.lang.class = {
    Name co.lang.string
}

a Employee = { Name = "Kamesh" };
b := a;
c Employee = { Name = "Kamesh" };

a == b;   // true
a == c;   // true
b == c;   // true

b.Name = "Ramesh";   // changes a.Name too
c.Name = "Ramesh";   // does not change a.Name
```

Why:

- `a` and `b` are the **same object**
- `c` is a **different object** with the same earlier value

2.3 `==` means value equality

In FoLang, `==` compares **values**, not object identity.

That rule is uniform across all object kinds, including built-in types and user-defined types.

So:

```
a == b
```

means:

- compare contents deeply
- return `true` when the values match
- even if `a` and `b` are not the same object

2.4 Mutation accessors

The accessor used for mutation depends on the object type:

```
scalar (int, float, bool, string, char, byte)   → .value
struct field                                    → .fieldname
class member                                    → .membername
map entry                                       → .key
array / list element                            → .index
```

Examples:

```
count.value = 30
emp.name = "Rao"
marks.math = 95
nums.0 = 42  or nums[0] = 42 //both forms supported
```

2.5 Function-call behaviour

Function parameters introduce a **new local binding**.

So:

- rebinding a parameter does **not** affect the caller
- mutating the object through the parameter **does** affect the caller if both still refer to the same object

```
checkPositive(a co.lang.int)->(co.lang.bool) = {
    a = 20;         // local rebinding only — caller unchanged
    a.value = 30;   // object mutation — caller sees 30
}
```

If `positive_int` is passed to `checkPositive(a)`:

```
a = 20         → local rebinding only
a.value = 30   → mutates the passed object
```

---

3. Literal Objects

All literals in FoLang create **Literal Objects**.

```
"Kumar"   → Literal Object — string
42        → Literal Object — int
true      → Literal Object — bool
```

A Literal Object is an **anonymous object created from a literal expression**.

Literal Objects participate in value equality just like every other object in FoLang:

```
10 == 10   // true
```

In FoLang, `==` compares **values**, not object identity.

So two literal expressions with the same value compare equal, but that does not mean they are the same object.

Binding a Literal Object

When a literal is assigned to a named object, the name refers to the object created from that literal expression.

```
a co.lang.int = 10;
```

Here, `a` refers to the anonymous integer object created from literal `10`.

So:

```
a.value = 30;
```

mutates that same object, and `a` now has value `30`.

The important points are:

- literal expressions create anonymous objects
- literal-created objects are still ordinary objects unless another policy is applied
- `==` compares values, not identity
- once bound to a name, a literal-created object behaves like any other normal mutable object
- immutability applies only when explicitly requested, for example through `makeImmutable(...)`

Why Literal Objects Cannot Be Mutated Directly

Mutation can occur in two ways:

1. Through rebinding

   a co.lang.int = 10; a = 20;

   The above statement is valid because a is valid identifier and named handle to `co.lang.int` type.

   10 = 20; // ❌ invalid

   The above statement is invalid because 10 is not a valid identifier and if 10 in literal object type there is no named handle.

   The rebinding happens through named handles which are valid identifiers of `folang`

2. Through a property or method

   An object may also be mutated through one of its mutable properties or methods. a co.lang.int = 10; a.value = 20;

   A literal object cannot be mutated directly because it does not provide properties/methods that can be accessed.

   10.value = 20; // ❌ invalid

```
a co.lang.int = 10;
a.value = 30;
```

is valid after binding, a bare literal such as `10` cannot be mutated directly because there is no name or handle through which to perform mutation.

Not the Same as `toSnapshot(...)`

A literal object created by a literal expression is **not** the same thing as the internal snapshot representation produced by `co.utils.toSnapshot(...)`.

- literal expression → anonymous object
- `toSnapshot(x)` → internal snapshot representation used to reconstruct a fresh local object later

These are related concepts, but they are not the same mechanism.

Summary

- literal objects are real objects
- literal expressions create anonymous objects
- identical literals compare equal by value
- identical literals are not automatically the same object
- literal-created objects are mutable by default once bound to a handle
- only `makeImmutable(...)` makes an object immutable

---

4. Object Behaviour Policies

Any object can be given a behaviour policy using `co.utils.*`.

All four policy calls are **in-place transformations**:

- the object itself changes behaviour kind
- there is no wrapper object
- there is no alternate binding to capture
- the original name now refers to the transformed object

All policies are **deep by default**.
They flow through nested structs, members, collection elements, and all reachable objects in the graph unless the specification later states otherwise.

4.1 Immutable

```
co.utils.makeImmutable(positive_int)
```

After this call the object itself is immutable.

This is an in-place transformation, not a wrapper.

Any attempt to mutate the object or any reachable object beneath it is a **compiler error** where detectable statically, and a **runtime error** otherwise.

```
positvie_int co.lang.int = 10;
positive_int.value = 30   // ❌ compiler error
positive_int = 30         // ❌ compiler error
```

For nested objects:

```
Employee co.lang.struct = { address Address; }
Address co.lang.Address = {city co.lang.string; state co.lang.string; lane co.lang.string;pin co.lang.string;}


emp Employee = Employee{
    address = Address{
            city: "Pune";

    }
}
co.utils.makeImmutable(emp);

emp = Employee{
    address=Address{
            city: "ABC";

    }
} // ❌ compiler error


emp.address = newAddress              // ❌ compiler error
emp.address.city.value = "Mumbai"     // ❌ compiler error
```

Immutability is deep and total.

4.2 Value Immutable

```
positive_int co.lang.int = 20;

co.utils.makeValueImmutable(positive_int);

positive_int.value = 30; // ❌ current object is immutable
positive_int = 30;       // ✅ binding may reference a new object

co.utils.makeValueImmutable(emp);

emp.address.city = "ABC";       // ❌
emp.address.city.value = "ABC"; // ❌

emp = Employee{
    address= Address{
        city: "ABC";
    }
}; // ✅
```

Difference between Immutable and Immutable Value

```
    makeValueImmutable(x)
        └── current object graph cannot change

    makeImmutable(x)
        ├── current object graph cannot change
        └── binding cannot be reassigned
```

Table

Operation	Binding	Current value/object graph
`makeValueImmutable(x)`	Mutable	Immutable
`makeImmutable(x)`	Immutable	Immutable

4.3 Shared

```
co.utils.makeShared(positive_int)
```

A Shared Object is safe for concurrent and multi-threaded use.

The programmer continues to use the same object model — same accessors, same syntax — while FoLang guarantees the required safety properties internally.

Shared behaviour is deep:

- all reachable objects within the shared object are also shared

Note on Analogies

Comparisons to things like Java's `AtomicInteger` or `ConcurrentHashMap` are **analogies only**.

They may help explain the concept, but they are **not part of the formal language contract**.

FoLang specifies the behavioural guarantee, not the exact runtime implementation.

A good explanatory statement is:

> A shared `co.lang.int` may be thought of similarly to an atomic integer, and a shared map may be thought of similarly to a concurrent map. These comparisons are explanatory only. FoLang does not require any particular internal runtime representation.

4.4 CopyOnWrite

```
co.utils.copyOnWrite(positive_int)
```

A CopyOnWrite Object passes by reference like a normal object.

The copy is deferred.
It is created only when mutation is attempted in a context that must not affect the original source object.

```
process(a co.lang.int)->() = {
    a.value = 99;   // deep copy is made here — caller's object unchanged
}

co.utils.copyOnWrite(positive_int)
process(positive_int)

// positive_int still holds its old value
```

Copy-on-write is deep.
When a copy is made, the entire reachable object graph is copied.

Cyclic references must be handled by the runtime/compiler's structural clone logic with proper identity tracking.

4.5 toSnapshot

```
co.utils.toSnapshot(positive_int)
```

`toSnapshot` converts an object into a **snapshot representation** — a value descriptor, not a live object. The snapshot representation itself cannot be mutated.

When passed to a function, the compiler/runtime uses the snapshot representation to construct a fresh independent local variable bound to the parameter name. That local is a normal mutable object with no shared identity with the original. All of this happens automatically — the developer writes only `co.utils.toSnapshot(positive_int)`.

```
process(a co.lang.int)->() = {
    a.value = 99;   // mutates the fresh local — positive_int completely unaffected
}

process(co.utils.toSnapshot(positive_int))

// positive_int unchanged
```

The flow:

```
positive_int
    ↓  co.utils.toSnapshot(positive_int)
    snapshot representation         ←  value descriptor — immutable by nature, not by policy
    ↓  passed to function
    compiler constructs fresh local 'a' from the snapshot representation
    'a' is a new independent normal object — mutations never reach positive_int
```

`toSnapshot` is deep — the snapshot representation covers the entire reachable object graph.

---

5. Policy Summary

Object Kind	Mutation allowed	Caller sees mutation	Thread safe	Copy on write	Deep
Literal expression object	❌ directly in source — no handle	—	value-like use	—	✅
Normal (default)	✅ via accessor	✅	❌	❌	—
Immutable	❌	—	✅	—	✅
Shared	✅ via accessor	✅	✅	❌	✅
CopyOnWrite	✅ on own copy	❌	✅	✅	✅
toSnapshot result	✅ on reconstructed local object	❌	independent snapshot	—	✅

---

6. No Type Fragmentation

FoLang deliberately avoids special types for mutability or concurrency concerns.

There is no need for separate public type families such as:

```
ImmutableInt
SharedMap
CopyOnWriteList
AtomicInt
ConcurrentMap
```

Instead, FoLang keeps the type model uniform and applies behaviour policies through `co.utils.*`.

In many ecosystems, a programmer must constantly choose between:

```
normal integer      vs atomic integer
normal map          vs concurrent map
mutable value       vs immutable wrapper
raw structure       vs copy-on-write structure
```

FoLang instead aims to provide:

- one object model
- one programming style
- one mental model

while still allowing the programmer to opt into immutability, sharing, copy-on-write, or literal conversion when needed.

---

7. What Still Needs Precision

The philosophy is sound, but the formal specification still needs to define these precisely:

```
aliasing behaviour                    → when two names refer to the same object
rebinding vs mutation interaction     → edge-case coverage
deep policy propagation rules         → exactly which objects are reachable
cyclic reference handling in COW      → identity-tracking specification
visibility of mutation across calls   → formal function-call model
identity vs value equality            → what operators or built-in expose identity
policy stacking                       → can an object be both shared and COW?
                                          can an object be both shared and immutable?
```

---

8. Formal Philosophy Statement

> All managed FoLang objects use reference semantics by default. `co.lang.cstruct` is an explicitly value-semantic ABI representation and is an exception to managed-object reference semantics.
> In FoLang, everything is an object and managed objects are mutable by default.
> Assignment of managed objects copies references, `co.lang.cstruct` assignment copies values, and `==` compares values deeply.
> Developers may opt into immutability, shared behaviour, copy-on-write behaviour, or literal conversion depending on their needs.
> All behaviour policies are deep — they apply to the entire reachable object graph unless stated otherwise by the formal specification.
> This policy model is uniform across managed types, so programmers do not need separate type families for ordinary, immutable, concurrent, or snapshot-oriented use.
> Familiar analogies such as atomic integers or concurrent maps may help explain the design, but they are not part of the formal implementation contract.

FoLang Language Definition and Documentation License

- copied and redistributed in any medium or format;
- adapted, modified, translated, and extended;
- used for commercial or non-commercial purposes.

A person exercising these permissions must:

- give appropriate attribution;
- provide a link to the CC BY 4.0 licence;
- indicate whether modifications were made;
- retain notices supplied with the licensed material where required;
- not imply endorsement by the FoLang project or its contributors.

Independent Use and Implementation

This licence applies to the copyrightable expression contained in the FoLang language definition and documentation.

It does not restrict the independent use or implementation of programming-language ideas, concepts, features, procedures, systems, or methods of operation.

For example, this licence does not claim exclusive rights over general programming-language concepts such as:

- continuations;
- modules;
- structs and classes;
- dynamic, lexical, or mixed scope;
- functions and closures;
- pattern matching;
- algebraic data types;
- dependent types;
- concurrency and parallelism;
- coroutines and asynchronous execution. etc.,

Another project may independently implement such concepts without copying the copyrightable expression of the FoLang documentation.

The FoLang-specific documentation describing how these concepts are expressed through FoLang syntax, grammar, semantic rules, examples, diagrams, and explanatory text remains subject to this licence when that material is copied or adapted.

Software Licences

This licence does not change or replace the licences applied to:

- the FoLang frontend source code;
- the default backend source code;
- other FoLang software components;
- generated compiler binaries;
- third-party software;
- third-party documentation or assets.

The FoLang frontend and backend continue to be governed by their separately stated software licences.

Other Rights

CC BY 4.0 does not grant rights relating to:

- trademarks;
- project names and branding;
- logos;
- patents;
- third-party material;
- privacy, publicity, or personality rights.

Such rights, where applicable, remain governed separately.

Suggested Attribution

> FoLang Language Definition and Documentation, including its syntax, grammar, and semantic-rule descriptions, by Kemeswara Rao Mithipati and FoLang contributors, licensed under CC BY 4.0.

When modifications are made, the attribution should also indicate that the material was modified.

Example:

> Based on the FoLang Language Definition and Documentation by Kemeswara Rao Mithipati and FoLang contributors, licensed under CC BY 4.0. Modified from the original. ***

Appendix A - Complete FoLang EBNF Grammar

The following grammar is the normative lexical and syntactic grammar for FoLang.

```
(*
   FoLang consolidated EBNF — decision-complete draft, revision 22
   Derived from language-ref.md.

   Revision 6 incorporated the clarified FoLang termination and identifier
   rules. Revisions 7 through 11 refined lexical disambiguation and selected a
   C++-compatible literal subset. Revision 12 formalizes the distinction
   between a body-closing brace and a brace that merely closes an expression.
   Revision 13 aligns dependent-index and method-activation semantics with the
   language reference. Revision 14 records the physical-nesting exceptions,
   ordinary local-function scope, @co.dap.inner call-site lexical context,
   and the package-level annotated-function constraint. Revision 15 recorded
   the initial project-local operator-source bootstrap and its later
   project-local, non-exported operator model.
   Revision 16 synchronized the attached merged grammar with the finalized
   operator decisions, and revision 17 updated its decision metadata without
   changing production bodies. Revision 18 aligns type and container
   declarations with the current parser contract: co.lang.dependentType is both
   a type-producing return kind and a legal direct declaration kind; a function-shaped type
   constructor has exactly one type-producing result; generic clauses are kept
   by every supported named type/container class; and a comma in a grouped form
   must be followed by another item. Revision 19 makes application-entry type
   declarations explicitly non-generic and restricted to the reference's
   allowlist; admits contextual `.for` member access; and records normalized
   companion/class operator ownership, mode, and operand rules.

   FoLang adopts a selected C++-compatible subset of built-in literal
   spellings, subject to these FoLang rules:
     - the boolean literals are co.const.true and co.const.false
     - the null/none literal is co.const.none
     - numeric digit separators are not supported
     - a floating point requires a digit on both sides of the decimal point
     - the C++ pointer literal nullptr is not introduced
     - the C++ user-defined-literal operator"" mechanism is not introduced
   A user-defined-type value is written with object-construction, for example
   Employee{name: "Rao", id: 1}, which is an expression rather than a literal
   token. There is therefore no separate user-defined-literal production.

   Notation:
     =       defines a production
     ;       ends a production
     |       alternative
     [ ... ] optional
     { ... } zero or more
     ( ... ) grouping
     "..."   terminal text
     ? ... ? a precisely described lexical or context-sensitive terminal

   STATUS
   ------
   The language reference remains authoritative. Where that document did not
   select a lexical or parsing rule, this revision makes an explicit design
   decision based on FoLang requirements and C++ backend compatibility. Every such
   decision is labelled DECISION-* in this grammar and in the companion
   decision register so it can be copied into language-ref.md.

   PRINCIPAL DECISIONS
   -------------------
   DECISION-SYN-001: A semicolon is mandatory after every simple statement
                     whose production uses statement-end. Newlines never
                     terminate statements and there is no semicolon insertion.
                     Built-in directives are self-delimiting and do not take a
                     trailing semicolon. A block-bodied declaration is not
                     followed by a semicolon.
   DECISION-SYN-006: TERMINATION MODEL, stated once and applied uniformly.

                     HARD ends. Exactly one is always required.
                       ";"  ends a simple statement, an expression-bodied or
                            type-bodied declaration, and every forward
                            declaration.
                       "}"  ends a declaration body, a function body, a
                            function-pattern body, or a standalone block
                            statement. No ";" follows a body-selected "}".

                     SOFT end within one construct.
                       ","  closes the current enum variant, map/object entry,
                            annotation item, parameter, argument, or grouped
                            declarator while allowing another item to follow.
                            It does not terminate the enclosing statement. A
                            trailing "," is permitted where DECISION-COL-001
                            allows it.

                     NO terminator.
```

```
                    Built-in directives and annotations are
                    self-delimiting; see DECISION-DIR-001.

                  EXPRESSION-BRACE RULE. A "}" that closes object
                  construction, a map expression, an anonymous class
                  expression, or another braced expression ends only that
                  expression. The enclosing simple statement still needs
                  its ";":

                      emp := Employee{ id: 1, name: "Rao" };
                      this.return Employee{ id: 1 };
                      cfg co.lang.map = { "a": 1, "b": 2 };

                  BODY-BRACE RULE. The following direct body forms end at
                  their closing brace and take no following semicolon:

                      Employee co.lang.struct = { id co.lang.int; }
                      classify(n) => { this.return "positive"; }
                      someFArg co.lang.function =
                          (a co.lang.int)->(co.lang.int) = {
                              this.return a;
                          }

DECISION-SYN-007: Body-versus-expression selection is encoded explicitly.
                  A direct body alternative uses body-closure-guard, which
                  rejects an immediately following semicolon. A competing
                  expression alternative uses non-block-expression or
                  non-anonymous-function-expression so that the same direct
                  body cannot be reparsed as an expression plus semicolon.
                  Parenthesized, postfixed, or otherwise composed braced
                  expressions remain expressions and require the enclosing
                  statement terminator.

DECISION-SYN-008: PHYSICAL NESTING. Independent named type and container
                  declarations remain package-owned primary declarations and
                  cannot be physically declared inside another declaration,
                  function, or executable block. Member functions, lifecycle
                  methods, and signature/module type components are members
                  or contract slots, not independently nested package
                  declarations.

                  Two explicit syntactic exceptions exist:

                    local function    local-function-declaration is a named
                                      block-local function form. It requires
                                      a return-type clause and a block body.

                    anonymous form    anonymous-function-expression,
                                      anonymous-class-expression, block and
                                      other value expressions may appear
                                      wherever expression is admitted.
                                      forall-type is an anonymous
                                      polymorphic type expression wherever
                                      type-expression is admitted.

                  A lambda-expression is narrower: it is admitted only as
                  a direct argument of map, filter, reduce, forEach, sortBy
                  or groupBy. Nesting that call inside another expression
                  does not remove the direct-argument relationship.

                  None of these anonymous forms creates an independently
                  addressable package declaration identity. @co.dap.local,
                  @co.dap.nested and @co.dap.inner annotate separately
                  declared declarations and do not physically relocate them.

DECISION-SYN-009: ANNOTATED PRIMARY FUNCTIONS. Package source files forbid
                  loose ordinary functions. annotated-function-primary is a
                  syntactic envelope only for annotation-defined primary
                  declaration kinds, such as a macro, decorator, native
                  boundary declaration, or another annotation whose
                  specification explicitly grants primary-declaration
                  status. Adding an arbitrary annotation to an ordinary
                  function does not make it legal at package-file scope.
                  This legality check is semantic because annotation meaning
                  is resolved after parsing.

DECISION-SCOPE-001: ORDINARY LOCAL FUNCTIONS. A local-function-declaration
                  has block-local identity and is not a package member. Its
                  free runtime names resolve from the lexical declaration
                  context in which the function is physically declared.
                  Calling it does not replace that environment with the
                  caller's runtime scope.

DECISION-SCOPE-002: @co.dap.inner is an association annotation, not local-
```

```
                    function syntax and not physical nesting. An executable
                    declaration annotated @co.dap.inner resolves parameters
                    and locals first, then free runtime names through the
                    lexical scope chain of the active attachment or call site.
                    It does not search arbitrary caller frames as
                    @co.dap.dynamicscope does. Types, imports, annotations and
                    co.* names continue to use ordinary static resolution.
                    These are semantic checks; the general annotation
                    production intentionally remains unchanged.

DECISION-TYP-004: A dependent-type argument and an array dimension are
                    INDEX positions. An index is an integer literal or a
                    name; no operator, call or index expression may appear.
                    A name must resolve to a type or value parameter in
                    scope, or to a @co.dap.const compile-time constant.
                    @co.dap.final denotes an immutable binding and does not
                    qualify, because an immutable value need not be known at
                    compile time.

                    An index is a NON-NEGATIVE integer. Enforcement is split
                    across two phases, and both reject at compile time:

                      literal index    the grammar guarantees it. No prefix
                                       operator is reachable from
                                       dependent-index, so -1 cannot be
                                       written. ->([-1]) and Vector(-1) are
                                       parse errors positioned at the "-".
                      named index      the checker must verify it. After
                                       substituting a @co.dap.const name it
                                       rejects the program unless the
                                       resolved value is a non-negative
                                       integer:

                                           @co.dap.const OFFSET co.lang.int = -1;
                                           buf co.lang.int->([OFFSET]);
                                           v Vector(OFFSET);

                                       Both are compile errors. The checker
                                       already resolves the constant to
                                       compare types under DECISION-TYP-005,
                                       so the test costs nothing extra, and
                                       the diagnostic should name the
                                       constant and its declaration site.

                    The same check rejects a constant that resolves to a
                    non-integer, and a name that is not a @co.dap.const and
                    not a parameter in scope. Because both index positions
                    share the dependent-index production, the rule applies
                    identically to array dimensions and to dependent-type
                    arguments. Zero is permitted; language-ref.md declares
                    the zero-length array co.lang.int->([0]).

DECISION-TYP-005: DEPENDENT-TYPE EQUALITY, stated once. Two dependent types
                    are equal when their constructors are the same and their
                    indices are pairwise equal. An index comparison has
                    exactly three cases:

                      integer literal    compared by value
                      @co.dap.const      substituted by its literal value,
                                         then compared by value
                      parameter          compared by name identity

                    Consequences, all decidable by inspection:

                      Vector(3)    vs Vector(3)      equal
                      Vector(n)    vs Vector(n)      equal
                      Vector(n)    vs Vector(m)      REJECTED
                      Vector(SIZE) vs Vector(1024)   equal when
                                                     @co.dap.const SIZE = 1024
                      Vector(n+1)  vs Vector(1+n)    not expressible;
                                                     DECISION-TYP-004 rejects
                                                     the index at parse time

                    FoLang deliberately does NOT decide index equality up to
                    arithmetic. Accepting n+1 == 1+n requires symbolic
                    reasoning, and there is no partial version of it: the next
                    reasonable request is 2*n == n+n, and the checker becomes
                    a theorem prover by accretion. Length-arithmetic
                    signatures such as concat(Vector(n), Vector(m)) ->
                    Vector(n+m) are therefore out of scope; return a
                    dynamically sized type and check at run time instead.

                    What remains covers the common cases, because they need
                    only same-parameter identity:
```

```
                  multiply(a Matrix(r, n), b Matrix(n, c)) -> Matrix(r, c)
                  dotProduct(a Vector(n), b Vector(n)) -> co.lang.int
                  zip(a Vector(n), b Vector(n)) -> Vector(n)

DECISION-TYP-006: Dependent types are CHECKED, never INFERRED. Every
                  dependent type appears in a written signature, so the
                  checker runs only in check mode. This is what keeps
                  checking decidable without a constraint solver, and it is
                  why FoLang does not adopt Hindley-Milner style
                  whole-program inference. Relaxing it would reintroduce the
                  need to infer index VALUES, not merely types.

DECISION-SEM-002: ACTIVATION AND METHOD RESOLUTION. An instance may be
                  activated by @co.ddap.use, making its functions callable
                  as methods on the receiver. Activation is explicit and
                  block-scoped, so importing a package activates nothing and
                  can never change how an existing call resolves. For a call
                  xs.map(f) the compiler tries, in order: a class method or
                  companion-unit function on the receiver's type; an
                  activated extension; an activated instance function whose
                  typeclass declares the method with the receiver as first
                  parameter; otherwise an error. First match wins, so a
                  type's own declarations always outrank anything activated
                  into scope. Within one scope a method name may be
                  activated at most once per receiver type; a second
                  activation is an error naming both sources. None of this
                  is expressible syntactically; see the note on
                  use-directive.

DECISION-SEM-001: INSTANCE SELECTION AND PLACEMENT. A typeclass instance is
                  selected BY NAME; FoLang never searches visible packages
                  for an instance matching a type. Because nothing is
                  inferred, no coherence check is needed and unique package
                  names or aliases fully disambiguate. Separately, an
                  instance is declared in the package defining the typeclass
                  or the package defining the type, that exact package and
                  not a sub-package. That placement rule is a convention
                  enforced during name resolution, not a correctness
                  requirement. Both are semantic constraints the grammar
                  cannot express; instance-declaration accepts a misplaced
                  instance and the compiler reports it. See the note on that
                  production.

DECISION-DIR-001: Standalone built-in directives end at their complete
                  directive form. A closing argument parenthesis, when the
                  directive has arguments, is sufficient; no semicolon is
                  accepted or required by the directive production.
DECISION-OP-001:  Built-in operators use the precedence table encoded in
                  section 11. Assignment has the lowest built-in precedence.
DECISION-OP-002:  Runtime assignment operators are right-associative.
                  Therefore a = b = c parses as a = (b = c). An assignment
                  expression yields the assigned value. FoLang's separately
                  specified target-first, left-to-right evaluation order is
                  retained.
DECISION-OP-003:  := and ?= are statement-level definition operators, not
                  general expression operators; they cannot be chained.
                  ::= remains reserved and is not accepted by this grammar.
DECISION-OP-007:  A constant-expression may contain a registered custom
                  operator at any declared precedence, but no runtime
                  assignment operator may occur anywhere in its expression
                  subtree. Grouping, arguments, literals, and other nested
                  expression forms do not bypass this recursive guard.
DECISION-LEX-001: Source files are UTF-8. FoLang identifiers are the
                  ASCII subset [A-Za-z][A-Za-z0-9_]*, but may not contain
                  consecutive underscores or end in an underscore. A lone
                  _ is a dedicated contextual token, never an identifier.
DECISION-LEX-002: // and non-nesting /* ... */ comments are supported.
                  Line breaks are ordinary whitespace outside literals.
DECISION-LEX-003: After comments, literals, and closed scanner-known
                  composite spellings such as @@new are recognized, the
                  lexer consumes each remaining complete maximal contiguous
                  run of symbol characters as one symbolic token candidate. The run
                  is never split into shorter operators as a fallback. It is
                  classified as a structural spelling, contextual metadata,
                  registered expression operator, or unrecognized token.
DECISION-LEX-010: A registered expression operator containing more than one
                  symbol character requires an explicit boundary on each
                  operand-facing side. Whitespace, comments, and applicable
                  delimiters establish a boundary. Structural and metadata
                  spellings are not subject to this rule merely because they
                  contain multiple symbols.
DECISION-BACKEND-001: Each resolved user-defined FoLang identifier is lowered
                  to C++ by appending the suffix _fo. Built-in names,
```

keywords, and compiler-internal symbols use their own
compiler-defined lowering rules.
DECISION-LIT-000: FoLang accepts the selected C++-compatible numeric,
character, and string literal spellings for the configured
backend dialect and preserves their complete source
lexemes. A C++ backend may emit those lexemes unchanged.
co.const.true, co.const.false, and co.const.none use
backend-defined lowering and are not raw C++ literals.
The C++ pointer literal nullptr is not introduced.
DECISION-LIT-004: WITHDRAWN in revision 7. FoLang has no user-defined
literal token. A user-defined-type value is built by
object-construction, which is an ordinary expression.
DECISION-LIT-001: Integer literals use C++-compatible binary,
leading-zero octal, decimal, and hexadecimal forms and
standard integer suffixes. FoLang does not permit digit
separators.
DECISION-LIT-002: Floating literals use the selected C++-compatible
decimal and hexadecimal forms, exponent rules, and
standard or backend-supported extended floating suffixes.
FoLang does not permit digit separators and requires a
digit on both sides of a decimal point.
DECISION-LIT-003: The alpha release accepts only unprefixed character and
string literals without escapes. Encoding prefixes,
escapes, universal character names, and raw strings are
reserved and produce unsupported-feature diagnostics.
DECISION-COL-001: Commas separate enum variants, map entries, annotation-map
entries, and object initializers; a trailing comma is
allowed. In an enum, the comma is a soft item boundary and
the closing brace is the hard structural end of the enum
body. Object and annotation-map fields use colon.
DECISION-BLK-001: A block may end in one unterminated tail expression. That
expression is the block value and is not a statement.
DECISION-EXT-001: The active precedence table is assembled from the
language-owned built-in and pre-declared registrations plus
the current compilation's custom registrations. Every
implementation uses mode=overload; omission defaults to
overload. mode=override, mode=extends, and mode=define are
rejected. The alpha profile implements infix, prefix and
postfix; other fixities are reserved.

REVISION 22 SYMBOLIC-RUN AND OPERATOR-BOUNDARY MODEL

DECISION-LEX-003 is revised: after comments, literals, and closed
scanner-known composite spellings are recognized, the
lexer consumes each remaining maximal contiguous run of
symbol characters. The complete run is classified as a
fixed structural spelling, contextual metadata spelling,
registered expression operator, or unrecognized symbolic
token. An unrecognized run is rejected without fallback
splitting into shorter operators.
DECISION-LEX-010: A registered expression operator containing more than one
symbol character requires an explicit operand boundary on
every operand-facing side. Whitespace, comments, and
applicable delimiters provide a boundary. Structural and
metadata spellings are exempt unless parsed as expression
operators.
DECISION-OP-004: WITHDRAWN in revision 24. ++ and -- are removed from the
built-in prefix/postfix grammar and precedence table. They
receive no special prohibition; an unregistered contiguous
++ or -- run fails under DECISION-LEX-003.

REVISION 21 UNIFORM OPERATOR REGISTRATION MODEL

DECISION-OPLIB-001: Registered built-in, pre-declared, and custom symbols all
receive implementations through ordinary operator overload
functions in a built-in extension unit, struct companion
unit, or class. Operator functions cannot be loose package
functions. Distinct normalized signatures are overloads; an
equivalent signature is a duplicate. Operator override and
extends modes remain unsupported.
DECISION-OPLIB-003: A custom symbol has exactly one registration in the fixed
operator library, but may have zero or more distinct
implementation overloads. Language-owned symbols cannot be
registered locally.
DECISION-OPBOOT-001: operator_library_folder selects the project-local fixed
operators.fol bootstrap surface. Missing configuration,
folder, or file means no local custom registrations.
DECISION-OPBOOT-002: A dedicated lexer/parser accepts exactly the source-only
marker @co.dap.library(type=operator) and the fixed
_ co.lang.library = { ... } declaration. Its body admits
only co.lang.operator registrations and emits no artifact.
DECISION-OPART-001/002/003: WITHDRAWN. Operator registrations and tables do
not cross an ordinary library boundary and are never
serialized into .folib or .folenc artifacts.

```
DECISION-OPBOOT-003: Before ordinary tokenization, the compiler combines the
                     language-owned registrations with the current compilation's
                     local custom registrations and builds immutable maximal-
                     munch, fixity, precedence, associativity, and arity tables.
                     Imports contribute no operator metadata.
DECISION-OPDECL-001: A custom registration has required fixity, precedence,
                     associativity, and arity properties and optional
                     commutative, idempotent, identity, foldable, vectorizable,
                     distributes_over, and desugar properties. Required keys occur once;
                     optional keys occur at most once. Implementation annotations
                     carry only symbol and optional mode.
DECISION-OPDECL-002: The documented mathematical/modifier glyph set is
                     pre-declared by the language with complete parse properties
                     and no required implementation. It cannot be registered
                     locally and is implemented through mode=overload.
DECISION-OPDECL-003: Symbols are recognized globally in the compilation;
                     implementations are selected later by owner, scope,
                     activation, operand types, and normalized signature. One
                     symbol/one concept is readability guidance, not a compiler
                     restriction.
DECISION-OPDECL-004: The operator source uses a separate grammar. Both the
                     operator-source lexer and ordinary lexer consume complete
                     symbol runs with no fallback splitting. A custom symbol is
                     recognized only by an exact whole-run match. Symbols that
                     contain // or /* are invalid because comments take
                     lexical priority. co.lang.operator is not an ordinary
                     declaration kind.

REVISION 5 DECISIONS
--------------------
DECISION-ANN-001: An annotation argument and an annotation-map entry both
                  accept "=" or ":" as the key/value binder, and a bare key
                  with no value is a flag. This admits the reference forms
                  @co.dap.oops(A: { inherit:true }) and
                  @co.dap.generic(type={T:{typename}, R:{variance:invariant,
                  bound=Number}}), which mix both binders freely.
DECISION-TYP-001: Every type derivation, not only a pointer derivation, may
                  carry a trailing attribute list. This admits the reference
                  form co.lang.int->(&, meta={type=out}).
DECISION-TYP-002: A function-shaped type constructor has exactly one
                  type-producing return item and its body binds a
                  type-expression, so
                  Vector(n co.lang.int)->(co.lang.dependentType) =
                  co.lang.int->([n]); is well formed. An algebraic `|` remains
                  part of a data/type-expression binding, not a second return
                  item. co.lang.dependentType is also a legal direct type
                  declaration kind: a declaration's container kind is the
                  kind produced by its defining expression. Where a token
                  sequence satisfies both type-expression and expression,
                  the type-expression reading is selected.
DECISION-TYP-003: An array dimension may be elided in any position, so
                  ->([,]) and ->([]) are both well formed.
DECISION-GEN-001: A generic parameter may declare its arity, so a
                  higher-kinded parameter is written Transformer(F(_), G(_)).
                  An arity slot is "_" or a named placeholder.
DECISION-KIND-001: Any built-in kind that has no dedicated production is
                   parsed by general-kind-declaration, which accepts a block
                   body, a type-expression body, or a forward form. This
                   stops a declaration such as blockormacro co.lang.kind =
                   block | macro; from silently parsing as a variable.
                   Kind names that also appear in the built-in data-type
                   table (co.lang.value, co.lang.nothing, co.lang.just) are
                   deliberately excluded; in a declarator they read as types.
DECISION-SYN-003: The named local-function exception uses a statement-level
                  local function declaration with both a return-type clause
                  and a block body. This admits someother()->()={...} while
                  keeping foo(); an expression statement rather than a
                  forward declaration. Its lexical scope is defined by
                  DECISION-SCOPE-001.
DECISION-SYN-004: Annotations may prefix an expression statement, admitting
                  @co.dap.lazy applied to x = add(1, 2);.
DECISION-SYN-005: A block is a statement in its own right and takes no
                  trailing semicolon.
DECISION-FUN-001: The "=" before a function block body is optional, so both
                  add(a T)->(T) = { ... } and add(a T)->(T) { ... } parse.
DECISION-FUN-002: A closure declaration uses an equals sign before one or
                  more parameter lists and ==>> before its expression body.
                  One parameter list declares an ordinary closure; two or
                  more parameter lists declare a curried closure.
DECISION-OP-005: ::= , ->> , <-> , the backtick, and the backslash are
                  reserved operator tokens. The lexer recognizes them and
                  the parser rejects them, so they cannot be silently reused
                  by a user-defined operator before the language assigns
                  them meaning. The reserved-for-future glyph set is
```

reserved the same way outside literals and declared
operator symbols.
DECISION-OP-006: "~", "#" and "^" are reserved PREFIX spellings, refused in
operand position exactly as DECISION-OP-005 refuses ::= ,
->> and <-> . Complement and length/count are candidate
meanings; none is assigned yet. Every other role of each
spelling is unaffected. "@" is not a prefix operator at
all: it introduces a directive or annotation and marks the
->(@) address derivation. Address, pointer and reference
variables carry their kind in the TYPE derivation and are
read and assigned like ordinary variables, so no use site
ever restates the kind. See the note on prefix-operator.
DECISION-LIT-005: FoLang boolean literals are co.const.true and
co.const.false and the null literal is co.const.none.
true and false are NOT reserved words and NOT literals.
Inside an annotation argument, a bare true, false, or True
is an ordinary annotation-value name.
REVISION 6 CLARIFICATIONS

DECISION-LEX-001 is restored without a leading-underscore extension:
ordinary identifiers must begin with an ASCII letter.
_x, _1, a__b, and a_ are not identifiers. A lone _ remains
available only through grammar productions that explicitly
admit the contextual underscore token.
DECISION-DIR-001 records that built-in directives are self-delimiting and
are therefore exceptions to simple-statement semicolon
termination.

REVISION 7 LEXICAL DISAMBIGUATION

Revision 7 recorded four scanner interactions inherited from the initial
C++-compatible literal draft. Later revisions simplified the literal set:
DECISION-LEX-005 was withdrawn in revision 9 and DECISION-LEX-007 in
revision 11. The active scanner constraints are DECISION-LEX-006 and
DECISION-LEX-008 together with ordinary maximal munch.

DECISION-LIT-006: A FoLang floating literal carries at least one digit on
EACH side of the decimal point. The C++ abbreviated forms
1. and .10 are rejected; write 1.0 and 0.10. The rule
applies to hexadecimal floating literals too, so 0x1.8p3
is valid while 0x1.p3 and 0x.8p3 are not. The exponent
form without a point, such as 1e5, is unaffected.

This narrows the C++ literal set. Every accepted
FoLang floating literal remains C++ compatible, while the
raw-lexeme policy of DECISION-LIT-000 remains applicable.

The rule is what makes the scanner simple. Because a
numeric literal can no longer end at a point, plain
maximal munch already yields the right tokens with no
lookahead of any kind:

```
1 .. 10        ->  1  ..  10
3.to_str()     ->  3  .  to_str ( )
3.14.to_str()  ->  3.14  .  to_str ( )
1.5 .. 2.5     ->  1.5  ..  2.5
1e5 .. 2e5     ->  1e5  ..  2e5
1 .... 10      ->  unrecognized symbolic run
```

DECISION-LEX-005: WITHDRAWN in revision 9. It existed only to stop a
numeric literal from swallowing a point that belonged to
the range operator or to member access. DECISION-LIT-006
removes the abbreviated forms that made that possible, so
the special scanning rule is no longer needed and
DECISION-LEX-003 maximal munch applies unmodified.
DECISION-LEX-006: An identifier token must NOT be immediately followed by
"_". This turns a trailing or doubled underscore into a
lexical error instead of silently splitting a_ into the
identifier a plus the contextual token _ , and a__b into
a plus __b. Restated positively: an identifier starts
with an ASCII letter, may contain single underscores
between nonempty alphanumeric segments, never contains
consecutive underscores, and never ends in an underscore.
DECISION-LIT-007: FoLang does NOT adopt the C++14 digit separator. A
numeric literal contains digits only, so 1'000 and 0x1'a
are rejected and 1000 and 0x1a are written plainly. The
apostrophe therefore has exactly one meaning in FoLang
source: it delimits a character literal. This is a
narrowing of the C++ literal set. The numeric
raw-lexeme policy of DECISION-LIT-000 still applies, and
the choice is reversible because admitting
a separator later would accept strictly more programs.

DECISION-LEX-007: WITHDRAWN in revision 11. The base-aware separator

```
                  adjacency rule existed only to disambiguate the
                  apostrophe between a digit separator and a character
                  literal. DECISION-LIT-007 removes the separator, so the
                  apostrophe is unambiguous and the rule is unnecessary.
DECISION-LEX-008: Adjacent encoding prefixes, raw-string introducers, and
                  backslashes in quoted literals begin reserved post-alpha
                  spellings. The scanner consumes the complete spelling and
                  reports an unsupported feature. With intervening
                  whitespace, those names remain ordinary identifiers.
DECISION-LEX-009: A SPECIAL METHOD is one complete spelling from a closed set,
                  scanned whole and classified from a table the way a built-in
                  method name is. "@@" is not a prefix operator over an
                  arbitrary identifier: @@new and @@init are the members, and
                  any other "@@" name is a lexical error that names the
                  admissible set. Keeping the check in the scanner means the
                  parser's lifecycle-name production never enumerates them, so
                  the set is extended in one place. See special-method.

CONFORMANCE NOTE ON DECISION-SYN-001
-----------------------------------
Semicolons remain mandatory for productions that explicitly use
statement-end, including ordinary declarations and executable statements.
Built-in directives are a deliberate exception and are self-delimiting.
Block-bodied declarations end structurally at their closing brace.

Context-sensitive rules such as source-file kind, declaration legality,
type checking, visibility, operator ownership, capture, and definite
initialization remain semantic constraints and are documented separately.

REVISION 5 CHANGE LOG
---------------------
Constructs that revision 4 could not parse, now accepted:
  1  inner/nested functions in a block        local-function-declaration
  2  type-constructor bodies                  type-constructor-binding
  3  @co.dap.oops(A: {...})                   annotation-binder
  4  bare flag keys such as {typename}        annotation-map-entry
  5  "=" inside an annotation map             annotation-binder
  6  co.lang.int->(&, meta={type=out})        attribute tails on every
                                              derivation form
  7  co.lang.int->([,])                       array-dimension-content
  8  Transformer(F(_), G(_))                  generic-parameter,
                                              generic-arity-clause
  9  @co.dap.lazy over x = add(1, 2);         expression-statement
 10  add(a T)->(T) { ... } without "="        function-definition
 11  closure = (f int, x int) ==>> x * f;     closure-declaration
     curry = (f int)(v int) ==>> f * v;       curried closure form
 12  a bare block used as a statement         statement

Silent misparse removed:
     ~28 built-in kinds had no declaration production and were absorbed by
     variable-declaration. They are now parsed by general-kind-declaration.

Token inventory completed:
     ::= ->> <-> backtick backslash are reserved-operator; the reserved
     glyph set is reserved-future-operator.

Reference divergences corrected or labelled:
     booleans are co.const.true/false, co.const.none added, and true and
     false remain ordinary names rather than FoLang literals. Revision 6
     restores ASCII-letter-first identifiers and makes built-in directives
     self-delimiting without removing any planned grammar production.

Hygiene:
     resolved 10 formerly unreferenced productions. co-path,
     contextual-keyword, hard-reserved-word, octal-digit-sequence,
     result-binding, and white-space were wired into the grammar;
     declaration-prefix, external-variable-declaration, index-expression,
     and member-access-expression were deleted. The ambiguous
     word-type-specification was removed, and array-dimension plus
     type-or-value-argument were narrowed to drop subsumed alternatives.

REVISION 7 CHANGE LOG
---------------------
  - DECISION-LIT-004 withdrawn; production folang-user-defined-literal
    removed; literal now reduces to builtin-literal. A user-defined-type
    value is object-construction, already defined in section 11.
  - DECISION-LEX-005 added: "." is not absorbed into a numeric literal when
    another "." follows, so the range operator survives maximal munch.

REVISION 8 CHANGE LOG
---------------------
  - DECISION-LEX-005 extended: "." is also not absorbed when an
    identifier-start character follows, so 3.to_str() scans as 3 . to_str
    and the parser resolves the dot as member access. The trailing-dot
```

```
    float 1. is retained, so revision 8 did not yet narrow the C++
    literal set then in use. Revision 9 later removed that abbreviated
    form. The revision 7 residual note recommending (3).to_str() is
    withdrawn.

REVISION 14 CHANGE LOG
----------------------
  - Reconciled the grammar commentary and decision register with
    language-ref(36).md physical-nesting rules.
  - DECISION-SYN-008 records that independent named type/container
    declarations cannot be physically nested, while ordinary local functions
    and anonymous expression/type-expression forms are explicit exceptions.
    Existing productions already encoded this distinction, so no executable
    production was widened.
  - DECISION-SCOPE-001 records declaration-site lexical scope and block-local
    identity for ordinary local functions.
  - DECISION-SCOPE-002 records call-site lexical-context resolution for
    separately declared executable @co.dap.inner declarations. The annotation
    remains parsed by the general annotation production; scope validation is
    semantic.
  - DECISION-SYN-009 closes the annotated-function-primary loophole:
    arbitrary annotations do not legalize a loose package-level function.
    Only annotation-defined primary declaration kinds may use that envelope.
  - Corrected the stale revision-12 production and reachability counts in the
    grammar footer and regenerated validation from the actual revision-14
    production graph.

REVISION 13 CHANGE LOG
----------------------
  - DECISION-SEM-002 added: @co.ddap.use activates extension and instance
    methods alike. use-directive gains a closed use-field list with the keys
    from and methods, matching import-field, so a mistyped key is a parse
    error. There is no separate "extensions" key: an extension already
    declares what it extends through @co.dap.extension on the method itself,
    so the activation site has nothing to restate. Resolution order and the
    one-activation-per-receiver rule are semantic and recorded as a note.
  - DECISION-SEM-001 added: instance selection is by name, so no coherence
    check is required; instance placement is restricted to the typeclass's
    or the type's exact package as a convention. Both are semantic, so they
    are recorded as a note on instance-declaration. No production changes.
  - DECISION-TYP-004 mechanized: type-or-value-argument and array-dimension
    now take the new production dependent-index instead of
    constant-expression and expression. Arithmetic, calls and index
    expressions are rejected syntactically in both positions.
  - DECISION-TYP-005 added: the dependent-type equality rule, its three
    index comparison cases, and an explicit statement of what is out of
    scope and why.
  - DECISION-TYP-006 added: dependent types are checked, never inferred.
  - constant-expression is retained; it is still used for enum variant
    values, where ordinary constant arithmetic remains permitted.
  - DECISION-TYP-004 records that an index is non-negative, that a literal
    index is guaranteed by the grammar because no prefix operator is
    reachable, and that a named index must be verified by the checker after
    @co.dap.const substitution. The rule applies identically to array
    dimensions and dependent-type arguments, since both share
    dependent-index. Zero remains permitted.
  - Verified against language-ref.md: every dependent-type argument and
    array dimension in the document is a literal or a name, so no example
    changes.

REVISION 12 CHANGE LOG
----------------------
  - DECISION-SYN-007 added. Direct body forms and expression forms are now
    formally separated with zero-width contextual guards rather than only
    relying on ordered choice.
  - Named UDT/container bodies use body-close, which rejects an immediate
    semicolon after the body-closing brace.
  - A direct pattern block, function definition, function-kind inline body,
    named block, labeled block, or standalone block statement uses
    body-closure-guard.
  - Object construction, map literals, anonymous class expressions, and
    other braced expressions remain expressions; their enclosing simple
    statements still require semicolons.
  - Stale literal-decision wording was corrected after removal of numeric
    digit separators in revision 11.

REVISION 11 CHANGE LOG
----------------------
  - DECISION-LIT-007 added: the C++14 digit separator is not adopted. The
    production digit-separator is deleted and the five digit-sequence
    productions that referenced it are simplified to digits only.
  - DECISION-LEX-007 WITHDRAWN. With no separator, the apostrophe has a
    single meaning and needs no adjacency test.
  - Net effect across revisions 9 and 11: two of the four revision 7
```

```
       scanning rules are now gone. What remains is DECISION-LEX-006, the
       identifier trailing-underscore guard, and DECISION-LEX-008, the
       reserved-literal adjacency rule.

   REVISION 10 CHANGE LOG
   ----------------------
     - DECISION-SYN-006 added: the termination model is now stated once,
       covering the ";" and "}" hard ends, the "," soft separator, the
       self-delimiting directives and annotations, and the nuance that an
       expression-closing "}" does not terminate its statement.
     - function-object-declaration no longer demands ";" after a block body,
       which contradicted every sibling declaration. It gains
       function-object-binding, which also admits the reference form
       oObj co.lang.function = add; that revision 9 could not parse.
     - pattern-result gains a hard end: an expression-bodied function-pattern
       clause now takes ";", while a block-bodied clause still ends at "}".
       Previously such a clause had no terminator at all.
     - Audited declaration- and statement-level productions against the
       intended model. Revision 12 later encoded the remaining direct
       body-versus-expression priority explicitly.

   REVISION 9 CHANGE LOG
   ---------------------
     - DECISION-LIT-006 added: a floating literal needs a digit on each side
       of the point, so 1. and .10 are rejected in favour of 1.0 and 0.10.
       fractional-constant and hexadecimal-fractional-constant lose their
       abbreviated alternatives. This narrows the C++ literal set while
       preserving the numeric raw-lexeme policy of DECISION-LIT-000.
     - DECISION-LEX-005 WITHDRAWN. With the abbreviated forms gone, a numeric
       literal can never end at a point, so DECISION-LEX-003 maximal munch
       produces the correct token stream unaided. The scanner needs no
       numeric lookahead and the parser never re-lexes.
     - Net effect: one scanning rule removed, two productions simplified, and
       every ambiguity discussed in revisions 7 and 8 eliminated at source
       rather than worked around.
     - DECISION-LEX-006 added: an identifier may not be followed by "_", so a
       trailing or doubled underscore is an error rather than a token split.
     - DECISION-LEX-007 added: base-aware digit-separator adjacency rule.
     - DECISION-LEX-008 added: reserved-literal adjacency rule.
     - Production count corrected.

   Verified after this revision: 311 unique productions; no duplicate
   production; no undefined nonterminal reference; no unterminated production;
   balanced EBNF grouping delimiters; 289 productions reachable from
   compilation-unit; and 22 intentional lexical/token-summary or informative
   companion roots with no unclassified unreachable production.
*)

(* ====================================================================== *)
(* 1. Compilation units                                                   *)
(* ====================================================================== *)

compilation-unit = package-source-file
                 | application-entry-file
                 | library-surface-file ;

package-source-file = file-preamble, primary-declaration ;

application-entry-file = file-preamble, { entry-item } ;

library-surface-file = file-preamble, library-declaration ;

file-preamble = { file-directive } ;

file-directive = import-directive
               | alias-directive
               | use-directive
               | dynamic-runtime-directive
               | pragma-directive
               | generic-directive ;

entry-item = file-directive
           | entry-type-declaration
           | bare-function-pattern-clause
           | capturing-function-pattern-clause
           | statement ;

(*
   DECISION-SYN-008 / DECISION-SYN-009:
   This is the complete package-source primary-declaration entry point.
   Independent named type/container declarations are reachable here, not from
   statement or block-item. annotated-function-primary is reserved for
   annotation-defined primary declaration kinds; it does not permit an
   arbitrary annotated ordinary function at package-file scope.
```

```
*)
primary-declaration = struct-declaration
                    | cstruct-declaration
                    | enum-declaration
                    | union-declaration
                    | data-declaration
                    | class-declaration
                    | interface-declaration
                    | signature-declaration
                    | module-declaration
                    | unit-declaration
                    | type-declaration
                    | object-declaration
                    | instance-declaration
                    | matcher-instance-declaration
                    | function-object-declaration
                    | delegate-declaration
                    | named-block-declaration
                    | annotated-contract-declaration
                    | annotated-function-primary
                    | type-constructor-primary
                    | forward-type-declaration
                    | general-kind-declaration
                    | package-alias-declaration ;

(* ===================================================================== *)
(* 2. Directives, annotations, and metadata                              *)
(* ===================================================================== *)

annotations = { annotation } ;

one-or-more-annotations = annotation, { annotation } ;

(*
   DECISION-SCOPE-002:
   @co.dap.inner uses this ordinary annotation syntax. Whether it is legal on
   the annotated declaration, how an attachment is established, and how free
   runtime names resolve are semantic properties; the lexer/parser does not
   create a physically nested declaration node.
*)
annotation = "@", qualified-name,
             [ "(", [ annotation-argument-list ], ")" ] ;

annotation-argument-list = annotation-argument,
                           { ",", annotation-argument }, [ "," ] ;

(*
   DECISION-ANN-001: "=" and ":" are interchangeable binders.
   The optional group is written as a unit so a recursive-descent or PEG
   parser backtracks it cleanly: for a bare value such as co.lang.int the
   group tries "co" then finds no binder, abandons the group as a whole, and
   the value is matched by annotation-value.
*)
annotation-argument = [ annotation-key, annotation-binder ], annotation-value ;

annotation-binder = "=" | ":" ;

annotation-key = identifier, { "-", identifier } ;

annotation-value = literal
                 | type-expression
                 | qualified-name
                 | declaration-reference
                 | annotation-list
                 | annotation-map
                 | annotation-arrow-pair ;

annotation-list = "[", [ annotation-value,
                         { ",", annotation-value }, [ "," ] ], "]" ;

annotation-map = "{", [ annotation-map-entry,
                        { ",", annotation-map-entry }, [ "," ] ], "}" ;

(*
   DECISION-COL-001 / DECISION-ANN-001:
   A map entry uses either binder, and a bare key is a flag whose value is the
   boolean true. This admits {T:{typename}} and {variance:invariant,
   bound=Number} and {type=out}.
*)
annotation-map-entry = annotation-key, annotation-binder, annotation-value
                     | annotation-key ;

annotation-arrow-pair = string-literal, "=>", string-literal ;
```

```
(* DECISION-DIR-001: standalone directives are self-delimiting. *)
import-directive = "@co.ddap.import", "(", import-field,
                  { ",", import-field }, [ "," ], ")" ;

import-field = ( "package" | "library" | "src-library" | "expect"
               | "as" | "realm" | "parent-realm" ), "=", annotation-value ;

(* The alias target must be a co.* path, so co-path is used directly. *)
alias-directive = "@co.ddap.alias", "(", co-path, ",",
                  "as", "=", string-literal, [ "," ], ")" ;

(*
   DECISION-SEM-002, activation. A closed field list, matching import-field, so
   a mistyped key such as "method" is a parse error rather than a silently
   ignored argument.

   "from" names a DECLARATION, unlike "package" which names a package only:

       from="stringextension"           bare, resolved in the current package
       from="tc.ListFunctor"            alias + instance
       from="ext.stringextension"       alias + unit
       from="abc.tc.ListFunctor"        full package + instance
       from="abc.ext.stringextension"   full package + unit

   "methods" is the only list attribute. It activates named functions whatever
   "from" resolves to, an extension unit or a typeclass instance. An extension
   already declares what it extends through @co.dap.extension on the method
   itself, so the activation site has nothing to restate. The list may be
   omitted to activate everything the source provides.

   These are semantic constraints; this production accepts any combination.
*)
use-directive = "@co.ddap.use", "(", use-field, { ",", use-field },
                [ "," ], ")" ;

use-field = ( "from" | "methods" ), "=", annotation-value ;

dynamic-runtime-directive = "@co.ddap.dynamicruntime",
                            [ "(", [ annotation-argument-list ], ")" ] ;

pragma-directive = ( "@co.pdap.compiler" | "@co.pdap.scale" ),
                   [ "(", [ annotation-argument-list ], ")" ] ;

generic-directive = "@co.ddap.", identifier,
                    [ "(", [ annotation-argument-list ], ")" ] ;

(* ====================================================================== *)
(* 3. Names and references                                                *)
(* ====================================================================== *)

declaration-name = identifier | "_" ;

qualified-name = ( identifier | "co" ), { ".", identifier } ;

co-path = "co", ".", identifier, { ".", identifier } ;

declaration-reference = qualified-function-reference | qualified-name ;

qualified-function-reference = qualified-name, "(", [ type-list ], ")",
                               return-type-clause ;

(*
   DECISION-LEX-009: a special method is a CLOSED set of complete spellings, not the
   "@@" prefix applied to an arbitrary identifier. The scanner resolves the whole
   spelling against that set and emits one token, exactly as it resolves a built-in
   method name from its table, so "@@" never reaches the parser as a prefix and a name
   outside the set is a lexical error naming the admissible ones.

   The set is listed under "Special methods" in language-ref.md. Adding a special
   method means adding its spelling here and to the scanner's table; nothing in the
   parser enumerates them.
*)
lifecycle-name = special-method ;

special-method = "@@new" | "@@init" ;

special-binding = result-binding | self-binding ;

(* $ alone is the self-referential let binding. *)
self-binding = "$" ;

(* $1, $2, ... capture the previous result in a =>> delegation chain. *)
result-binding = "$", digit, { digit } ;
```

```
wildcard = "_" ;

(* ========================================================================= *)
(* 4. Type syntax                                                            *)
(* ========================================================================= *)

type-expression = forall-type | union-type-expression ;

(*
   DECISION-SYN-008:
   forall-type is an anonymous polymorphic TYPE EXPRESSION. It is not a
   declaration prefix and creates no package-owned declaration identity.
*)
forall-type = "forall", "(", type-parameter-list, ")", ".", type-expression ;

type-parameter-list = identifier, { ",", identifier } ;

union-type-expression = arrow-type-expression,
                        { "|", arrow-type-expression } ;

arrow-type-expression = type-postfix-expression,
                        [ "->", arrow-type-tail ]
                      | "(", [ function-type-parameter,
                               { ",", function-type-parameter } ],
                        ")", "->", arrow-type-tail ;

arrow-type-tail = type-derivation
                | parenthesized-type-list
                | type-expression ;

type-postfix-expression = type-atom, { type-argument-list } ;

(* A parenthesized type atom groups exactly one type. `(A, B)` remains a valid
   tuple/grouping expression in section 11, but it is not silently reclassified
   as one type atom. A comma in a function-type head must introduce another
   parameter; `(A,)->(B)` is therefore invalid. *)
type-atom = qualified-name
          | "(", type-expression, ")" ;

type-argument-list = "(", [ type-or-value-argument,
                            { ",", type-or-value-argument } ], ")" ;

(*
   DECISION-TYP-004:
   A dependent-type argument is an INDEX, not a general expression. It admits
   an integer literal or a name only. Arithmetic, calls, indexing and every
   other operator are rejected here. That restriction is what keeps
   dependent-type equality decidable by inspection instead of by a solver.

   A name used as an index must resolve either to a type or value parameter
   in scope, or to a @co.dap.const compile-time constant. @co.dap.final marks
   an immutable binding and is NOT sufficient, because an immutable value need
   not be known at compile time while an index must be.

       @co.dap.const SIZE co.lang.int = 1024;
       buf Vector(SIZE);        legal, SIZE substitutes to 1024
       @co.dap.final n co.lang.int = readInput();
       bad Vector(n);           rejected, immutable but not constant

   Because the restriction is syntactic, the diagnostic can name the offending
   operator: "arithmetic is not permitted in a dependent index".

   An index is non-negative. A literal index cannot be negative because no
   prefix operator is reachable here, so -1 is a parse error. A named index
   must be verified by the checker after substitution:

       @co.dap.const OFFSET co.lang.int = -1;
       buf co.lang.int->([OFFSET]);   compile error, resolves to -1
       v Vector(OFFSET);              compile error, resolves to -1

   Zero is permitted, as in the zero-length array co.lang.int->([0]).
*)
type-or-value-argument = type-expression | dependent-index ;

dependent-index = integer-literal | qualified-name ;

type-list = type-expression, { ",", type-expression } ;

parenthesized-type-list = "(", [ type-list ], ")" ;

type-derivation = "(", derivation-specification, ")" ;

(*
   DECISION-TYP-001:
```

```
   Every derivation form may carry a trailing attribute list. The bare
   derivation-attribute-list alternative covers the repr/sign/region word and
   address forms, so the former word-type-specification production, which was
   textually identical to it, has been removed.
*)
derivation-specification = pointer-specification
                         | array-specification
                         | reference-specification
                         | range-type-specification
                         | slice-type-specification
                         | thunk-type-specification
                         | address-type-specification
                         | derivation-attribute-list ;

pointer-specification = pointer-stars,
                        [ ",", derivation-attribute-list ] ;

pointer-stars = ? one contiguous symbolic run consisting only of one or
                  more "*" characters; its length is the pointer degree ? ;

reference-specification = ( "&" | "&&" | "~" ),
                          [ ",", derivation-attribute-list ] ;

address-type-specification = "@", [ ",", derivation-attribute-list ] ;

thunk-type-specification = "^", [ ",", derivation-attribute-list ] ;

slice-type-specification = "[:]", [ ",", derivation-attribute-list ] ;

range-type-specification = "..", [ ",", derivation-attribute-list ] ;

array-specification = array-dimension-group, { array-dimension-group },
                      [ ",", derivation-attribute-list ] ;

array-dimension-group = "[", array-dimension-content, "]" ;

(* DECISION-TYP-003: any dimension may be elided, including the first. *)
array-dimension-content = [ array-dimension ], { ",", [ array-dimension ] } ;

(* integer-literal and identifier are already reachable through expression. *)
(*
   DECISION-TYP-004 applies to array dimensions too. The array derivation is
   the representation underlying a dependent type, as in

       Vector(n co.lang.int)->(co.lang.dependentType) = co.lang.int->([n]);

   so admitting arithmetic here would reintroduce it behind the dependent
   type. Every array dimension in language-ref.md is a literal or a name.
*)
array-dimension = "..." | "." | dependent-index ;

derivation-attribute-list = derivation-attribute,
                            { ",", derivation-attribute } ;

derivation-attribute = annotation-key, "=", annotation-value ;

return-type-clause = "->", "(", [ return-item-list ], ")" ;

return-item-list = return-item, { ",", return-item } ;

return-item = [ identifier ], type-expression ;

(* ========================================================================= *)
(* 5. Common declaration components                                          *)
(* ========================================================================= *)

(*
   DECISION-GEN-001:
   A generic parameter may declare arity, giving higher-kinded parameters such
   as Transformer(F(_), G(_)).
*)
generic-parameter-clause = "(", generic-parameter,
                           { ",", generic-parameter }, ")" ;

generic-parameter = identifier, [ generic-arity-clause ] ;

generic-arity-clause = "(", generic-arity-slot,
                       { ",", generic-arity-slot }, ")" ;

generic-arity-slot = "_" | identifier ;

kind-options = "->", "(", [ annotation-argument-list ], ")" ;

field-declaration = annotations, identifier, type-expression,
```

```
                    [ "=", expression ], statement-end ;

embedded-field-declaration = annotations, type-expression, statement-end ;

value-specification = annotations, identifier, type-expression, statement-end ;

(* DECISION-SYN-002: comma-separated variable declarations are one statement. *)
variable-declaration = annotations, typed-variable-declarator,
                       { ",", typed-variable-declarator }, statement-end ;

typed-variable-declarator = identifier, type-expression,
                            [ "=", expression ] ;

inferred-variable-declaration = annotations, inferred-variable-declarator,
                                { ",", inferred-variable-declarator },
                                statement-end ;

inferred-variable-declarator = identifier, definition-operator, expression ;

definition-operator = ( ":=" | "?=" ),
                      multi-symbol-infix-operator-boundary-guard ;

(* ================================================================== *)
(* 6. Data and type declarations                                      *)
(* ================================================================== *)

struct-declaration = annotations, declaration-name,
                     [ generic-parameter-clause ], "co.lang.struct", "=",
                     struct-body ;

struct-body = "{", { struct-member }, body-close ;

(*
   language-ref.md, "Struct Rules": structs cannot have default values to
   fields/members — the struct declaration remains pure data, and all behaviour
   lives in the companion unit. A cstruct is a further-restricted C-compatible
   data representation, so the same restriction applies there. Both therefore
   use pure-field-declaration, which is field-declaration WITHOUT the
   initializer option; the initializer remains available to the other
   containers that reference field-declaration directly.
*)
pure-field-declaration = annotations, identifier, type-expression,
                         statement-end ;

struct-member = pure-field-declaration | embedded-field-declaration ;

cstruct-declaration = annotations, declaration-name,
                      [ generic-parameter-clause ], "co.lang.cstruct", "=",
                      cstruct-body ;

cstruct-body = "{", { pure-field-declaration }, body-close ;

enum-declaration = annotations, declaration-name,
                   [ generic-parameter-clause ], "co.lang.enum", "=",
                   enum-body ;

enum-body = "{", [ enum-variant,
                   { enum-separator, enum-variant }, [ enum-separator ] ],
            body-close ;

(*
   DECISION-COL-001: enum variants are comma-separated. The comma is a soft
   boundary between variants; the closing brace is the hard structural end of
   the enum body. A trailing comma is permitted.
*)
enum-separator = "," ;

enum-variant = annotations, identifier,
               [ "(", [ type-list ], ")" ],
               [ "=", constant-expression ] ;

union-declaration = annotations, declaration-name,
                    [ generic-parameter-clause ], "co.lang.union", "=",
                    union-body ;

union-body = "{", { field-declaration }, body-close ;

data-declaration = annotations, declaration-name,
                   [ generic-parameter-clause ], "co.lang.data", "=",
                   data-variant, { "|", data-variant }, statement-end ;

data-variant = qualified-name,
               [ "(", [ type-list ], ")" ] ;
```

```
type-declaration = annotations, declaration-name,
                   [ generic-parameter-clause ], type-declaration-kind,
                   [ kind-options ], [ "=", type-expression ], statement-end ;

type-declaration-kind = "co.lang.type"
                      | "co.lang.typealias"
                      | "co.lang.newtype"
                      | "co.lang.opaquetype"
                      | "co.lang.subtype"
                      | "co.lang.supertype"
                      | "co.lang.associatedtype"
                      | "co.lang.refinementType"
                      | "co.lang.dependentType"
                      | "co.lang.typetype"
                      | "co.lang.typekind" ;

(* Entry declarations deliberately omit generic-parameter-clause. *)
entry-type-declaration = annotations, declaration-name,
                         ( "co.lang.type"
                         | "co.lang.typealias"
                         | "co.lang.newtype"
                         | "co.lang.opaquetype"
                         | "co.lang.subtype"
                         | "co.lang.supertype"
                         | "co.lang.dependentType" ),
                         [ kind-options ], [ "=", type-expression ],
                         statement-end ;

forward-type-declaration = annotations, declaration-name,
                           [ generic-parameter-clause ],
                           forward-declarable-kind, [ kind-options ],
                           statement-end ;

forward-declarable-kind = "co.lang.struct"
                        | "co.lang.cstruct"
                        | "co.lang.class"
                        | "co.lang.interface"
                        | "co.lang.signature"
                        | "co.lang.module"
                        | "co.lang.enum"
                        | "co.lang.union"
                        | "co.lang.data"
                        | "co.lang.object"
                        | "co.lang.instance"
                        | "co.lang.function" ;

package-alias-declaration = declaration-name, "co.lang.package", statement-end ;

(*
   DECISION-KIND-001:
   A built-in kind with no dedicated production is parsed here rather than
   falling through to variable-declaration. Ordered choice matters: a specific
   declaration in section 6 or 7 is tried first, and in statement position
   variable-declaration is preferred, so an ordinary declarator is unaffected.
*)
general-kind-declaration = annotations, declaration-name,
                           [ generic-parameter-clause ],
                           general-declarable-kind, [ kind-options ],
                           general-kind-binding ;

general-kind-binding = "=", general-kind-block
                     | "=", type-expression, statement-end
                     | "=", non-block-expression, statement-end
                     | statement-end ;

general-kind-block = "{", { general-kind-member }, body-close ;

general-kind-member = field-declaration
                    | embedded-field-declaration
                    | signature-type-component
                    | function-declaration
                    | function-specification ;

general-declarable-kind = "co.lang.realm"
                        | "co.lang.loader"
                        | "co.lang.role"
                        | "co.lang.record"
                        | "co.lang.property"
                        | "co.lang.indexer"
                        | "co.lang.trait"
                        | "co.lang.mixin"
                        | "co.lang.extension"
                        | "co.lang.typeclass"
                        | "co.lang.concept"
```

```
                       | "co.lang.macro"
                       | "co.lang.template"
                       | "co.lang.lambda"
                       | "co.lang.behavior"
                       | "co.lang.method"
                       | "co.lang.namespace"
                       | "co.lang.stex"
                       | "co.lang.kind"
                       | "co.lang.level"
                       | "co.lang.order"
                       | "co.lang.rank"
                       | "co.lang.hokrlt"
                       | "co.lang.alias" ;

(* ================================================================== *)
(* 7. Containers and behavioral declarations                          *)
(* ================================================================== *)

unit-declaration = annotations, declaration-name,
                   [ generic-parameter-clause ], "co.lang.unit", "=",
                   unit-body ;

unit-body = "{", { function-declaration }, body-close ;

class-declaration = annotations, declaration-name,
                    [ generic-parameter-clause ], "co.lang.class",
                    [ kind-options ], "=", class-body ;

class-body = "{", { class-member }, body-close ;

class-member = field-declaration
             | function-declaration
             | lifecycle-method-declaration ;

lifecycle-method-declaration = annotations, lifecycle-name,
                               parameter-list, [ return-type-clause ],
                               function-definition ;

interface-declaration = annotations, declaration-name,
                        [ generic-parameter-clause ], "co.lang.interface", "=",
                        interface-body ;

interface-body = "{", { function-specification }, body-close ;

signature-declaration = annotations, declaration-name,
                        [ generic-parameter-clause ], "co.lang.signature", "=",
                        signature-body ;

signature-body = "{", { signature-member }, body-close ;

signature-member = value-specification
                 | function-specification
                 | signature-type-component ;

signature-type-component = annotations, declaration-name,
                           [ generic-parameter-clause ], "co.lang.type",
                           [ "=", type-expression ], statement-end ;

module-declaration = annotations, declaration-name,
                     [ generic-parameter-clause ], "co.lang.module",
                     [ kind-options ], "=", module-body ;

module-body = "{", { module-member }, body-close ;

module-member = variable-declaration
              | inferred-variable-declaration
              | function-declaration
              | signature-type-component ;

library-declaration = annotations, declaration-name, "co.lang.library", "=",
                      library-body ;

library-body = "{", { library-member }, body-close ;

library-member = import-directive
               | struct-declaration
               | cstruct-declaration
               | function-declaration ;

object-declaration = annotations, declaration-name,
                     [ generic-parameter-clause ], "co.lang.object",
                     [ kind-options ], "=", object-body ;

object-body = "{", { field-declaration | function-declaration }, body-close ;
```

```
(*
   DECISION-SEM-001, instance coherence. Recorded here because the grammar
   cannot express it and an implementer must not infer its absence.

   An instance is declared in the package that defines the typeclass, or the
   package that defines the type. That exact package: sub-packages are distinct
   packages and do not qualify. A typeclass may live in any package, so the
   rule is structural rather than tied to co.*.

       abc.tc.ListFunctor      for=Functor, type=List           legal
       myapp.ab.TreeFunctor    for=Functor, type=myapp.ab.Tree  legal
       other.util.ListFunctor  for=Functor, type=List           MISPLACED

   This is a PLACEMENT convention, not a correctness requirement. FoLang
   selects an instance BY NAME and never searches visible packages for one that
   happens to match a type, so two instances for the same typeclass and type
   pair are not ambiguous; a call names the one it means. No coherence check is
   required, and unique package names or aliases fully disambiguate.

   Method syntax such as xs.map(f) is unrelated. It calls an associated
   function in a companion unit, which lives in the type's own package and is
   therefore unambiguous by construction.

   A misplaced instance PARSES correctly; this production accepts it. The check
   belongs to name resolution, where the diagnostic can name the typeclass, the
   type, and the two packages in which the instance would have been legal.
*)
instance-declaration = annotations, declaration-name,
                       [ generic-parameter-clause ], "co.lang.instance",
                       [ kind-options ], "=", instance-body ;

instance-body = "{", { function-declaration | variable-declaration }, body-close ;

matcher-instance-declaration = annotations, declaration-name,
                               [ generic-parameter-clause ],
                               ( "co.lang.Matcher" | "co.lang.matcher" ),
                               [ kind-options ], "=", instance-body ;

annotated-contract-declaration = one-or-more-annotations, declaration-name,
                                 [ generic-parameter-clause ], "=",
                                 contract-body ;

contract-body = "{", { function-specification | value-specification }, body-close ;

named-block-declaration = annotations, declaration-name,
                          [ generic-parameter-clause ], "co.lang.block", "=",
                          block, body-closure-guard ;

delegate-declaration = annotations, declaration-name,
                       [ generic-parameter-clause ], "co.lang.delegate", "=",
                       function-type, statement-end ;

(*
   DECISION-SYN-006:
   An inline function body is a block and ends at "}" with no semicolon; a
   binding to an existing callable is an expression and ends at ";". Both
   forms appear in the reference:

       someFArg co.lang.function = (a co.lang.int)->(co.lang.int) = {
           this.return a * 2;
       }
       oObj co.lang.function = add;
*)
function-object-declaration = annotations, declaration-name,
                              [ generic-parameter-clause ],
                              "co.lang.function", "=",
                              function-object-binding ;

(*
   DECISION-SYN-007:
   A direct anonymous function is the inline body of this function-kind
   declaration and ends at its closing brace. Any other expression binding,
   including object construction or a callable reference, ends with ";".
*)
function-object-binding = anonymous-function-expression,
                          body-closure-guard
                        | non-anonymous-function-expression,
                          statement-end ;

(*
   DECISION-SYN-009:
   Parsing this envelope does not by itself make the function legal. Semantic
   analysis must confirm that at least one resolved annotation defines a legal
```

```
   primary declaration kind. Otherwise the source is a forbidden loose
   package-level function.
*)
annotated-function-primary = one-or-more-annotations, function-declaration ;

(*
   DECISION-TYP-002:
   A type constructor returns exactly one type-producing result, so it does not
   use the ordinary return-item-list (which permits zero or multiple results).
   A `|` combines type-producing result kinds into that one union result; a
   comma cannot introduce another result. Its body binds a type-expression. The
   type-expression alternative precedes the expression alternative, so
   Vector(n co.lang.int)->(co.lang.dependentType) = co.lang.int->([n]);
   is parsed as a type, not as an unparseable expression.

   The algebraic form Option(T) co.lang.data = Some(T) | None(); is handled by
   data-declaration. Its `|` similarly belongs to one union type binding rather
   than spelling multiple function returns.
*)
type-constructor-primary = annotations, function-name, parameter-list,
                           { parameter-list }, type-constructor-return-clause,
                           type-constructor-binding ;

type-constructor-return-clause = "->", "(", type-constructor-result-kind,
                                 { "|", type-constructor-result-kind }, ")" ;

type-constructor-result-kind = "co.lang.dependentType"
                             | "co.lang.type"
                             | "co.lang.typetype"
                             | "co.lang.typekind"
                             | "co.lang.kind" ;

type-constructor-binding = function-definition
                         | function-delegation
                         | "=", type-expression, statement-end
                         | "=", non-block-expression, statement-end
                         | statement-end ;

(* ================================================================== *)
(* 8. Functions                                                       *)
(* ================================================================== *)

function-declaration = annotations, [ receiver-clause ], function-name,
                       parameter-list, { parameter-list },
                       [ return-type-clause ], function-binding ;

(* Lifecycle names have their own class-member production and are not ordinary
   function names in a unit, module, local block, or package declaration. *)
function-name = identifier ;

receiver-clause = "(", ( type-expression
                       | identifier, type-expression ), ")" ;

parameter-list = "(", [ parameter,
                        { ",", parameter }, [ "," ] ], ")" ;

parameter = [ "..." ], [ "~" ], identifier, [ "?" ],
            [ type-expression ], [ "=", expression ] ;

function-binding = function-definition
                 | function-delegation
                 | function-alias-binding
                 | statement-end ;

(* DECISION-FUN-001: the "=" before a block body is optional. *)
function-definition = [ "=" ], block, body-closure-guard ;

function-delegation = ( "=>" | "=>>" ), expression,
                      { "=>>", expression }, statement-end ;

function-alias-binding = "=", non-block-expression, statement-end ;

function-specification = annotations, [ receiver-clause ], function-name,
                         parameter-list, { parameter-list },
                         [ return-type-clause ], statement-end ;

function-type = "(", [ function-type-parameter,
                       { ",", function-type-parameter } ], ")",
                return-type-clause ;

(* Reference examples permit both `(co.lang.int)` and `(value co.lang.int)`. *)
function-type-parameter = type-expression
                        | identifier, type-expression ;
```

```
anonymous-function-expression = [ "forall", "(", type-parameter-list, ")", "." ],
                                parameter-list, return-type-clause,
                                [ "=" ], block ;

lambda-expression = "|", [ lambda-parameter,
                           { ",", lambda-parameter } ], "|", "=>",
                    ( expression | block ) ;

lambda-parameter = identifier, [ type-expression ] ;

(*
   DECISION-FUN-002:
   The equals sign makes this a named closure declaration. It is followed by
   one or more parameter lists and the four-character `==>>` marker. One list
   is an ordinary closure and two or more lists make the closure curried. This
   is distinct from `=>`, which introduces lambdas and bare function-pattern
   clauses, and from `=>>`, which performs function delegation/chaining.
*)
closure-declaration = annotations, identifier, "=", parameter-list,
                      { parameter-list }, "==>>", expression,
                      statement-end ;

(*
   DECISION-SYN-003 / DECISION-SYN-008 / DECISION-SCOPE-001:
   This is FoLang's sole named local-function syntax. It requires a return-type
   clause and a block body, which keeps foo(); an expression statement. It has
   block-local identity, is never a package member, and resolves free runtime
   names from its lexical declaration context. Source-form restrictions, such
   as the application entry profile, remain semantic checks.
*)
local-function-declaration = annotations, function-name, parameter-list,
                             { parameter-list }, return-type-clause,
                             function-definition ;

(* ====================================================================== *)
(* 9. Function-pattern groups and patterns                                *)
(* ====================================================================== *)

(*
   A function-pattern group is the maximal set of entry-file clauses with the
   same name. All clauses in one group have the same arity and use the same
   form: either every clause is bare or every clause begins with let.

   The bare form is non-capturing. The let form must capture at least one
   definitely initialized surrounding entry-file runtime binding. Capture,
   compatible result types, clause reachability, overlap, and exhaustiveness
   are semantic checks over the complete group.

   `=>` introduces a bare pattern clause. `==>>` instead introduces the body
   of a named closure declaration after its parameter lists, and `=>>` is the
   function-delegation operator from section 8.
*)

bare-function-pattern-clause = annotations, identifier, pattern-parameter-list,
                               [ where-clause ], "=>", pattern-result ;

capturing-function-pattern-clause = annotations, "let", identifier,
                                    pattern-parameter-list,
                                    [ where-clause ], "=", pattern-result ;

pattern-parameter-list = "(", [ pattern,
                                { ",", pattern } ], ")" ;

where-clause = ".where", "(", expression, ")" ;

(*
   A where guard is evaluated only after every parameter pattern has matched.
   Bindings introduced by those patterns are in scope in the guard and result.
   The guard is required semantically to have type co.lang.bool.
*)

(*
   DECISION-SYN-006: a block-bodied clause ends at "}"; an expression-bodied
   clause takes the semicolon.

       fib(0) => 0;
       fib(n) => fib(n-1) + fib(n-2);
       classify(n).where(n > 0) => { this.return "positive"; }
*)
pattern-result = block, body-closure-guard
               | non-block-expression, statement-end ;

(*
   Expression-bodied clauses end with `;`, including capturing let clauses.
```

```
   Block-bodied clauses end at `}` and do not take `;`. A newline is never a
   clause terminator.
*)

pattern = wildcard
        | literal-pattern
        | binding-pattern
        | constructor-pattern
        | record-pattern
        | tuple-pattern
        | qualified-name ;

literal-pattern = literal
                | ( "+" | "-" ),
                  ( integer-literal | floating-literal ) ;

binding-pattern = identifier ;

constructor-pattern = qualified-name, "(", [ pattern,
                                             { ",", pattern } ], ")" ;

(* Record-pattern fields have no trailing comma: every comma must introduce the
   following record-pattern-field before the closing brace. *)
record-pattern = qualified-name, "{", [ record-pattern-field,
                                        { ",", record-pattern-field } ], "}" ;

record-pattern-field = identifier, [ ":", pattern ] ;

tuple-pattern = "(", pattern, ",", pattern,
                { ",", pattern }, ")" ;

match-case = ".case", "(", match-case-body, ")" ;

match-case-body = pattern, [ ":", expression ], "=>",
                  ( expression | block ) ;

match-default = ".default", "(", ( expression | block ), ")" ;

(* ================================================================== *)
(* 10. Statements and blocks                                          *)
(* ================================================================== *)

(*
   DECISION-SYN-001:
   - Every simple statement whose production uses statement-end ends with ";".
   - Built-in directives are self-delimiting and are not simple statements.
   - A newline is whitespace and never terminates a statement.
   - A block statement and a block-bodied declaration do not take a trailing
     semicolon merely because their final token is "}".

   DECISION-BLK-001:
   A final expression without a semicolon is a block tail expression, not an
   expression statement. It supplies the block's value.
*)
block = "{", { block-item }, [ block-tail-expression ], "}" ;

block-item = statement ;

block-tail-expression = expression ;

statement = variable-declaration
          | inferred-variable-declaration
          | grouped-variable-declaration
          | let-value-declaration
          | local-function-declaration
          | closure-declaration
          | multiple-assignment-statement
          | return-statement
          | expression-statement
          | labeled-block
          | block-statement
          | empty-statement ;

(* Grouped declarators have no trailing comma: every comma must introduce the
   following typed-variable-declarator before the closing parenthesis. *)
grouped-variable-declaration = "(", typed-variable-declarator,
                               { ",", typed-variable-declarator }, ")",
                               statement-end ;

let-value-declaration = "let", identifier, [ type-expression ], "=",
                        expression, statement-end ;

(*
   DECISION-OP-003:
```

```
   := and ?= occur only in inferred-variable-declaration. They are not
   assignment-expression operators and cannot participate in a = b = c-style
   chain. ::= remains reserved for a future feature and is rejected.
*)

(* Multiple assignment is a statement because it has multiple destinations. *)
multiple-assignment-statement = assignment-target, ",", assignment-target,
                                { ",", assignment-target }, "=",
                                expression-list, statement-end ;

assignment-target = postfix-expression
                  | tuple-assignment-target ;

tuple-assignment-target = "(", assignment-target, ",", assignment-target,
                          { ",", assignment-target }, ")" ;

return-statement = ( "this" | "self" ), ".return",
                   [ expression-list ], statement-end ;

(* DECISION-SYN-004: an expression statement may carry annotations. *)
expression-statement = annotations, non-block-expression, statement-end ;

labeled-block = identifier, ":", block, body-closure-guard ;

empty-statement = ";" ;

expression-list = expression, { ",", expression } ;

statement-end = ";" ;

(*
   DECISION-SYN-007 — zero-width contextual guards:

   body-closure-guard rejects a semicolon when the enclosing production has
   selected a declaration/function/pattern/block BODY. It examines the next
   significant token after whitespace and comments and consumes no token.

   non-block-expression rejects an expression when its complete source span
   is admissible as an unparenthesized block. When the same span could be read
   both as a block and as another braced expression (for example `{}`), the
   direct body/block reading has priority. A grouped block, a block with a
   postfix suffix, or an operator expression containing a block is still an
   expression.

   non-anonymous-function-expression applies the same priority rule when the
   complete source span is admissible as an unparenthesized
   anonymous-function-expression.
*)
body-close = "}", body-closure-guard ;

body-closure-guard =
    ? zero-width condition: the next significant token is not ";", or there is no next token ? ;

block-statement = block, body-closure-guard ;

non-block-expression = expression, non-block-expression-guard ;

non-block-expression-guard =
    ? zero-width condition: the complete source span is not admissible as an
      unparenthesized block production; when both block and another braced
      expression are possible, the block reading has priority ? ;

non-anonymous-function-expression =
    expression, non-anonymous-function-expression-guard ;

non-anonymous-function-expression-guard =
    ? zero-width condition: the complete source span is not admissible as an
      unparenthesized anonymous-function-expression production; that direct
      body reading has priority over the general expression reading ? ;

(* ====================================================================== *)
(* 11. Expressions and built-in operator precedence                       *)
(* ====================================================================== *)

(*
   DECISION-OP-001 — built-in precedence, highest to lowest:

   100  postfix: calls, indexing, member access, postfix !          left
    90  exponentiation: **                                        right
    80  prefix: +, -, !  (~, #, ^ reserved; @ is not a prefix)      right
    70  multiplicative: *, /, %                                   left
    60  additive: +, -                                            left
    55  ranges: .., <.., ..<, <..<                                none
    50  relational: <, <=, >, >=                                  left
```

```
    45  equality: ==, !=                                          left
    40  bitwise AND: &                                            left
    38  bitwise XOR: ^                                            left
    36  bitwise OR: |                                             left
    30  logical AND: &&                                           left
    20  logical OR: ||                                            left
    10  assignment: =, +=, -=, *=, /=, %=, **=, &=, ^=, |=       right

   Operands are still evaluated according to FoLang's normative left-to-right
   and target-first evaluation rules. Associativity determines grouping, not
   the order in which operand subexpressions are evaluated.

   DECISION-EXT-001 / DECISION-OPBOOT-003:
   Before operator-dependent source is tokenized, the compiler parses the fixed
   local operator library with its dedicated lexer/parser, combines those custom
   registrations with the language-owned built-in and pre-declared registrations,
   and builds one immutable operator table. Imports contribute no operator
   metadata. Existing and custom implementations use mode=overload and add no
   precedence entries. Operator mode=override is rejected.
*)
expression = assignment-expression
           | extended-operator-expression ;

(* DECISION-OP-002: right recursion makes assignment right-associative. *)
assignment-expression = logical-or-expression,
                        [ runtime-assignment-operator,
                          assignment-expression ] ;

runtime-assignment-operator = "="
                            | compound-assignment-operator ;

compound-assignment-operator = ( "+=" | "-=" | "*=" | "/=" | "%="
                               | "**=" | "&=" | "^=" | "|=" ),
                               multi-symbol-infix-operator-boundary-guard ;

constant-expression = ( logical-or-expression
                      | extended-operator-expression ),
                      ? zero-width condition: no runtime-assignment-operator
                        occurs anywhere in this constant-expression subtree ? ;

logical-or-expression = logical-and-expression,
                        { logical-or-operator, logical-and-expression } ;

logical-or-operator = "||", multi-symbol-infix-operator-boundary-guard ;

logical-and-expression = bitwise-or-expression,
                         { logical-and-operator, bitwise-or-expression } ;

logical-and-operator = "&&", multi-symbol-infix-operator-boundary-guard ;

bitwise-or-expression = bitwise-xor-expression,
                        { "|", bitwise-xor-expression } ;

bitwise-xor-expression = bitwise-and-expression,
                         { "^", bitwise-and-expression } ;

bitwise-and-expression = equality-expression,
                         { "&", equality-expression } ;

equality-expression = relational-expression,
                      { equality-operator, relational-expression } ;

equality-operator = ( "==" | "!=" ),
                    multi-symbol-infix-operator-boundary-guard ;

relational-expression = range-expression,
                        { relational-operator, range-expression } ;

relational-operator = "<" | ">" | multi-symbol-relational-operator ;

multi-symbol-relational-operator = ( "<=" | ">=" ),
                                   multi-symbol-infix-operator-boundary-guard ;

(* A range expression contains at most one range operator. *)
range-expression = additive-expression,
                   [ range-operator, [ additive-expression ] ]
                 | range-operator, additive-expression ;

range-operator = ( ".." | "<.." | "..<" | "<..<" ),
                 multi-symbol-range-operator-boundary-guard ;

additive-expression = multiplicative-expression,
                      { additive-operator, multiplicative-expression } ;
```

```
additive-operator = "+" | "-" ;

multiplicative-expression = unary-expression,
                            { multiplicative-operator, unary-expression } ;

multiplicative-operator = "*" | "/" | "%" ;

unary-expression = { prefix-operator }, power-expression ;

(*
   DECISION-OP-004 is withdrawn: ++ and -- are absent from the built-in
   prefix/postfix grammar. An unregistered contiguous occurrence is handled by
   the general symbolic-run rule rather than by a special operator rule.

   DECISION-OP-006: "~", "#" and "^" are RESERVED prefix spellings. They are
   listed here so no user-defined operator can claim them, and the parser
   refuses them in operand position with a reserved-operator diagnostic, the
   same treatment DECISION-OP-005 gives ::= , ->> and <-> . Candidate meanings
   include complement and length/count; none is assigned yet. Each spelling
   keeps every other role it already has: "~" marks a named parameter and the
   ->(~) heap reference, "^" is bitwise xor and the ->(^) thunk derivation.

   "@" is NOT a prefix operator. It introduces a directive or an annotation,
   and it marks the ->(@) address derivation. A variable of an address,
   pointer or reference kind is read and assigned like any other variable —
   the kind is carried by the type derivation and never restated at the use
   site — so there is no address-of prefix:

       someInt co.lang.int = 10;
       someAdd co.lang.int->(@);
       someAdd = someInt;          no "@" at the use site
       co.out.println(someAdd);
*)
prefix-operator = "+" | "-" | "!"
                | reserved-prefix-operator ;

reserved-prefix-operator = "~" | "#" | "^" ;

(* Right recursion makes exponentiation right-associative. *)
power-expression = postfix-expression,
                   [ power-operator, unary-expression ] ;

power-operator = "**", multi-symbol-infix-operator-boundary-guard ;

postfix-expression = primary-expression,
                     { postfix-suffix | postfix-operator } ;

postfix-operator = "!" ;

postfix-suffix = call-suffix
               | index-suffix
               | member-suffix
               | match-suffix ;

call-suffix = "(", [ argument-list ], ")" ;

argument-list = argument, { ",", argument }, [ "," ] ;

argument = ( [ identifier, "=" ], expression )
         | block
         | lambda-expression
         | wildcard ;

(* Lambda and wildcard are contextual method-call arguments. A lambda must be
   a direct argument of a receiver-qualified
   map/filter/reduce/forEach/sortBy/groupBy call. Wildcard is admitted only in
   the first iterator-index slot of a receiver-qualified each call. Transparent
   grouping around the member callee does not change either rule. contains and
   containsVal require an actual comparison value and reject wildcard. *)

index-suffix = "[", [ expression-list ], "]" ;

(* "for" is hard-reserved for comprehensions but remains contextual after a
   member dot because the built-in method table includes `.for`. *)
member-suffix = ".", ( identifier | "for" | lifecycle-name ) ;

(*
   Member access and indexing are already expressed by postfix-expression with
   member-suffix and index-suffix; the former duplicate productions were
   removed in revision 5.
*)

primary-expression = literal
                   | special-binding
```

```
                    | "this"
                    | "self"
                    | qualified-name
                    | grouped-expression
                    | tuple-expression
                    | array-literal
                    | map-literal
                    | object-construction
                    | anonymous-class-expression
                    | block
                    | anonymous-function-expression
                    | let-expression
                    | comprehension-expression ;

grouped-expression = "(", expression, ")" ;

tuple-expression = "(", expression, ",", expression,
                   { ",", expression }, ")" ;

array-literal = "[", [ expression,
                       { ",", expression }, [ "," ] ], "]" ;

map-literal = "{", [ map-entry,
                     { ",", map-entry }, [ "," ] ], "}" ;

map-entry = expression, ":", expression ;

(* DECISION-COL-001: object fields use colon and comma. *)
object-construction = type-postfix-expression, "{",
                      [ object-field-initializer,
                        { ",", object-field-initializer }, [ "," ] ], "}" ;

object-field-initializer = identifier, ":", expression ;

(*
   DECISION-SYN-008:
   The forms below are expressions, not independent named declarations.
   Their syntactic containment does not create a package-level nested identity.
*)
anonymous-class-expression = "co.lang.class", "{",
                             { class-member }, "}" ;

let-expression = "let", "(", "{", let-binding,
                 { ",", let-binding }, "}", ")",
                 ".in", "(", "{", expression, "}", ")" ;

let-binding = ( identifier | special-binding ), "=", expression ;

comprehension-expression = "for", "(", comprehension-binding, ")",
                           ".yield", "(", expression-list, ")" ;

comprehension-binding = pattern, "<-", expression ;

(*
   At least one case is required semantically. The grammar accepts zero so a
   malformed chain produces a semantic diagnostic rather than a parse error.
*)
match-suffix = ".match", [ "(", [ expression ], ")" ],
               { match-case }, [ match-default ] ;

multi-symbol-infix-operator-boundary-guard =
    ? zero-width condition: a multi-symbol infix operator has an explicit
      boundary on both operand-facing sides; a boundary is whitespace, a
      comment, or an applicable delimiter, checked before separators are
      discarded ? ;

multi-symbol-range-operator-boundary-guard =
    ? zero-width condition: a multi-symbol range operator has an explicit
      boundary on every side for which an operand is present ? ;

extended-operator-expression =
    ? expression containing a registered non-built-in operator, parsed by
      precedence climbing from its declared fixity, precedence, associativity,
      and arity; all built-in subexpressions obey the table above, and every
      registered multi-symbol operator satisfies the operand-facing boundary
      rule for its fixity ? ;

(* ================================================================== *)
(* 11a. Control-flow chain shapes (informative)                       *)
(* ================================================================== *)

(*
   FoLang has no imperative if, else, while, or foreach keyword. `for` is
   reserved for comprehension-expression; the associated-function control
```

```
   forms below are otherwise parsed as
   postfix-expression with member-suffix and call-suffix, where argument
   admits a block. The productions below are INFORMATIVE. They document the
   canonical shapes the semantic analyzer enforces; they are not a second
   parse path and must not be entered from expression.

   Enforcing these shapes in the parser instead would require unbounded
   lookahead to distinguish a control chain from any other method chain, so
   the check belongs to semantic analysis.
*)

informative-condition-chain =
    "(", expression, ")", ".do", "(", block, ")",
    { ".otherwise", "(", expression, ")", ".do", "(", block, ")" },
    [ ".otherwise", ".do", "(", block, ")" ] ;

informative-loop-chain =
    "(", expression, ")", ".loop", "(", block, ")",
    { ".otherwise", "(", expression, ")", ".loop", "(", block, ")" },
    [ ".otherwise", ".loop", "(", block, ")" ] ;

informative-mixed-chain =
    "(", expression, ")", informative-branch-verb, "(", block, ")",
    { ".otherwise", "(", expression, ")",
      informative-branch-verb, "(", block, ")" },
    [ ".otherwise", informative-branch-verb, "(", block, ")" ] ;

informative-branch-verb = ".do" | ".loop" ;

informative-ternary-chain =
    "(", expression, ")", ".return", "(", expression, ")",
    { ".otherwise", "(", expression, ")", ".return", "(", expression, ")" },
    ".otherwise", ".return", "(", expression, ")" ;

informative-each-chain =
    postfix-expression, ".each", "(", ( identifier | "_" ), ",", identifier,
    ")", ".do", "(", block, ")" ;

informative-contains-chain =
    postfix-expression, ".contains", "(", expression, ")",
    ".do", "(", block, ")",
    [ ".otherwise", ".do", "(", block, ")" ] ;

informative-pipeline-chain =
    postfix-expression,
    { ( ".filter" | ".map" | ".reduce" | ".forEach"
      | ".sortBy" | ".groupBy" | ".fold" ),
      "(", argument-list, ")" } ;

(* ====================================================================== *)
(* 12. Literals and lexical grammar                                       *)
(* ====================================================================== *)

(*
   DECISION-LEX-001:
   Source text is UTF-8. A U+FEFF byte-order mark is permitted only as the
   first code point and is otherwise an error.

   A FoLang identifier begins with an ASCII alphabetic character. Remaining
   characters are ASCII alphabetic characters, decimal digits, or isolated
   underscores. Consecutive underscores and a trailing underscore are lexical
   errors. The spelling "_" is a dedicated contextual token used for discard,
   wildcard, or filename-derived declaration names; it is never an identifier.

   DECISION-BACKEND-001:
   A resolved user-defined FoLang identifier is lowered to C++ by appending
   the suffix "_fo". The no-consecutive-underscore and no-trailing-underscore
   rules ensure this lowering never creates a C++-reserved double underscore.

   DECISION-LEX-002:
   Horizontal whitespace, line terminators, line comments, and non-nesting
   block comments are discarded between tokens. A line terminator has no
   statement-termination meaning.

   DECISION-LEX-003 / DECISION-LEX-010:
   After comments, literals, and closed scanner-known composite spellings are
   recognized, the lexer consumes each remaining complete maximal contiguous
   symbol run. The run is never split into
   shorter valid operators as a fallback. It is classified by grammar context
   as a fixed structural spelling, contextual metadata spelling, registered
   expression operator, or unrecognized symbolic token. Comment introducers are
   recognized before symbolic-run scanning. Parentheses and other delimiters
   terminate a run; therefore +(+a) contains two separate + tokens.
```

```
   A registered expression operator containing more than one symbol character
   requires an explicit boundary on every operand-facing side. Whitespace, a
   comment, or an applicable delimiter supplies that boundary. Structural forms
   such as T->(**) are exempt because -> and the pointer-star run are not parsed
   as expression operators in that context.

   DECISION-LIT-000:
   The lexer stores the COMPLETE original numeric, character, or string
   literal lexeme in the AST. A C++ backend may emit that raw lexeme unchanged.
   The productions below define the selected C++-compatible numeric, character,
   and string forms used by this revision. Backend-conditionally-supported
   suffixes are accepted only when the configured C++ compiler supports them.

   co.const.true, co.const.false, and co.const.none are FoLang-defined literal
   forms and use backend-defined lowering rather than raw C++ lexeme emission.

   This decision does not import the C++ nullptr literal or the C++
   user-defined-literal operator"" mechanism.

   DECISION-LIT-004 (WITHDRAWN in revision 7):
   FoLang has no user-defined literal token. A value of a user-defined type is
   written with object-construction, for example Employee{name: "Rao", id: 1},
   which is an expression in section 11 rather than a literal in section 12.

   DECISION-LEX-005 was withdrawn in revision 9. DECISION-LIT-006 removes
   the abbreviated floating forms that formerly required special dot scanning.
*)

literal = builtin-literal ;

builtin-literal = integer-literal
                | floating-literal
                | string-literal-sequence
                | character-literal
                | boolean-literal
                | none-literal ;

(*
   DECISION-LIT-006 — floating literal shape, normative:

   A floating literal carries at least one digit on each side of the point.
   The abbreviated C++ forms are rejected:

       1.0            valid         1.        rejected, write 1.0
       0.10           valid         .10       rejected, write 0.10
       0x1.8p3        valid         0x1.p3    rejected
       1e5            valid         0x.8p3    rejected

   Because a numeric literal can no longer terminate at a point, the point
   that follows a number always belongs to some other token. DECISION-LEX-003
   maximal munch therefore produces the correct token stream with no numeric
   lookahead, no parser re-lexing, and no special case:

       1 .. 10        ->  1  ..  10            range over integers
       1.0 .. 0.10    ->  1.0  ..  0.10        range over floats
       1.5 .. 2.5     ->  1.5  ..  2.5
       1e5 .. 2e5     ->  1e5  ..  2e5
       3.to_str()     ->  3  .  to_str ( )     member access on an integer
       3.14.to_str()  ->  3.14  .  to_str ( )  float, then member access
       1 ... 10       ->  1  ...  10           ... is not an infix operator,
                                               so the parser rejects it
       1 .... 10      ->  unrecognized symbolic run; no fallback splitting

   A diagnostic for the rejected literal forms should name the replacement,
   for example "floating literal needs a digit on both sides of the point;
   write 1.0". A diagnostic for the multi-dot range forms should suggest
   1.0 .. 0.10.

   The narrowing keeps every FoLang floating literal inside the C++ floating
   literal set, so DECISION-LIT-000 raw-lexeme passthrough is unaffected.
   Range operators still require the operand-facing boundaries defined by
   DECISION-LEX-010; numeric maximal munch does not waive that requirement.
*)

(* DECISION-LIT-001: C++-compatible built-in integer literal spelling. *)
integer-literal = ( binary-integer-literal
                  | octal-integer-literal
                  | decimal-integer-literal
                  | hexadecimal-integer-literal ),
                  [ integer-suffix ] ;

binary-integer-literal = ( "0b" | "0B" ), binary-digit-sequence ;

octal-integer-literal = "0", [ octal-digit-sequence ] ;
```

```
decimal-integer-literal = nonzero-digit, { decimal-digit } ;

hexadecimal-integer-literal = hexadecimal-prefix,
                              hexadecimal-digit-sequence ;

hexadecimal-prefix = "0x" | "0X" ;

binary-digit-sequence = binary-digit, { binary-digit } ;

octal-digit-sequence = octal-digit, { octal-digit } ;

decimal-digit-sequence = decimal-digit, { decimal-digit } ;

hexadecimal-digit-sequence = hexadecimal-digit, { hexadecimal-digit } ;

(*
   DECISION-LIT-007 — no digit separator, normative:

   FoLang numeric literals contain digits only. The C++14 digit separator is
   NOT adopted, so an apostrophe never occurs inside a numeric literal and
   always delimits a character literal.

       1000  0xFFFFFFFF  0b101100100110      valid
       1'000  0x1'a  0b1011'0010             rejected

   Consequences:
     - The apostrophe has exactly one meaning in FoLang source: it opens or
       closes a character literal. No base-aware adjacency test is needed and
       DECISION-LEX-007 is withdrawn.
     - A character literal admits any translation character except the
       apostrophe, the backslash, CR and LF, so a space, a semicolon and a
       comma are ordinary c-characters. A line break is written with an
       escape such as \n.
     - This is a NARROWING of the C++ literal set, so every FoLang numeric
       literal remains a valid C++ numeric literal and DECISION-LIT-000
       numeric raw-lexeme passthrough remains valid.
     - It is also REVERSIBLE. Admitting a separator later accepts strictly
       more programs, so no FoLang source written against this revision would
       break. Reinstating it would require restoring the withdrawn
       DECISION-LEX-007 adjacency rule.
*)

integer-suffix = unsigned-suffix,
                 [ long-suffix | long-long-suffix | size-suffix ]
               | long-suffix, [ unsigned-suffix ]
               | long-long-suffix, [ unsigned-suffix ]
               | size-suffix, [ unsigned-suffix ] ;

unsigned-suffix = "u" | "U" ;

long-suffix = "l" | "L" ;

long-long-suffix = "ll" | "LL" ;

size-suffix = "z" | "Z" ;

(* DECISION-LIT-002: C++-compatible decimal and hexadecimal floating forms. *)
floating-literal = decimal-floating-literal
                 | hexadecimal-floating-literal ;

decimal-floating-literal = fractional-constant,
                           [ exponent-part ],
                           [ floating-point-suffix ]
                         | decimal-digit-sequence,
                           exponent-part,
                           [ floating-point-suffix ] ;

(*
   DECISION-LIT-006: a floating literal carries at least one digit on EACH
   side of the point. 1. and .10 are rejected; write 1.0 and 0.10.
*)
fractional-constant = decimal-digit-sequence, ".",
                      decimal-digit-sequence ;

hexadecimal-floating-literal = hexadecimal-prefix,
                               ( hexadecimal-fractional-constant
                               | hexadecimal-digit-sequence ),
                               binary-exponent-part,
                               [ floating-point-suffix ] ;

hexadecimal-fractional-constant = hexadecimal-digit-sequence, ".",
                                  hexadecimal-digit-sequence ;
```

```
exponent-part = ( "e" | "E" ), [ sign ], decimal-digit-sequence ;

binary-exponent-part = ( "p" | "P" ), [ sign ], decimal-digit-sequence ;

sign = "+" | "-" ;

floating-point-suffix = "f" | "F" | "l" | "L"
                      | "f16" | "F16"
                      | "f32" | "F32"
                      | "f64" | "F64"
                      | "f128" | "F128"
                      | "bf16" | "BF16" ;

(* DECISION-LIT-003: alpha character and string literal subset. *)
character-literal = single-quote, alpha-basic-c-character, single-quote ;

alpha-basic-c-character =
    ? any translation character except apostrophe, backslash,
      carriage return, or line feed ? ;

string-literal-sequence = string-literal, { string-literal } ;

string-literal = double-quote, { alpha-basic-s-character }, double-quote ;

alpha-basic-s-character =
    ? any translation character except double quote, backslash,
      carriage return, or line feed ? ;

(*
   DECISION-LEX-008 — reserved post-alpha literal spellings, normative:

   An adjacent u8, u, U, or L prefix, an R raw-string introducer, or a
   backslash within a quoted literal starts a reserved post-alpha spelling.
   The scanner consumes the complete spelling and reports an unsupported
   feature. With intervening whitespace, u8, u, U, L, and R remain ordinary
   identifiers.
*)
double-quote = ? Unicode scalar value U+0022 ? ;

single-quote = ? Unicode scalar value U+0027 ? ;

backslash = ? Unicode scalar value U+005C ? ;

(*
   DECISION-LIT-005:
   FoLang booleans are co.const.true and co.const.false, and the null literal
   is co.const.none. These spellings also match qualified-name; the literal
   reading is selected. A bare true, false, or True inside an annotation
   argument is an ordinary annotation-value name, not a literal.
*)
boolean-literal = "co.const.true" | "co.const.false" ;

none-literal = "co.const.none" ;


(*
   The revision 6 production folang-user-defined-literal was removed in
   revision 7. FoLang has no user-defined literal token. A value of a
   user-defined type is written with object-construction in section 11, for
   example Employee{name: "Rao", id: 1} or Point{x: 0.0, y: 0.0}.
*)

(*
   DECISION-LEX-001 with DECISION-LEX-006:

   An ordinary identifier begins with an ASCII letter a-z or A-Z. An
   underscore may occur only between nonempty alphanumeric segments. An
   identifier therefore never begins with an underscore, never contains
   consecutive underscores, and never ends in an underscore.

   The shape of the production alone does not enforce the last two rules: a
   greedy scanner would match a_ as the identifier a followed by the
   contextual token _ , and a__b as a followed by __b, silently producing the
   wrong tokens instead of an error. DECISION-LEX-006 supplies the missing
   negative constraint, expressed here as the trailing guard: the character
   immediately after a complete identifier must not be "_".

       name, myVar2, v1_hr, a_b_c   ->  identifiers
       _x, _1                       ->  errors, leading underscore
       a_, a__b                     ->  errors, DECISION-LEX-006 guard
       _                            ->  never an identifier; it is the
                                        contextual token admitted only by
                                        productions that spell "_" directly
```

```
   Reserved-word rejection remains a token-class check applied after the
   character sequence is recognized, using hard-reserved-word.

   DECISION-BACKEND-001 appends _fo to a resolved user identifier. Because no
   identifier ends in an underscore or contains a doubled underscore, the
   lowered C++ name never contains "__" and never matches a C++ reserved
   identifier.
*)
identifier = identifier-head, { "_", identifier-segment },
             identifier-trailing-guard ;

identifier-trailing-guard =
    ? a zero-width assertion that the next character is not "_" ? ;

identifier-head = ascii-letter, { ascii-alphanumeric } ;

identifier-segment = ascii-alphanumeric, { ascii-alphanumeric } ;

ascii-alphanumeric = ascii-letter | decimal-digit ;

ascii-letter = "A" | "B" | "C" | "D" | "E" | "F" | "G" | "H"
             | "I" | "J" | "K" | "L" | "M" | "N" | "O" | "P"
             | "Q" | "R" | "S" | "T" | "U" | "V" | "W" | "X"
             | "Y" | "Z"
             | "a" | "b" | "c" | "d" | "e" | "f" | "g" | "h"
             | "i" | "j" | "k" | "l" | "m" | "n" | "o" | "p"
             | "q" | "r" | "s" | "t" | "u" | "v" | "w" | "x"
             | "y" | "z" ;

binary-digit = "0" | "1" ;

octal-digit = "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" ;

hexadecimal-digit = decimal-digit
                  | "a" | "b" | "c" | "d" | "e" | "f"
                  | "A" | "B" | "C" | "D" | "E" | "F" ;

digit = decimal-digit ;

decimal-digit = "0" | nonzero-digit ;

nonzero-digit = "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9" ;

(* DECISION-LIT-005 removed true and false from this set. *)
hard-reserved-word = "co" | "let" | "this" | "for" | "forall" | "fo" ;

contextual-keyword = "self" ;

(*
   DECISION-OP-005:
   These tokens appear in the reference operator table but have no assigned
   meaning. The lexer recognizes each as one token and the parser rejects it,
   so a user-defined operator cannot silently claim it before the language
   assigns it a meaning.
*)
reserved-operator = "::=" | "->>" | "<->" | "`" | backslash ;

reserved-future-operator = ? one character from the hard-reserved future
                             syntax set, explicitly excluding every
                             language-predeclared operator glyph; unavailable
                             for use, declaration, or overload ? ;

(*
   Informative token-class summary. The parser consumes tokens; separators are
   discarded between them.
*)
token = identifier
      | keyword-token
      | literal
      | delimiter-token
      | symbolic-token
      | reserved-future-operator ;

keyword-token = hard-reserved-word | contextual-keyword ;

delimiter-token = "(" | ")" | "{" | "}" | "[" | "]"
                | "," | ";" | double-quote | single-quote ;

symbolic-token = ? the complete maximal contiguous run of one or more symbol
                   characters after comments, literals, and closed composite
                   spellings are recognized; the run is preserved whole for
                   contextual classification and is never split as a
                   fallback ? ;
```

```
token-separator = white-space ;

line-comment = "//", { ? any Unicode scalar value except CR or LF ? } ;

block-comment = "/*", { block-comment-character }, "*/" ;

block-comment-character = ? any Unicode scalar value that does not begin the
                            two-character sequence */ ? ;

line-break = "\r\n" | "\n" | "\r" ;

horizontal-white-space = " " | "\t" | "\f" ;

white-space = horizontal-white-space | line-break | line-comment | block-comment ;

(* ========================================================================== *)
(* 13. Operator source grammar (separate start symbol)                        *)
(* ========================================================================== *)

(*
   DECISION-OPDECL-004.

   This is a SEPARATE grammar with its own start symbol. It is not reachable
   from compilation-unit and must not be entered from it. The compiler parses
   the configured operator source area with this grammar first, builds the
   operator table, and only then parses ordinary source with the main grammar.

   The operator source parser accepts one fixed source-only library declaration.
   Its body contains only operator registrations. Its lexer reads each declaration
   name as one complete maximal symbol run. The ordinary lexer later applies the
   same whole-run rule after loading the completed operator table. Neither lexer
   falls back to splitting an unknown run into shorter operators.

       arithmetic-symbol-character is any character that is not an ASCII
       letter, digit, underscore, whitespace, or one of the delimiters
       ( ) { } [ ] , ; " '

   A registration whose symbol is language-owned, whether a built-in or a
   pre-declared glyph, is rejected per DECISION-OPDECL-002. Semantic validation
   additionally requires fixity, precedence, associativity, and arity exactly
   once; optional metadata keys may occur at most once.

   operator-source-file is an INTENTIONAL SECOND ROOT. It is deliberately
   unreachable from compilation-unit and is listed as such in the validation
   manifest, alongside the informative-* companions and the lexical roots.
*)

operator-source-file = operator-library-declaration ;

operator-library-declaration = operator-library-marker, "_",
                               "co.lang.library", "=",
                               operator-library-body ;

operator-library-marker = "@co.dap.library", "(", "type", "=",
                          "operator", ")" ;

operator-library-body = "{", { operator-declaration }, body-close ;

operator-declaration = operator-symbol, "co.lang.operator", "=",
                       operator-body ;

operator-body = "{", operator-property, { ",", operator-property },
                [ "," ], "}", body-closure-guard ;

operator-property = "fixity", annotation-binder, operator-fixity
                  | "precedence", annotation-binder,
                    ( "0" | decimal-integer-literal )
                  | "associativity", annotation-binder,
                    operator-associativity
                  | "arity", annotation-binder, operator-arity
                  | "commutative", annotation-binder, boolean-literal
                  | "idempotent", annotation-binder, boolean-literal
                  | "identity", annotation-binder, operator-identity-value
                  | "foldable", annotation-binder, boolean-literal
                  | "vectorizable", annotation-binder, boolean-literal
                  | "distributes_over", annotation-binder,
                    operator-symbol-list
                  | "desugar", annotation-binder, string-literal ;

operator-fixity = "infix" | "postfix" | "prefix"
                | "circumfix" | "postcircumfix" | "precircumfix"
                | "mixfix" | "ternary" | "distfix" ;

operator-associativity = "left" | "right" | "none" ;
```

```
operator-arity = "unary" | "binary" | "ternary"
               | decimal-integer-literal ;

operator-identity-value = literal ;

operator-symbol-list = "[", [ operator-symbol-reference,
                       { ",", operator-symbol-reference }, [ "," ] ], "]" ;

operator-symbol-reference = character-literal | string-literal ;

operator-symbol = ? a maximal run of one or more symbol characters, where a
                    symbol character is any character that is not an ASCII
                    letter, digit, underscore, whitespace, or one of the
                    delimiters ( ) { } [ ] , ; " ' ; the run must not be a
                    language-owned or hard-reserved symbol and must not contain
                    // or /* ? ;
```

Appendix B - Grammar Decisions and Rationale

FoLang Grammar and Semantic Decision Register — Revision 24

- Grammar: `folang.ebnf`
- Grammar SHA-256: `12e1673fb7d2624ef4dad405a6859a1dd42952ec2d092db2ebcdb284469c30fc`
- Language reference basis: `../language-ref.md`
- Language reference SHA-256: `6aac6a6e782197c0da5da206209de695443434ca3a42dd6cf18ec014df638ea7`
- Status: decision-complete grammar and semantic register aligned with the current language reference
- Planned syntax policy: productions described as planned in `../language-ref.md` remain in the complete grammar unless explicitly removed. Release-specific availability is handled by the parser/compiler conformance profile.
- Revision 24 adds the whole-symbol-run and operator-boundary model. A contiguous symbolic run is preserved as one candidate and is never split into shorter operators as a fallback. Grammar context distinguishes structural spellings, metadata spellings, and registered expression operators. Every multi-symbol expression operator requires explicit operand-facing boundaries. `++` and `--` are removed from the built-in prefix/postfix grammar; when unregistered they fail through the general symbolic-run rule.

Termination model

FoLang distinguishes **body braces** from **expression braces** by the enclosing production. The brace character alone does not determine termination.

```
emp := Employee{ id: 1, name: "Rao" };  // object construction expression: ; required

classify(n) => {                        // function-pattern body: no ; after }
    this.return "positive";
}

someFArg co.lang.function =             // inline function-kind body: no ; after }
    (a co.lang.int, b co.lang.int)->(co.lang.int) = {
        this.return a + b;
    }

Employee co.lang.struct = {             // UDT body: no ; after }
    id co.lang.int;
    name co.lang.string;
}
```

A comma is a soft end inside an enum or other grouped construct. It closes the current item but not the enclosing statement. Built-in directives are self-delimiting and take no semicolon.

Physical nesting and scope model

FoLang distinguishes an independently named declaration from a construct that is merely nested syntactically.

```
outer()->() = {
    value co.lang.int = 10;

    inner()->() = {                         // named local function: permitted
        co.out.println(value);
    }

    operation := (x co.lang.int)->(co.lang.int) = {
        this.return x * 2;                  // anonymous function expression
    };

    worker := co.lang.class {               // anonymous class expression
        run(x co.lang.int)->(co.lang.int) = {
            this.return operation(x);
        }
    }.init();
}

transformer co.lang.type = forall(T).(T)->(T); // anonymous type expression
```

Independent package-owned classes, structs, cstructs, enums, unions, modules, units, interfaces, signatures, type declarations, instances, matchers, macros, templates, and similar primary declarations cannot be physically nested. Member methods and module/signature type components are members or contract slots rather than independent nested package declarations.

An ordinary local function is physically declared in its enclosing executable block and uses declaration-site lexical scope. `@co.dap.inner` is different: it annotates a separately declared association and executable declarations use call-site lexical-context resolution as defined by `DECISION-SCOPE-002`.

Package-level function envelope

Ordinary loose functions are forbidden in package source files. The `annotated-function-primary` production exists for annotation-defined primary declaration kinds. Parsing that envelope does not establish legality; semantic analysis must confirm that a resolved annotation explicitly grants primary-declaration status.

Operator bootstrap and artifact model

FoLang separates operator registration from operator implementation.

```
language-owned built-in or pre-declared symbol
    -> symbol and parse properties already registered by the language

project-local custom symbol
    -> registered once inside the fixed operators.fol operator library
    -> carries parse and optional optimization metadata

all registered symbols
    -> implementations use mode=overload in a legal function owner
    -> multiple distinct normalized operand signatures are permitted
    -> mode=override and mode=extends are unsupported
```

`operator_library_folder` in `fol-conf.yaml` identifies a source-only bootstrap area excluded from package discovery. Its fixed `operators.fol` is parsed by a dedicated lexer/parser and must have this outer shape:

```
@co.dap.library(type=operator)
_ co.lang.library = {
    // co.lang.operator declarations only
}
```

The operator library is not imported and produces no artifact. Operators do not cross ordinary library boundaries. Both lexers preserve one complete contiguous symbol run and never fall back to shorter operators. Grammar context classifies the whole spelling as structural syntax, contextual metadata, a registered expression operator, or an unrecognized token. A multi-symbol expression operator requires explicit boundaries on every operand-facing side, so a registered `+-` is written `a +- b`; `a + -b` denotes separate operators.

Operator implementations are normal functions and cannot be loose at package scope:

- built-in operand type: `@co.dap.extension` function inside a unit;
- struct: same-package companion unit;
- class: class operator method;
- module, enum, union, interface, signature, and cstruct: unsupported.

A duplicate custom symbol declaration is an error. A duplicate normalized implementation signature is also an error, but one registered custom symbol may have multiple distinct overload signatures. Using one symbol consistently for one concept is recommended for readability and is not compiler-enforced.

Pre-declared mathematical/modifier glyphs are language-owned registrations with no required built-in implementation. They cannot be declared locally, but they can be implemented through `mode=overload` exactly like `+` or `*`.

Decision index

Decision	Status	Normative decision
`DECISION-ANN-001`	Active	Annotation arguments and annotation-map entries accept `=` or `:` as binders. A bare key is a flag. Mixed binders are permitted within one annotation.
`DECISION-BACKEND-001`	Active	Each resolved user-defined FoLang identifier is lowered to C++ by appending `_fo`. Built-ins, keywords, and compiler-generated names use separate compiler-defined lowering.
`DECISION-BLK-001`	Active	A block may end with one unterminated tail expression. That final expression is the block value and is not an expression statement.
`DECISION-COL-001`	Active	Commas separate enum variants, map entries, annotation-map entries, object initializers, parameters, arguments, and other grouped items. A comma is a soft end inside the enclosing construct; a permitted trailing comma does not terminate the enclosing statement. Object and annotation-map fields use `:`.
`DECISION-DIR-001`	Active	Built-in directives are self-delimiting. Their complete directive form ends the directive; no trailing semicolon is accepted or required.
`DECISION-EXT-001`	Active	The contextual registered precedence table is built from the language-owned operator registrations plus the current compilation's custom declarations from the configured operator library. Built-in, pre-declared, and custom symbols all receive implementations through `mode=overload`; omitted mode defaults to overload. `mode=override`, `mode=extends`, `mode=define`, and other explicit modes are rejected. A custom declaration supplies complete parse metadata. The alpha profile implements infix, prefix, and postfix; other fixities remain reserved until their delimiter/slot grammars are defined.
`DECISION-FUN-001`	Active	The `=` before a function block body is optional. Both `f()->T = { ... }` and `f()->T { ... }` are valid.
`DECISION-FUN-002`	Active	A named closure uses `name = (parameters) ==>> expression;`. Additional adjacent parameter lists make it curried: `name = (first)(second) ==>> expression;`.
`DECISION-GEN-001`	Active	Generic parameters may declare arity, including higher-kinded forms such as `Transformer(F(_), G(_))`. An arity slot is `_` or a named placeholder. Supported named type/container declarations retain their complete generic parameter clause; application-entry declarations, library declarations, and package aliases do not accept one.
`DECISION-KIND-001`	Active	A built-in kind without a dedicated production is parsed by `general-kind-declaration`, which supports block, type-expression, expression, and forward forms. Dedicated declarations and ordinary variable declarations retain priority in their contexts.
`DECISION-LEX-001`	Active	Source is UTF-8, but ordinary identifiers use ASCII letters, digits, and isolated internal underscores. An identifier begins with an ASCII letter, cannot contain consecutive underscores, cannot end in an underscore, and has no minimum-length requirement. Lone `_` is contextual, not an identifier.
`DECISION-LEX-002`	Active	FoLang supports `//` line comments and non-nesting `/* ... */` block comments. Line breaks are whitespace outside literals.

`DECISION-LEX-003`	Active	After comments, literals, and closed scanner-known composite spellings such as `@@new` are recognized, the lexer consumes each remaining complete maximal contiguous run of symbol characters as one candidate. The whole run is classified by grammar context as a fixed structural spelling, contextual metadata spelling, registered expression operator, or unrecognized symbolic token. It is never split into shorter operators as a fallback. Comment openers are recognized before symbolic-run scanning.
`DECISION-LEX-010`	Active	A registered expression operator containing more than one symbol character requires an explicit boundary on every operand-facing side. Whitespace, comments, and applicable delimiters supply boundaries, and boundary presence is checked before separators are discarded. Infix operators require both sides, prefix operators the operand side after the symbol, and postfix operators the operand side before it. Structural and metadata spellings are exempt unless parsed as expression operators.
`DECISION-LEX-005`	Withdrawn in revision 9	The special numeric dot-scanning rule was removed after abbreviated floating forms such as `1.` and `.10` were rejected. Numeric-literal recognition and the current whole-symbol-run classifier now distinguish decimal points, range spellings, and member access without that rule.
`DECISION-LEX-006`	Active	After recognizing an identifier, the scanner verifies that the next character is not `_`. This converts trailing or doubled underscores into lexical errors instead of silently splitting them into multiple tokens.
`DECISION-LEX-007`	Withdrawn in revision 11	The apostrophe digit-separator adjacency rule was removed because FoLang no longer supports numeric digit separators.
`DECISION-LEX-008`	Active	Adjacent encoding prefixes, raw-string introducers, and backslashes in quoted literals begin reserved post-alpha spellings. The scanner consumes the complete spelling and reports an unsupported feature; separated names remain identifiers.
`DECISION-LEX-009`	Active	A special method is one complete spelling from a closed scanner-known set. `@@` is not a prefix operator over an arbitrary identifier: spellings such as `@@new` and `@@init` are classified as whole tokens, and any unrecognized `@@` spelling is a lexical error that reports the admissible set.
`DECISION-LIT-000`	Active	FoLang accepts a selected C++-compatible subset of numeric, character, and string literal spellings. The frontend preserves their complete raw lexemes, and a C++ backend may emit those lexemes unchanged. `co.const.true`, `co.const.false`, and `co.const.none` use backend-defined lowering. `nullptr` is not introduced.
`DECISION-LIT-001`	Active	Integer literals support C++-compatible binary, leading-zero octal, decimal, hexadecimal, and standard integer suffix forms. Numeric digit separators are not supported.
`DECISION-LIT-002`	Active	Floating literals support selected C++-compatible decimal and hexadecimal forms, exponents, and configured suffixes. Digit separators are not supported, and a decimal point requires a digit on both sides.
`DECISION-LIT-003`	Active	The alpha release accepts only unprefixed character and string literals without escapes. Prefixes, escapes, universal character names, and raw strings are reserved and rejected with unsupported-feature diagnostics.

`DECISION-LIT-004`	Withdrawn in revision 7	FoLang has no separate user-defined-literal token or C++ `operator""` mechanism. A value of a user-defined type is created through ordinary object construction.
`DECISION-LIT-005`	Active	Boolean literals are `co.const.true` and `co.const.false`; none/null is `co.const.none`. Bare `true`, `false`, and `none` are not FoLang literals.
`DECISION-LIT-006`	Active	A floating literal has at least one digit on each side of its decimal point. Write `1.0`, `0.10`, and `0x1.8p3`; forms such as `1.`, `.10`, `0x1.p3`, and `0x.8p3` are rejected.
`DECISION-LIT-007`	Active	FoLang does not adopt C++14 apostrophe digit separators or underscore digit separators. Numeric digit sequences contain digits only.
`DECISION-OP-001`	Active	Built-in operators use the precedence table encoded in the grammar. Runtime assignment has the lowest built-in precedence.
`DECISION-OP-002`	Active	Runtime assignments are right-associative, so `a = b = c` groups as `a = (b = c)`. Assignment expressions yield the assigned value; FoLang evaluation-order rules remain separate.
`DECISION-OP-003`	Active	`:=` and `?=` are statement-level definition operators, not general expression operators, and cannot be chained. Each requires an explicit boundary on both the name-facing and value-facing side. `::=` remains reserved.
`DECISION-OP-004`	Withdrawn in revision 24	Earlier drafts recognized `++` and `--` as built-in prefix and postfix operators. They are removed from the operator inventory, precedence table, and grammar. There is no special rejection rule: an unregistered contiguous `++` or `--` spelling is rejected by the general whole-symbol-run rule in `DECISION-LEX-003`, while separate operators must be written with a boundary, such as `+ +a` or `+ (+a)`.
`DECISION-OP-005`	Active	`::=`, `->>`, `<->`, backtick, backslash, `#`, and comment openers remain hard-reserved and cannot be declared or overloaded. The documented mathematical/modifier glyph set is not in this rejected category: those glyphs are pre-declared language-owned operators and may receive `mode=overload` implementations.
`DECISION-OP-006`	Active	`~`, `#`, and `^` are reserved prefix spellings and are rejected in operand-prefix position. `#` has no semantics and cannot be used, defined, overloaded, or overridden. `^` remains the built-in infix XOR symbol and `~` retains its named-parameter and `-> (~)` roles. `@` is not a prefix expression operator.
`DECISION-OP-007`	Active	A constant expression may use a registered custom operator at any declared precedence, but runtime assignment is recursively forbidden throughout the complete constant-expression subtree. Grouping, call arguments, collection/object elements, and other nested expression forms cannot hide an assignment from this guard. Constant evaluation and custom-operator foldability remain semantic checks after parsing.

`DECISION-OPLIB-001`	Active	Operator `mode=overload` is supported for every registered symbol—built-in, pre-declared, or project-local custom. Because operator implementations are normal functions, they cannot be declared loose at package scope: built-in operands use an `@co.dap.extension` function inside a unit, structs use their same-package companion unit, and classes use class operator methods. Receiver normalization determines the complete operand signature and its count must match the registered arity. Equivalent normalized signatures are duplicates; distinct signatures are overloads. Modules, enums, unions, interfaces, signatures, and cstructs cannot own operator implementations. Operator `mode=override` and `mode=extends` are unsupported; ordinary class-method overriding through `@co.dap.override` remains separate.
`DECISION-OPLIB-002`	Withdrawn in revision 17	The separately distributable `operator` library-kind model was removed. `type=operator` remains only as the source marker for the configured project-local bootstrap surface and never produces an independently imported artifact.
`DECISION-OPLIB-003`	Active	A project-local custom operator symbol has exactly one registration in the compilation's fixed operator library. Duplicate declarations, aliases, merges, selection, and remapping are rejected. The registered symbol may have zero or more implementation overloads in legal owners; each distinct normalized operand signature is permitted and an equivalent signature is a duplicate error. Language-owned built-in and pre-declared symbols cannot be registered locally and are implemented through the same overload mechanism.
`DECISION-OPLIB-004`	Withdrawn in revision 17	Operator-specific imports and activation were removed. Historical operator-specific/imported metadata activation was removed; ordinary imports do not supply operator metadata.
`DECISION-OPLIB-005`	Withdrawn in revision 17	Historical operator manifest/export models were removed; no operator table is embedded in an ordinary library artifact.
`DECISION-OPLIB-006`	Withdrawn in revision 17	Historical imported operator-table bootstrap was removed. Only language-owned registrations and the current compilation's local operator library build the contextual operator table.
`DECISION-OPBOOT-001`	Active	`operator_library_folder` in `fol-conf.yaml` identifies the project-local operator source area. A relative path is resolved from the project root. The compiler checks the fixed file `<operator_library_folder>/operators.fol`; absent configuration, folder, or file means no local new operators. The area is excluded from ordinary package discovery.
`DECISION-OPBOOT-002`	Active	The configured `operators.fol` is parsed first by a dedicated operator-source lexer and parser. It must contain exactly `@co.dap.library(type=operator)` followed by `_ co.lang.library = { ... }`, whose body contains only `co.lang.operator` declarations. It is a source-only bootstrap surface compiled into its owning project, produces no independent artifact, and is not ordinary FoLang/package source.

`DECISION-OPART-001`	Withdrawn in revision 22	Library operator export was removed. An operator implementation is bound to a class, a companion unit, or an extension unit, and `library-member` admits only import directives, struct declarations, cstruct declarations, and function declarations. No production can therefore place an operator in a library surface file, and no operator table is emitted into a `.folib` or `.folenc` artifact.
`DECISION-OPART-002`	Withdrawn in revision 22	Operator-table loading on import was removed. A direct library import loads the projected symbol table only. An importer that wants operator notation for an imported boundary struct declares its own symbol in its own operator source area and implements it in an extension unit.
`DECISION-OPART-003`	Withdrawn in revision 22	The exported operator entry schema was removed with `DECISION-OPART-001`. Nothing operator-related is serialized into a library artifact.
`DECISION-OPBOOT-003`	Active	The frontend reads configuration, parses and validates the local operator library, combines its custom registrations with the language-owned built-in and pre-declared registrations, and builds immutable symbol/fixity/precedence tables before ordinary source tokenization. Imports contribute no operator metadata. Both lexers preserve a complete contiguous symbol run and perform exact whole-run classification without fallback splitting. The parse is single-pass and import-order independent.
`DECISION-OPDECL-001`	Active	A new custom symbol is registered only by `SYMBOL co.lang.operator = { ... }` inside the fixed `_ co.lang.library` operator-source body. The declaration carries required `fixity`, `precedence`, `associativity`, and `arity`, plus the optional `commutative`, `idempotent`, `identity`, `foldable`, `vectorizable`, `distributes_over`, and `desugar` metadata. Required properties occur exactly once; optional properties occur at most once; duplicates, omissions, unknown keys, invalid types, and invalid ranges are errors. `@co.dap.operator` at implementation sites carries only `symbol` and optional `mode`.
`DECISION-OPDECL-002`	Active	The documented mathematical/modifier glyph set is pre-declared by the language with symbol, fixity, precedence, associativity, and arity but no required implementation. These glyphs are language-owned registrations: they parse everywhere, cannot be redeclared in the local operator library, and may receive any number of distinct `mode=overload` implementation signatures under `DECISION-OPLIB-001`. A missing matching implementation is a resolution error.
`DECISION-OPDECL-003`	Active	Operator symbols are global to a compilation while implementation availability is resolved by owner, scope, activation, operand types, and normalized signature. The tokenizer recognizes each registered symbol throughout the compilation; an unavailable implementation therefore produces a resolution diagnostic rather than a syntax error. Using one symbol for one recognizable concept across overloads is recommended for readability but is not a compiler-enforced rule.

`DECISION-OPDECL-004`	Active	The operator source area is parsed by a separate grammar rooted at `operator-source-file`. The exact outer declaration is `@co.dap.library(type=operator) _ co.lang.library = { ... }`, and the body admits only operator registrations. `co.lang.operator` is absent from ordinary declaration kinds. The dedicated and ordinary lexers preserve complete contiguous symbol runs; a custom symbol is recognized only by an exact whole-run match and an unknown run is never split into shorter operators. Symbols containing `//` or `/*` are rejected because a comment opener terminates the preceding symbolic run and takes lexical priority.
`DECISION-SCOPE-001`	Active	An ordinary local/inner function has block-local identity and resolves free runtime names from its lexical declaration context. Calling it does not replace that environment with the caller's runtime scope.
`DECISION-SCOPE-002`	Active	`@co.dap.inner` is an association annotation on a separately declared declaration, not physical nesting. Executable inner-associated declarations resolve free runtime names through the lexical scope chain of the active attachment/call site, without unrestricted runtime caller-chain lookup. Compile-time names remain statically resolved.
`DECISION-SEM-001`	Active	Typeclass instances are selected explicitly by name; FoLang performs no implicit instance search. An instance must be declared in the exact package defining either the typeclass or the represented type. Selection and placement are semantic name-resolution checks.
`DECISION-SEM-002`	Active	`@co.ddap.use` explicitly and block-scopingly activates extension-unit or typeclass-instance functions as methods. Receiver-owned declarations take priority, followed by activated extensions and then activated instances. A method name may be activated at most once per receiver type in one scope. Parser `CallKind` values remain provisional; any early control-chain lowering must preserve the original member-call chain so resolution can select an overriding user declaration without reconstructing lost syntax. Contextual lambda/wildcard admission requires a receiver-qualified method and is unchanged by transparent grouping of that member callee.
`DECISION-SYN-001`	Active	Every simple statement whose production uses `statement-end` requires `;`. Newlines never terminate statements and FoLang performs no semicolon insertion. Built-in directives are exceptions because they are self-delimiting.
`DECISION-SYN-002`	Active	Comma-separated variable declarators form one declaration statement and share one final semicolon.
`DECISION-SYN-003`	Active	The sole named local-function syntax requires a return-type clause and a block body. This preserves `foo();` as an expression statement. The local function has block-local identity and declaration-site lexical scope.
`DECISION-SYN-004`	Active	Annotations may prefix an expression statement.
`DECISION-SYN-005`	Active	A standalone block is a statement and takes no trailing semicolon.

`DECISION-SYN-006`	Active	Termination depends on syntactic role, not merely on the final character. `;` ends simple, expression-bodied, type-bodied, and forward forms. A body-selected `}` ends UDT/container bodies, function bodies, function-pattern bodies, and standalone block statements. A brace that closes object construction, a map, an anonymous class, or another braced expression does not end the enclosing statement.
`DECISION-SYN-007`	Active	Body-versus-expression selection is explicit. Direct body branches use `body-closure-guard`, which rejects an immediately following semicolon. Competing expression branches use `non-block-expression` or `non-anonymous-function-expression`, preventing a body from being reparsed as an expression plus `;`. Grouped, postfixed, or otherwise composed braced forms remain expressions.
`DECISION-SYN-008`	Active	Independent named type and container declarations cannot be physically nested. Ordinary named local functions are the explicit named exception. Anonymous functions, lambdas/callback blocks, anonymous classes, ordinary value expressions, and `forall` type expressions may be nested wherever their expression/type-expression grammar permits and create no package-level declaration identity.
`DECISION-SYN-009`	Active	`annotated-function-primary` is only a syntactic envelope for annotation-defined primary declaration kinds. An arbitrary annotation does not legalize a loose ordinary function at package-file scope; that legality is checked semantically after annotation resolution.
`DECISION-TYP-001`	Active	Every type derivation may carry a trailing attribute list, not only pointer derivations.
`DECISION-TYP-002`	Active	A function-shaped type constructor has exactly one type-producing result. Its result may be one of `co.lang.dependentType`, `co.lang.type`, `co.lang.typetype`, `co.lang.typekind`, or `co.lang.kind`, or one union joined by `|`; commas and named/multiple result items are rejected. Its body may bind a type expression, with the type-expression reading taking priority. `co.lang.dependentType`, `co.lang.typetype`, and `co.lang.typekind` are also direct `type-declaration-kind` alternatives and are excluded from general-kind routing.
`DECISION-TYP-003`	Active	An array dimension may be elided in any position, including `->([])` and `->([,])`.
`DECISION-TYP-004`	Active	A dependent-type argument and an array dimension are index positions, not general expressions. An index is a non-negative integer literal or a name resolving to an in-scope parameter or `@co.dap.const` compile-time constant. Arithmetic, calls, indexing, and other operators are rejected.
`DECISION-TYP-005`	Active	Dependent types are equal when their constructors match and indices are pairwise equal. Literal and substituted `@co.dap.const` indices compare by value; parameter indices compare by declaration identity. Equality is not decided modulo arithmetic.
`DECISION-TYP-006`	Active	Dependent types are checked against written signatures and are never inferred. FoLang does not infer index values or perform whole-program dependent-type inference.

Current lexical examples

```
Valid identifiers:   a, x, id, name, myVar2, v1_hr, a_b_c
Invalid identifiers: _x, _1, a_, a__b
Contextual token:    _

Valid numbers:       1000, 0b11110000, 0xFFFF0000, 3.141592
Invalid numbers:     1_000, 1'000, 1., .10
```

Planned syntax retention

Package aliasing, comprehensions, and other planned constructs remain in the complete grammar. Their availability in alpha, 0.x, 1.0, or later compiler profiles is a version-conformance decision, not a reason to delete their productions from the complete grammar.